

Deep Learning and Generative AI

Complete Study Guide for Final Exam

A Beginner-Friendly Comprehensive Guide

ePGD Program

IIT Bombay - Center for Machine Intelligence and Data Science

May 20, 2026

Preface

This study guide is designed for students who are encountering Deep Learning and Generative AI for the first time. We assume you have limited background in linear algebra and calculus, so we will explain every mathematical concept, formula, and notation as we go.

How to Use This Guide

- **Study Time:** This guide is designed for approximately 10 hours of focused study.
- **Mathematical Explanations:** Every formula is explained step-by-step with all notation clearly defined.
- **Examples:** Multiple worked examples are provided to build intuition.
- **Diagrams:** Visual representations help understand complex concepts.
- **Progressive Learning:** Topics build on each other, so follow the sequence.

What You'll Learn

1. **Part I:** Neural Network Fundamentals and Computer Vision
2. **Part II:** Sequence Models and Natural Language Processing
3. **Part III:** Attention Mechanisms and Transformers
4. **Part IV:** Modern Large Language Models
5. **Part V:** Prompting and Instruction Tuning
6. **Part VI:** Reinforcement Learning from Human Feedback
7. **Part VII:** Agentic AI Systems

Contents

Preface	iii
I Neural Network Foundations and Computer Vision	1
1 Introduction to Neural Networks	3
1.1 What is a Neural Network?	3
1.1.1 Motivation: Why Do We Need Neural Networks?	3
1.1.2 The Biological Inspiration	3
1.2 The Perceptron: The Simplest Neural Network	4
1.2.1 What is a Perceptron?	4
1.2.2 Mathematical Formulation	4
1.2.3 Complete Perceptron Formula	7
1.2.4 Vector Notation (Optional but Useful)	7
1.3 Learning: How Does a Perceptron Get Better?	8
1.3.1 The Learning Problem	8
1.3.2 Supervised Learning	8
1.3.3 The Perceptron Learning Algorithm	8
1.3.4 Why Does This Work? Geometric Intuition	10
1.4 Limitations of the Perceptron	11
1.4.1 The XOR Problem	11
1.4.2 From Perceptron to Multi-Layer Networks	12
2 Multi-Layer Neural Networks and Backpropagation	13
2.1 Architecture of Multi-Layer Networks	13
2.1.1 Layers and Connections	13
2.1.2 Fully Connected (Dense) Layers	13
2.1.3 Forward Propagation: Computing the Output	14
2.2 Activation Functions	15
2.2.1 Why Do We Need Activation Functions?	15
2.2.2 Common Activation Functions	15
2.3 Training Multi-Layer Networks: Backpropagation	18
2.3.1 The Challenge: How to Update Hidden Layer Weights	18
2.3.2 Loss Functions	18
2.3.3 Gradient Descent: The Optimization Algorithm	19
2.3.4 Backpropagation: Computing Gradients Efficiently	21
2.3.5 Mini-Batch Gradient Descent	25
2.4 Universal Approximation Theorem	25
2.5 Practice Problems: Chapters 1-2	25
2.5.1 Problem 1: Perceptron Calculation	25
2.5.2 Problem 2: Sigmoid Derivative	26

2.5.3	Problem 3: Backpropagation Gradient	26
2.5.4	Problem 4: ReLU vs Sigmoid	27
2.5.5	Problem 5: Binary Cross-Entropy Loss	28
2.6	Summary of Chapter 2	28
3	Convolutional Neural Networks for Computer Vision	31
3.1	Motivation: Why Special Networks for Images?	31
3.1.1	The Problem with Fully Connected Networks for Images	31
3.1.2	Key Ideas Behind CNNs	31
3.2	Convolution Operation	32
3.2.1	What is Convolution?	32
3.2.2	Mathematical Definition	32
3.2.3	Receptive Field	34
3.3	Building Blocks of CNNs	34
3.3.1	Convolutional Layer	34
3.3.2	Stride	34
3.3.3	Padding	35
3.3.4	Output Size Formula	35
3.3.5	Pooling Layers	36
3.3.6	Typical CNN Architecture Pattern	37
3.4	Famous CNN Architectures	38
3.4.1	LeNet-5 (1998)	38
3.4.2	AlexNet (2012)	38
3.4.3	VGGNet (2014)	39
3.4.4	ResNet (2015)	39
3.4.5	Other Notable Architectures	40
3.5	Regularization Techniques for Deep Networks	40
3.5.1	The Overfitting Problem	40
3.5.2	Dropout	40
3.5.3	Batch Normalization	42
3.5.4	Layer Normalization	44
3.6	Autoencoders	44
3.6.1	What is an Autoencoder?	44
3.6.2	Mathematical Formulation	45
3.6.3	Convolutional Autoencoders	46
3.6.4	Regularized Autoencoders	46
3.6.5	Class Autoencoder	47
3.6.6	Application: Image Segmentation with Autoencoders	48
3.7	Practice Problems: Chapter 3	49
3.7.1	Problem 1: Convolution Output Size	49
3.7.2	Problem 2: Max Pooling Operation	49
3.7.3	Problem 3: Dropout During Training	50
3.7.4	Problem 4: Batch Normalization	51
3.7.5	Problem 5: Autoencoder Reconstruction	51
3.7.6	Problem 6: CNN Parameter Counting	52
3.8	Summary of Chapter 3	53
II	Sequence Models and Natural Language Processing	55
4	Recurrent Neural Networks and Sequence Modeling	57
4.1	Motivation: Sequential Data	57

4.1.1	What is Sequential Data?	57
4.1.2	Why CNNs Don't Work Well for Sequences	57
4.2	Traditional Approach: N-gram Language Models	58
4.2.1	What is a Language Model?	58
4.2.2	N-gram Models	58
4.2.3	Estimating N-gram Probabilities	58
4.2.4	Perplexity	60
4.2.5	Problems with N-gram Models	60
4.2.6	Smoothing Techniques	61
4.3	Recurrent Neural Networks (RNNs)	63
4.3.1	The Core Idea: Recurrence	63
4.3.2	RNN Mathematical Formulation	63
4.3.3	Unrolling an RNN Through Time	64
4.3.4	Types of RNN Architectures	66
4.4	Training RNNs: Backpropagation Through Time (BPTT)	66
4.4.1	The Challenge	66
4.4.2	The Vanishing Gradient Problem	67
4.4.3	The Exploding Gradient Problem	67
4.5	Long Short-Term Memory (LSTM)	68
4.5.1	LSTM Architecture	68
4.5.2	LSTM Equations	68
4.5.3	Why LSTM Solves Vanishing Gradients	69
4.5.4	Deep LSTM	69
4.6	Gated Recurrent Unit (GRU)	71
4.6.1	GRU Equations	71
4.7	Bidirectional RNNs	71
4.7.1	Bidirectional RNN Architecture	71
4.8	Practice Problems: Chapter 4	72
4.8.1	Problem 1: Bigram Probability Calculation	72
4.8.2	Problem 2: Perplexity Calculation	73
4.8.3	Problem 3: Add-k Smoothing	74
4.8.4	Problem 4: LSTM Forward Pass	74
4.8.5	Problem 5: Vanishing Gradient in RNN	75
4.8.6	Problem 6: Sequence Probability with RNN	76
4.9	Summary of Chapter 4	77
5	Attention Mechanisms	79
5.1	Motivation: The Bottleneck Problem	79
5.1.1	Seq2Seq Models	79
5.2	Attention Mechanism (Bahdanau Attention)	79
5.2.1	The Core Idea	79
5.2.2	Attention Mathematical Formulation	80
5.2.3	Attention Visualization	81
5.3	Self-Attention	82
5.3.1	From Encoder-Decoder Attention to Self-Attention	82
5.3.2	Self-Attention Mechanism	82
5.3.3	Scaled Dot-Product Attention	83
5.3.4	Why Scaling by $\sqrt{d_k}$?	85
5.4	Multi-Head Attention	85
5.4.1	Multi-Head Attention Formula	85
5.5	Practice Problems: Chapter 5	86

5.5.1	Problem 1: Attention Score Calculation	86
5.5.2	Problem 2: Context Vector Computation	87
5.5.3	Problem 3: Scaled Dot-Product Attention	88
5.5.4	Problem 4: Multi-Head Attention Dimensions	89
5.5.5	Problem 5: Self-Attention vs Cross-Attention	89
5.6	Summary of Chapter 5	90
6	The Transformer Architecture	93
6.1	Motivation: Getting Rid of Recurrence	93
6.1.1	Limitations of RNNs	93
6.2	Transformer Architecture Overview	93
6.3	Positional Encoding	94
6.3.1	The Problem: No Sequential Information	94
6.3.2	The Solution: Positional Encodings	94
6.4	Encoder Layer	96
6.4.1	1. Multi-Head Self-Attention	96
6.4.2	2. Position-wise Feed-Forward Network	96
6.4.3	Residual Connections and Layer Normalization	96
6.5	Decoder Layer	97
6.5.1	1. Masked Multi-Head Self-Attention	97
6.5.2	2. Encoder-Decoder Attention	97
6.5.3	3. Position-wise Feed-Forward Network	98
6.6	Complete Transformer Forward Pass	98
6.7	Training the Transformer	99
6.7.1	Teacher Forcing	99
6.7.2	Loss Function	99
6.8	Why Transformers Work So Well	99
6.9	Transformer Hyperparameters	100
6.10	Practice Problems: Chapter 6	100
6.10.1	Problem 1: Positional Encoding	100
6.10.2	Problem 2: Feed-Forward Network Computation	101
6.10.3	Problem 3: Masked Self-Attention	102
6.10.4	Problem 4: Transformer Parameter Count	103
6.10.5	Problem 5: Comparing Transformer to RNN Complexity	104
6.11	Summary of Chapter 6	106
III	Modern Large Language Models and Generative AI	107
7	Modern Language Model Architectures	109
7.1	From Transformers to Large Language Models	109
7.1.1	What is a Language Model?	109
7.1.2	Pre-training and Fine-tuning Paradigm	109
7.2	BERT: Bidirectional Encoder Representations from Transformers	110
7.2.1	Key Idea	110
7.2.2	BERT Pre-training Objectives	110
7.2.3	BERT Architecture Details	111
7.2.4	Fine-tuning BERT	111
7.3	GPT: Generative Pre-trained Transformer	111
7.3.1	Key Idea	111
7.3.2	GPT Training Objective	112
7.3.3	GPT Architecture	112

7.3.4	GPT Evolution	112
7.4	T5: Text-to-Text Transfer Transformer	112
7.4.1	Unified Text-to-Text Framework	112
7.4.2	Architecture	113
7.4.3	Pre-training	113
7.5	BART: Bidirectional and Autoregressive Transformer	113
7.5.1	Combining BERT and GPT	113
7.5.2	Denoising Pre-training	113
7.6	Retrieval-Augmented Generation (RAG)	113
7.6.1	The Problem: Hallucination	113
7.6.2	RAG Solution	114
7.6.3	RAG Benefits	114
7.7	Decoding Strategies	114
7.7.1	Greedy Decoding	114
7.7.2	Beam Search	115
7.7.3	Sampling Methods	115
7.8	Practice Problems: Chapter 7	116
7.8.1	Problem 1: BERT Masked Language Modeling	116
7.8.2	Problem 2: GPT Autoregressive Generation	118
7.8.3	Problem 3: Temperature Sampling	118
7.8.4	Problem 4: Top-k Sampling	120
7.8.5	Problem 5: Nucleus (Top-p) Sampling	121
7.8.6	Problem 6: BERT vs GPT Architecture	121
7.9	Summary of Chapter 7	122
8	Prompting and Instruction Tuning	125
8.1	The Power of Prompting	125
8.1.1	What is Prompting?	125
8.2	Types of Prompting	125
8.2.1	Zero-Shot Prompting	125
8.2.2	Few-Shot Prompting	125
8.2.3	One-Shot Prompting	126
8.3	Chain-of-Thought (CoT) Prompting	126
8.3.1	The Idea	126
8.3.2	Zero-Shot CoT	126
8.3.3	Few-Shot CoT	126
8.4	Advanced Prompting Techniques	127
8.4.1	Self-Consistency	127
8.4.2	Tree of Thoughts	127
8.5	Instruction Tuning	128
8.5.1	The Problem with Base LLMs	128
8.5.2	Instruction Tuning Approach	128
8.5.3	Instruction Datasets	128
8.6	Evol-Instruct	129
8.6.1	Evolving Instructions for Complexity	129
8.6.2	Evol-Instruct Algorithm	129
8.7	Orca: Progressive Learning from GPT-4	129
8.7.1	Learning from Reasoning Traces	129
8.7.2	Explanation Tuning	129
8.7.3	Orca Training Process	130
8.8	Prompt Engineering Best Practices	130

8.8.1	General Principles	130
8.8.2	Common Pitfalls	131
8.9	Practice Problems: Chapter 8	131
8.9.1	Problem 1: Zero-shot vs Few-shot Prompting	131
8.9.2	Problem 2: Chain-of-Thought Reasoning	132
8.9.3	Problem 3: Self-Consistency with CoT	134
8.9.4	Problem 4: Prompt Engineering Evaluation	135
8.9.5	Problem 5: ReAct (Reasoning + Acting)	136
8.10	Summary of Chapter 8	138
9	Reinforcement Learning from Human Feedback (RLHF)	139
9.1	Motivation: Aligning AI with Human Values	139
9.1.1	The Alignment Problem	139
9.2	The RLHF Pipeline	139
9.3	Stage 1: Supervised Fine-Tuning (SFT)	140
9.3.1	Creating the Demonstration Dataset	140
9.3.2	SFT Training	140
9.4	Stage 2: Reward Modeling	141
9.4.1	The Challenge	141
9.4.2	Collecting Comparison Data	141
9.4.3	Training the Reward Model	141
9.4.4	Reward Model Architecture	142
9.5	Reinforcement Learning Fundamentals	143
9.5.1	RL Components for Text Generation	143
9.5.2	Policy	143
9.5.3	Value Functions	143
9.5.4	REINFORCE Algorithm	145
9.5.5	Importance Sampling	147
9.6	Stage 3: Reinforcement Learning with PPO	148
9.6.1	RL Framework for LLMs	148
9.6.2	The Objective Function	149
9.6.3	Proximal Policy Optimization (PPO)	149
9.6.4	RLHF Training Loop	150
9.7	Constitutional AI	150
9.7.1	The Idea	150
9.7.2	The Process	150
9.8	Challenges and Limitations of RLHF	151
9.8.1	Reward Model Limitations	151
9.8.2	Alignment Tax	151
9.8.3	Value Alignment	152
9.9	Practice Problems: Chapter 9	152
9.9.1	Problem 1: Reward Model Training	152
9.9.2	Problem 2: PPO Policy Gradient	153
9.9.3	Problem 3: KL Divergence Penalty	154
9.9.4	Problem 4: Constitutional AI Critique	155
9.9.5	Problem 5: RLHF Pipeline Stages	157
9.10	Summary of Chapter 9	159

10	Agentic AI Systems	161
10.1	From Language Models to Agents	161
10.1.1	What is an AI Agent?	161
10.1.2	Agent Architecture	162
10.2	ReAct: Reasoning + Acting	162
10.2.1	The Core Idea	162
10.2.2	ReAct Format	162
10.2.3	Available Actions	162
10.2.4	Benefits	163
10.3	Reflexion: Learning from Mistakes	163
10.3.1	The Idea	163
10.3.2	Reflexion Process	163
10.3.3	Benefits	163
10.4	Self-Refine: Iterative Self-Improvement	164
10.4.1	The Approach	164
10.4.2	Self-Refine Loop	164
10.4.3	Applications	164
10.5	ReWOO: Reasoning WithOut Observation	164
10.5.1	Motivation	164
10.5.2	ReWOO Approach	165
10.5.3	Benefits	165
10.6	HuggingGPT: LLM as Controller	165
10.6.1	The Vision	165
10.6.2	Architecture	165
10.6.3	Benefits	166
10.7	Tool Use and Function Calling	166
10.7.1	Augmenting LLMs with Tools	166
10.7.2	Function Calling	166
10.8	Challenges in Agentic AI	167
10.8.1	Reliability	167
10.8.2	Safety	167
10.8.3	Evaluation	167
10.8.4	Cost	167
10.9	Practice Problems: Chapter 10	167
10.9.1	Problem 1: Agent Planning with Tools	167
10.9.2	Problem 2: Reflexion Self-Improvement	169
10.9.3	Problem 3: Multi-Agent Collaboration	171
10.9.4	Problem 4: Tool Use vs Direct Generation	174
10.9.5	Problem 5: Agent Error Recovery	176
10.10	Summary of Chapter 10	180
A	Quick Reference	181
A.1	Key Formulas	181
A.1.1	Neural Networks	181
A.1.2	CNNs	181
A.1.3	RNNs	181
A.1.4	Attention and Transformers	182
A.1.5	RLHF	182
A.2	Architecture Comparison	182
A.3	Decoding Strategies	182
A.4	Study Tips for Exam	182

List of Figures

1.1	A simple artificial neuron receiving three inputs and producing one output	4
1.2	Step activation function: outputs 0 for negative z , and 1 for positive z	6
1.3	Perceptron learning: the decision boundary moves to better separate the two classes	11
1.4	XOR problem: No single straight line can separate blue points from red points	12
1.5	A multi-layer neural network with one hidden layer	12
2.1	Sigmoid activation function	16
2.2	Tanh activation function	16
2.3	ReLU activation function	17
2.4	Gradient descent iteratively moves toward the minimum of the loss function	21
3.1	Convolution operation: sliding a filter over the input	32
3.2	Stride determines the step size of the filter	35
3.3	Max pooling with 2×2 filter and stride 2	36
3.4	Typical CNN architecture: spatial dimensions decrease, depth increases	38
3.5	Residual block: learning the residual $F(x)$ instead of the full mapping	39
3.6	Dropout: randomly drop neurons during training	41
3.7	Autoencoder architecture: input $\rightarrow$ compressed latent code $\rightarrow$ reconstruction	45
3.8	Convolutional autoencoder: spatial dimensions decrease then increase	46
3.9	Autoencoder-based segmentation: outputs class label per pixel	48
4.1	RNN cell: takes input x_t and previous hidden state h_{t-1} , produces output y_t and new hidden state h_t	63
4.2	RNN unrolled: information flows from left to right through time	64
4.3	LSTM cell: the cell state $c_{t-1} \rightarrow c_t$ flows along the top with a forget multiply ($\times f_t$), an input update ($+ i_t \odot \tilde{c}_t$), then a tanh tap multiplied by the output gate to produce h_t	68
4.4	Deep LSTM with 3 layers: each layer's output becomes the next layer's input	69
4.5	Bidirectional RNN: combines forward and backward contexts	72
5.1	Seq2Seq model: entire input must pass through a single context vector	79
5.2	Attention weights show which input words are relevant for each output word	81
5.3	Multi-head attention: multiple attention operations in parallel	86
6.1	Transformer architecture: encoder-decoder with stacked attention and feed-forward sub-layers, each followed by Add & Norm. The decoder also attends to the encoder output via encoder-decoder (cross) attention.	94
6.2	Masking in decoder self-attention: can only attend to previous positions	97
7.1	RAG architecture: retrieve relevant documents, then generate answer	114
9.1	RLHF pipeline: Three stages to align language models	140

10.1	Agentic AI architecture: LLM interacts with environment, tools, and memory . .	162
------	--	-----

Part I

Neural Network Foundations and Computer Vision

Chapter 1

Introduction to Neural Networks

1.1 What is a Neural Network?

1.1.1 Motivation: Why Do We Need Neural Networks?

Imagine you want to teach a computer to recognize whether an image contains a cat or a dog. Traditional programming approaches would require you to write explicit rules like "if the image has pointy ears and whiskers, it's a cat." But this approach fails because:

- There are infinite variations in how cats and dogs look
- Writing rules for every possible case is impossible
- Lighting, angles, and backgrounds change the appearance

Key Idea

Neural networks learn patterns from examples rather than following explicit rules. Instead of programming the computer with rules, we show it many examples of cats and dogs, and it learns to distinguish them automatically.

1.1.2 The Biological Inspiration

Neural networks are inspired by how the human brain works. Your brain contains billions of neurons (nerve cells) that:

1. Receive signals from other neurons
2. Process these signals
3. Send output signals to other neurons

Let's understand this with a simple example: When you see a cat, light enters your eye and triggers neurons in your visual cortex. These neurons send signals to other neurons, which eventually lead to the recognition "this is a cat."

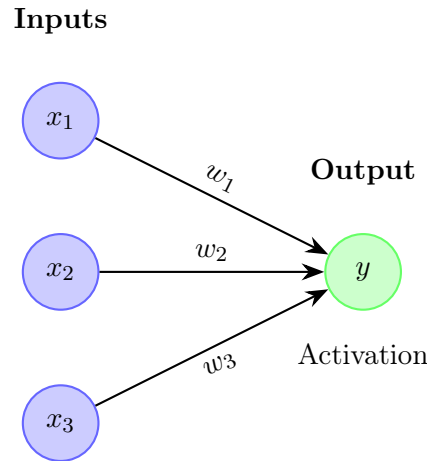


Figure 1.1: A simple artificial neuron receiving three inputs and producing one output

1.2 The Perceptron: The Simplest Neural Network

1.2.1 What is a Perceptron?

A perceptron is the simplest type of artificial neural network. It's like a single neuron that makes a decision based on multiple inputs.

Definition 1.1 (Perceptron). A perceptron is a mathematical function that takes several numerical inputs, multiplies each by a weight, sums them up, adds a bias term, and then applies a decision function to produce an output.

1.2.2 Mathematical Formulation

Let's break down the perceptron formula step by step.

Step 1: Understanding the Inputs

Suppose you want to decide whether to go outside for a walk. You might consider:

- x_1 = temperature (scaled from 0 to 1)
- x_2 = chance of rain (0 to 1)
- x_3 = how tired you feel (0 to 1)

Notation Explanation

Notation Explanation:

- x_1, x_2, x_3 are called **input features** or simply **inputs**
- The subscript (1, 2, 3) is just a label to distinguish different inputs
- We use the letter x by convention to represent inputs
- In general, if we have n inputs, we write them as $x_1, x_2, \dots, x_n$

Step 2: Understanding Weights

Not all factors are equally important. Maybe you care a lot about rain but less about temperature. We represent this using **weights**.

Notation Explanation**Weights:**

- Weights are denoted w_1, w_2, w_3 (corresponding to inputs x_1, x_2, x_3)
- A large positive weight means that input strongly influences the output
- A large negative weight means that input strongly opposes the output
- A weight close to zero means that input doesn't matter much

For our walking example:

$$w_1 = 0.3 \quad (\text{temperature has moderate importance})$$

$$w_2 = -0.8 \quad (\text{rain strongly discourages walking - negative weight})$$

$$w_3 = -0.5 \quad (\text{tiredness moderately discourages walking})$$

Step 3: Computing the Weighted Sum

Now we multiply each input by its weight and add them up:

$$z = w_1x_1 + w_2x_2 + w_3x_3 \tag{1.1}$$

Notation Explanation**Notation z :**

- z is called the **weighted sum** or **pre-activation**
- It's the total "score" before making a final decision
- We use z by convention (you might also see it called s for "sum")

Worked Example**Numerical Example:**

Suppose on a particular day:

- Temperature is pleasant: $x_1 = 0.7$
- 60% chance of rain: $x_2 = 0.6$
- You're somewhat tired: $x_3 = 0.4$

Calculate z :

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + w_3x_3 \\ &= (0.3)(0.7) + (-0.8)(0.6) + (-0.5)(0.4) \\ &= 0.21 - 0.48 - 0.20 \\ &= -0.47 \end{aligned}$$

The negative value suggests you probably shouldn't go for a walk (rain and tiredness outweigh the pleasant temperature).

Step 4: Adding a Bias Term

Sometimes we need an adjustable threshold. This is called the **bias**.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \quad (1.2)$$

Notation Explanation

Bias (b):

- The bias b shifts the decision boundary
- If b is positive, the perceptron is more likely to activate (say "yes")
- If b is negative, the perceptron is more likely to stay inactive (say "no")
- Think of b as your default mood: positive b means you're eager to walk, negative b means you prefer staying home

Step 5: Applying an Activation Function

Finally, we convert z into a clear decision using an **activation function**.

The simplest activation function is the **step function**:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases} \quad (1.3)$$

Notation Explanation

Activation Function Notation:

- y is the output of the perceptron (also called the **activation**)
- The step function is denoted $\sigma(z)$ or $f(z)$ (Greek letter sigma or function f)
- $\sigma(z)$ maps any number to either 0 or 1
- Output 1 means "yes, go for a walk"
- Output 0 means "no, stay home"

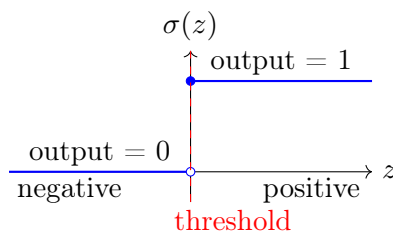


Figure 1.2: Step activation function: outputs 0 for negative z , and 1 for positive z

1.2.3 Complete Perceptron Formula

Putting it all together, a perceptron computes:

$$y = \sigma(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b) \quad (1.4)$$

Or in more compact notation:

$$y = \sigma\left(\sum_{i=1}^n w_ix_i + b\right) \quad (1.5)$$

Notation Explanation

Understanding the Summation Symbol $\sum$:

The symbol $\sum$ (capital Greek letter sigma) means "sum up."

- $\sum_{i=1}^n$ means "add up for i going from 1 to n "
- $\sum_{i=1}^n w_ix_i$ means $w_1x_1 + w_2x_2 + w_3x_3 + \cdots + w_nx_n$
- The variable i is just a counter (index)
- n is the total number of inputs

Example: If $n = 3$:

$$\sum_{i=1}^3 w_ix_i = w_1x_1 + w_2x_2 + w_3x_3$$

1.2.4 Vector Notation (Optional but Useful)

If you're comfortable with vectors, we can write the perceptron even more compactly.

Notation Explanation

Vector Notation:

We can group all inputs into a vector $\mathbf{x}$ and all weights into a vector $\mathbf{w}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad (\text{input vector})$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \quad (\text{weight vector})$$

The weighted sum can be written as a **dot product** (also called inner product):

$$z = \mathbf{w} \cdot \mathbf{x} + b = \mathbf{w}^T \mathbf{x} + b \quad (1.6)$$

where $\mathbf{w}^T$ means the **transpose** of $\mathbf{w}$ (turning a column vector into a row vector).

The dot product $\mathbf{w} \cdot \mathbf{x}$ equals $w_1x_1 + w_2x_2 + \cdots + w_nx_n$.

1.3 Learning: How Does a Perceptron Get Better?

1.3.1 The Learning Problem

When we first create a perceptron, the weights $w_1, w_2, \dots, w_n$ and bias b are random. The perceptron makes terrible predictions. **Learning** means adjusting these weights and bias so the perceptron makes better predictions.

Intuition

Think of learning like adjusting the tuning knobs on a radio. At first, the station is unclear (random weights). You turn the knobs (adjust weights) until the signal becomes clear (accurate predictions).

1.3.2 Supervised Learning

In supervised learning, we have a **training dataset** containing examples with known correct answers.

Definition 1.2 (Training Dataset). A training dataset consists of pairs $(x_1, x_2, \dots, x_n, t)$ where:

- $x_1, x_2, \dots, x_n$ are the input features
- t is the **target** (true label or correct answer)

Example 1.1. For email spam detection:

- Inputs: x_1 = number of exclamation marks, x_2 = contains word "free", x_3 = sender in contacts
- Target: $t = 1$ if spam, $t = 0$ if not spam

A training example might be:

- Email has 5 exclamation marks: $x_1 = 5$
- Contains "free": $x_2 = 1$
- Sender not in contacts: $x_3 = 0$
- This email is spam: $t = 1$

1.3.3 The Perceptron Learning Algorithm

The perceptron learning algorithm is beautifully simple. For each training example:

1. **Make a prediction:** Compute the output y using current weights and bias
2. **Calculate the error:** Compare prediction y with true label t
3. **Update weights:** If the prediction is wrong, adjust the weights to reduce the error

The Weight Update Rule

The perceptron updates weights using this formula:

$$w_i \leftarrow w_i + \eta \cdot (t - y) \cdot x_i \quad (1.7)$$

$$b \leftarrow b + \eta \cdot (t - y) \quad (1.8)$$

Notation Explanation

Understanding the Update Rule:

- $\leftarrow$ means "update to" or "assign"
- η (Greek letter eta) is the **learning rate** (a small positive number like 0.01)
- $(t - y)$ is the **error**: difference between target and prediction
- If $t = 1$ and $y = 0$: error = $1 - 0 = 1$ (missed a positive example)
- If $t = 0$ and $y = 1$: error = $0 - 1 = -1$ (falsely predicted positive)
- If $t = y$: error = 0 (correct prediction, no update needed)

The learning rate η controls how big each update step is:

- Large η : learn fast but might overshoot
- Small η : learn slowly but more stable
- Typical values: $\eta = 0.01$ to 0.1

Worked Example

Numerical Example of Weight Update:

Suppose we have a perceptron with:

- Current weights: $w_1 = 0.5$, $w_2 = -0.3$
- Current bias: $b = 0.1$
- Learning rate: $\eta = 0.1$

We see a training example:

- Inputs: $x_1 = 0.8$, $x_2 = 0.6$
- True label: $t = 1$

Step 1: Make prediction

$$\begin{aligned}
 z &= w_1x_1 + w_2x_2 + b \\
 &= (0.5)(0.8) + (-0.3)(0.6) + 0.1 \\
 &= 0.4 - 0.18 + 0.1 \\
 &= 0.32
 \end{aligned}$$

Since $z > 0$, the step function outputs $y = 1$.

Step 2: Calculate error

$$\text{error} = t - y = 1 - 1 = 0$$

Step 3: Update weights

Since error is 0, no update is needed. The perceptron predicted correctly!

Now let's try another example where prediction is wrong:

- Inputs: $x_1 = 0.3$, $x_2 = 0.9$

- True label: $t = 0$ (should predict 0)

Step 1: Make prediction

$$\begin{aligned} z &= (0.5)(0.3) + (-0.3)(0.9) + 0.1 \\ &= 0.15 - 0.27 + 0.1 \\ &= -0.02 \end{aligned}$$

Since $z < 0$, the step function outputs $y = 0$. Correct again!
Let's try one more where we make a mistake:

- Inputs: $x_1 = 0.9$, $x_2 = 0.1$
- True label: $t = 0$

Step 1: Make prediction

$$\begin{aligned} z &= (0.5)(0.9) + (-0.3)(0.1) + 0.1 \\ &= 0.45 - 0.03 + 0.1 \\ &= 0.52 \end{aligned}$$

Since $z > 0$, output is $y = 1$. But the true label is $t = 0$. **This is wrong!**

Step 2: Calculate error

$$\text{error} = t - y = 0 - 1 = -1$$

Step 3: Update weights

$$\begin{aligned} w_1 &\leftarrow w_1 + \eta \cdot (t - y) \cdot x_1 \\ &= 0.5 + (0.1)(-1)(0.9) \\ &= 0.5 - 0.09 = 0.41 \end{aligned}$$

$$\begin{aligned} w_2 &\leftarrow w_2 + \eta \cdot (t - y) \cdot x_2 \\ &= -0.3 + (0.1)(-1)(0.1) \\ &= -0.3 - 0.01 = -0.31 \end{aligned}$$

$$\begin{aligned} b &\leftarrow b + \eta \cdot (t - y) \\ &= 0.1 + (0.1)(-1) \\ &= 0.1 - 0.1 = 0.0 \end{aligned}$$

Notice how the weights changed to reduce future errors on similar inputs!

1.3.4 Why Does This Work? Geometric Intuition

Intuition

The perceptron learns by finding a **decision boundary** (a line or hyperplane) that separates the two classes.

Think of it like drawing a line on a map to separate cities into two regions. The weights determine the slope and position of this line.

Each time the perceptron makes a mistake, it adjusts the line slightly to better separate the classes.

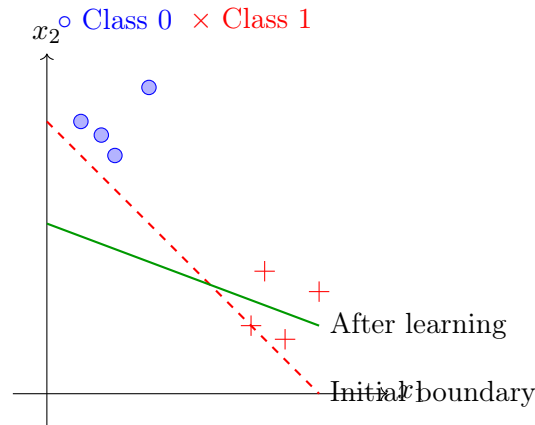


Figure 1.3: Perceptron learning: the decision boundary moves to better separate the two classes

1.4 Limitations of the Perceptron

1.4.1 The XOR Problem

While perceptrons are simple and elegant, they have a fundamental limitation: they can only solve **linearly separable** problems.

Definition 1.3 (Linear Separability). A dataset is linearly separable if you can draw a straight line (in 2D) or a flat hyperplane (in higher dimensions) that perfectly separates the two classes.

The famous **XOR problem** demonstrates this limitation.

What is XOR?

XOR stands for "exclusive OR." It's a logical operation where the output is 1 (true) if exactly one of the inputs is 1, but not both.

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Table 1.1: Truth table for XOR function

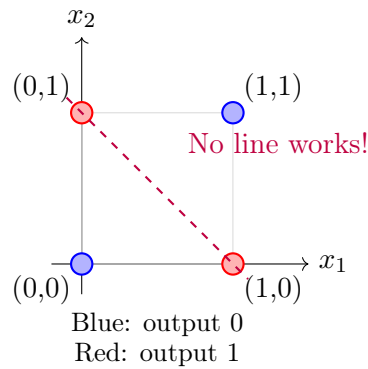


Figure 1.4: XOR problem: No single straight line can separate blue points from red points

Key Idea

A single perceptron cannot solve the XOR problem because the classes are not linearly separable. You cannot draw one straight line to separate the blue points (output 0) from the red points (output 1).

This limitation led to the development of **multi-layer neural networks**, which we'll study next.

1.4.2 From Perceptron to Multi-Layer Networks

The solution to the XOR problem and other complex problems is to stack multiple perceptrons in layers. This creates a **multi-layer perceptron (MLP)** or **feedforward neural network**.

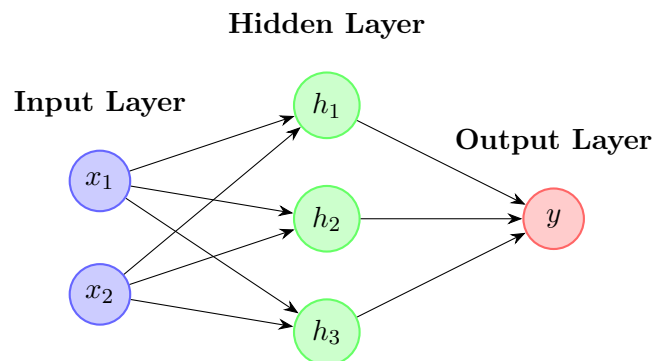


Figure 1.5: A multi-layer neural network with one hidden layer

In the next section, we'll explore how these multi-layer networks work and how they overcome the limitations of the single perceptron.

Chapter 2

Multi-Layer Neural Networks and Backpropagation

2.1 Architecture of Multi-Layer Networks

2.1.1 Layers and Connections

A multi-layer neural network consists of:

1. **Input Layer:** Receives the input features $x_1, x_2, \dots, x_n$
2. **Hidden Layers:** One or more intermediate layers that transform the inputs
3. **Output Layer:** Produces the final prediction(s)

Notation Explanation

Layer Notation:

- We label layers with superscripts in square brackets: $[0], [1], [2], \dots$
- Layer $[0]$ is the input layer
- Layers $[1], [2], \dots, [L - 1]$ are hidden layers
- Layer $[L]$ is the output layer (L is the total number of layers)
- Within each layer, we use subscripts for individual neurons: $h_1^{[1]}, h_2^{[1]}, \dots$

Example: $h_3^{[2]}$ means the 3rd neuron in the 2nd layer.

2.1.2 Fully Connected (Dense) Layers

In a **fully connected layer** (also called **dense layer**), every neuron in one layer connects to every neuron in the next layer.

$$h_j^{[l]} = \sigma \left(\sum_i w_{ji}^{[l]} h_i^{[l-1]} + b_j^{[l]} \right) \quad (2.1)$$

Notation Explanation

Breaking Down the Formula:

- $h_j^{[l]}$ = activation of the j -th neuron in layer l
- $w_{ji}^{[l]}$ = weight connecting neuron i in layer $l - 1$ to neuron j in layer l
- The subscripts ji mean "from neuron i to neuron j "
- $h_i^{[l-1]}$ = activation of neuron i in the previous layer ($l - 1$)
- $b_j^{[l]}$ = bias for neuron j in layer l
- σ = activation function (we'll discuss different types)
- $\sum_i$ means sum over all neurons i in the previous layer

2.1.3 Forward Propagation: Computing the Output

Forward propagation is the process of computing the network's output from the inputs by passing through all layers sequentially.

Worked Example

Detailed Example: Two-Layer Network for XOR

Let's build a network to solve XOR with:

- 2 inputs: x_1, x_2
- 2 hidden neurons: h_1, h_2 (in layer [1])
- 1 output neuron: y (in layer [2])

Suppose we use these weights (learned through training):

Layer 1 weights (input to hidden):

$$\begin{aligned} w_{11}^{[1]} &= 1.0, & w_{12}^{[1]} &= 1.0 & (\text{weights for } h_1) \\ w_{21}^{[1]} &= 1.0, & w_{22}^{[1]} &= 1.0 & (\text{weights for } h_2) \\ b_1^{[1]} &= -0.5, & b_2^{[1]} &= -1.5 & (\text{biases}) \end{aligned}$$

Layer 2 weights (hidden to output):

$$\begin{aligned} w_{11}^{[2]} &= 1.0, & w_{12}^{[2]} &= -2.0 \\ b_1^{[2]} &= -0.5 \end{aligned}$$

Now let's compute the output for input $(x_1, x_2) = (1, 0)$, which should give $y = 1$ for XOR.

Step 1: Compute hidden layer activations

For h_1 :

$$\begin{aligned} z_1^{[1]} &= w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + b_1^{[1]} \\ &= (1.0)(1) + (1.0)(0) + (-0.5) \\ &= 0.5 \end{aligned}$$

Using sigmoid activation (we'll explain this soon): $\sigma(0.5) \approx 0.62$

So $h_1 = 0.62$.

For h_2 :

$$\begin{aligned} z_2^{[1]} &= w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + b_2^{[1]} \\ &= (1.0)(1) + (1.0)(0) + (-1.5) \\ &= -0.5 \end{aligned}$$

Using sigmoid: $\sigma(-0.5) \approx 0.38$

So $h_2 = 0.38$.

Step 2: Compute output layer activation

$$\begin{aligned} z^{[2]} &= w_{11}^{[2]}h_1 + w_{12}^{[2]}h_2 + b^{[2]} \\ &= (1.0)(0.62) + (-2.0)(0.38) + (-0.5) \\ &= 0.62 - 0.76 - 0.5 \\ &= -0.64 \end{aligned}$$

Using sigmoid: $\sigma(-0.64) \approx 0.35$

Hmm, this is closer to 0 than 1. With better weights, we'd get closer to 1!

2.2 Activation Functions

2.2.1 Why Do We Need Activation Functions?

Intuition

Without activation functions, a neural network is just a sequence of linear operations (weighted sums). No matter how many layers you stack, it would be equivalent to a single layer.

Activation functions introduce **non-linearity**, allowing the network to learn complex patterns.

2.2.2 Common Activation Functions

1. Sigmoid Function

The sigmoid function squashes any input to a value between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.2)$$

Notation Explanation

Understanding the Formula:

- e is Euler's number ≈ 2.718 (the base of natural logarithms)
- e^{-z} means e raised to the power $-z$
- When z is large positive: $e^{-z} \approx 0$, so $\sigma(z) \approx \frac{1}{1+0} = 1$
- When z is large negative: e^{-z} is huge, so $\sigma(z) \approx 0$

- When $z = 0$: $\sigma(0) = \frac{1}{1+1} = 0.5$

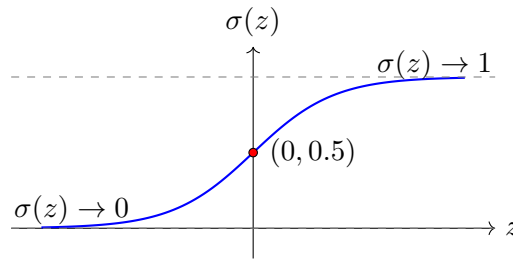


Figure 2.1: Sigmoid activation function

Advantages:

- Smooth, differentiable everywhere
- Output is interpretable as probability (0 to 1)

Disadvantages:

- **Vanishing gradient problem:** When $|z|$ is large, the gradient is very small
- Outputs are not zero-centered
- Expensive to compute (involves exponential function)

2. Hyperbolic Tangent (tanh)

The tanh function is similar to sigmoid but outputs values between -1 and 1.

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{2}{1 + e^{-2z}} - 1 \quad (2.3)$$

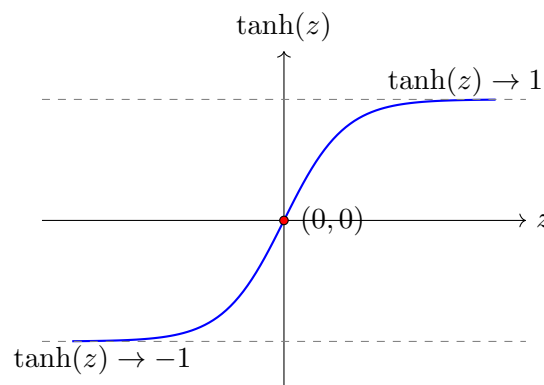


Figure 2.2: Tanh activation function

Advantages:

- Zero-centered (mean output is 0)
- Stronger gradients than sigmoid

Disadvantages:

- Still suffers from vanishing gradient problem
- Expensive to compute

3. ReLU (Rectified Linear Unit)

ReLU is the most popular activation function in modern deep learning.

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad (2.4)$$

Notation Explanation**Understanding $\max(0, z)$:**

The max function returns the larger of two values:

- If $z = 3$: $\max(0, 3) = 3$ (the larger value)
- If $z = -2$: $\max(0, -2) = 0$ (zero is larger)
- If $z = 0$: $\max(0, 0) = 0$

Essentially, ReLU outputs the input if it's positive, and zero otherwise.

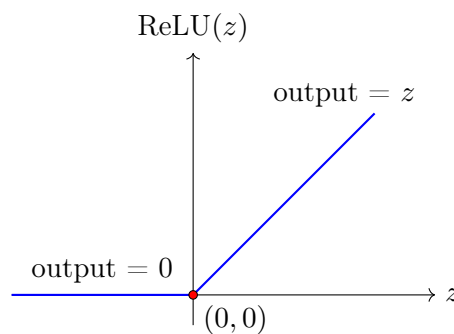


Figure 2.3: ReLU activation function

Advantages:

- Computationally very efficient (just a comparison and selection)
- Does not suffer from vanishing gradient for positive inputs
- Helps networks converge faster
- Induces sparsity (many activations become exactly 0)

Disadvantages:

- **Dying ReLU problem:** If a neuron's input is always negative, its gradient is always zero, and it stops learning
- Not zero-centered
- Not differentiable at $z = 0$ (though we typically ignore this in practice)

4. Leaky ReLU and Variants

To address the dying ReLU problem, several variants have been proposed:

$$\text{Leaky ReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases} \quad (2.5)$$

where α is a small constant like 0.01.

$$\text{Parametric ReLU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases} \quad (2.6)$$

where α is learned during training.

$$\text{ELU}(z) = \begin{cases} z & \text{if } z > 0 \\ \alpha(e^z - 1) & \text{if } z \leq 0 \end{cases} \quad (2.7)$$

2.3 Training Multi-Layer Networks: Backpropagation

2.3.1 The Challenge: How to Update Hidden Layer Weights

With a single perceptron, updating weights is straightforward because we can directly see the error at the output. But in a multi-layer network, **how do we know how to adjust the weights in the hidden layers?**

This is solved by the **backpropagation algorithm**, which efficiently computes gradients for all weights in the network.

2.3.2 Loss Functions

Before we can update weights, we need to define what "error" means. This is done with a **loss function** (also called **cost function** or **objective function**).

Definition 2.1 (Loss Function). A loss function $\mathcal{L}$ measures how far the network's predictions are from the true labels. The goal of learning is to minimize this loss.

Mean Squared Error (MSE) for Regression

When predicting continuous values (regression), we often use Mean Squared Error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y - t)^2 \quad (2.8)$$

Notation Explanation

Understanding the Formula:

- y = network's prediction
- t = true target value
- $(y - t)$ = error (difference between prediction and truth)
- $(y - t)^2$ = squared error (always positive, penalizes large errors more)
- The factor $\frac{1}{2}$ is a convenience for calculus (it cancels out when we take derivative)

Example: If $y = 0.8$ and $t = 1.0$:

$$\mathcal{L} = \frac{1}{2}(0.8 - 1.0)^2 = \frac{1}{2}(0.04) = 0.02$$

Cross-Entropy Loss for Classification

For binary classification (two classes), we use **binary cross-entropy**:

$$\mathcal{L}_{\text{CE}} = -[t \log(y) + (1 - t) \log(1 - y)] \quad (2.9)$$

Notation Explanation

Understanding Cross-Entropy:

- $t \in \{0, 1\}$ = true label (0 or 1)
- $y \in [0, 1]$ = predicted probability
- $\log$ = natural logarithm (logarithm base e)
- The negative sign makes the loss positive

When $t = 1$ (true class is 1):

$$\mathcal{L} = -\log(y)$$

If y is close to 1, $\log(y) \approx 0$, so loss is small.

If y is close to 0, $\log(y) \rightarrow -\infty$, so loss is huge.

When $t = 0$ (true class is 0):

$$\mathcal{L} = -\log(1 - y)$$

If y is close to 0, then $1 - y \approx 1$, so $\log(1 - y) \approx 0$, loss is small.

If y is close to 1, then $1 - y \approx 0$, so $\log(1 - y) \rightarrow -\infty$, loss is huge.

For multi-class classification, we use **categorical cross-entropy**:

$$\mathcal{L}_{\text{CCE}} = -\sum_{c=1}^C t_c \log(y_c) \quad (2.10)$$

where C is the number of classes, t_c is 1 for the true class and 0 for others, and y_c is the predicted probability for class c .

2.3.3 Gradient Descent: The Optimization Algorithm

The Concept of Gradient

The **gradient** of a function tells us the direction of steepest increase.

Definition 2.2 (Gradient). For a function $f(w_1, w_2, \dots, w_n)$, the gradient is a vector of partial derivatives:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \\ \vdots \\ \frac{\partial f}{\partial w_n} \end{bmatrix} \quad (2.11)$$

Notation Explanation

Understanding Derivatives:

- $\frac{\partial f}{\partial w_i}$ is the **partial derivative** of f with respect to w_i
- It measures how much f changes when w_i changes slightly, holding all other variables constant
- If $\frac{\partial f}{\partial w_i} > 0$: increasing w_i increases f
- If $\frac{\partial f}{\partial w_i} < 0$: increasing w_i decreases f
- ∇ (nabla) is the symbol for gradient

Simple Example:

Consider $f(w) = w^2$ (a parabola).

The derivative is $\frac{df}{dw} = 2w$.

At $w = 3$: $\frac{df}{dw} = 6$ (positive, so f is increasing).

At $w = -2$: $\frac{df}{dw} = -4$ (negative, so f is decreasing).

Gradient Descent Algorithm

To minimize the loss, we move in the **opposite direction** of the gradient (direction of steepest decrease).

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w} \quad (2.12)$$

Notation Explanation

Understanding the Update Rule:

- η = learning rate (step size)
- $\frac{\partial \mathcal{L}}{\partial w}$ = gradient of loss with respect to weight w
- The minus sign means we move *opposite* to the gradient (to decrease loss)
- If gradient is positive, we decrease w
- If gradient is negative, we increase w

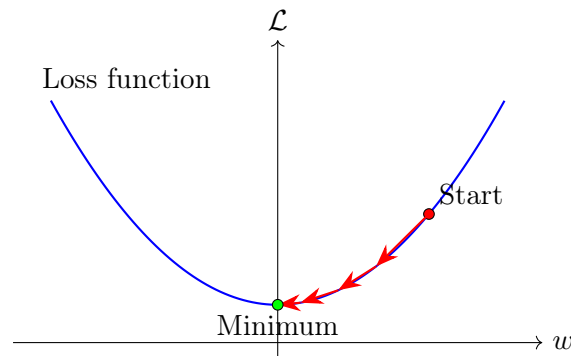


Figure 2.4: Gradient descent iteratively moves toward the minimum of the loss function

2.3.4 Backpropagation: Computing Gradients Efficiently

The Chain Rule of Calculus

Backpropagation relies on the **chain rule**, which tells us how to compute derivatives of composite functions.

Theorem 2.1 (Chain Rule). *If $y = f(u)$ and $u = g(x)$, then:*

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} \quad (2.13)$$

Worked Example

Simple Chain Rule Example:

Suppose $y = (3x + 2)^2$. We can think of this as:

- $u = 3x + 2$
- $y = u^2$

By the chain rule:

$$\begin{aligned} \frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dx} \\ &= (2u) \cdot (3) \\ &= 6u \\ &= 6(3x + 2) \\ &= 18x + 12 \end{aligned}$$

Let's verify at $x = 1$:

- $y = (3(1) + 2)^2 = 25$
- $\frac{dy}{dx} = 18(1) + 12 = 30$

This means at $x = 1$, a small increase in x increases y by approximately 30 times that increase.

Applying the Chain Rule to Neural Networks

In a neural network, the loss depends on the weights through a chain of computations:

$$\text{weights} \rightarrow \text{weighted sum} \rightarrow \text{activation} \rightarrow \text{next layer} \rightarrow \dots \rightarrow \text{loss} \quad (2.14)$$

The chain rule allows us to work backwards from the loss to compute gradients for all weights.

Key Idea

Backpropagation in Three Steps:

1. **Forward pass:** Compute all activations from input to output
2. **Compute output error:** Calculate how much the output differs from the target
3. **Backward pass:** Propagate the error backwards through the network using the chain rule

Detailed Backpropagation for a Two-Layer Network

Let's derive backpropagation for a simple network with:

- Input layer: x_1, x_2
- Hidden layer: h_1, h_2 (with sigmoid activation)
- Output layer: y (with sigmoid activation)
- Loss: Mean Squared Error

Forward Pass Equations:

Hidden layer:

$$z_1^{[1]} = w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + b_1^{[1]} \quad (2.15)$$

$$h_1 = \sigma(z_1^{[1]}) \quad (2.16)$$

$$z_2^{[1]} = w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + b_2^{[1]} \quad (2.17)$$

$$h_2 = \sigma(z_2^{[1]}) \quad (2.18)$$

Output layer:

$$z^{[2]} = w_1^{[2]}h_1 + w_2^{[2]}h_2 + b^{[2]} \quad (2.19)$$

$$y = \sigma(z^{[2]}) \quad (2.20)$$

Loss:

$$\mathcal{L} = \frac{1}{2}(y - t)^2 \quad (2.21)$$

Backward Pass:

Step 1: Gradient at the output

$$\frac{\partial \mathcal{L}}{\partial y} = y - t \quad (2.22)$$

Notation Explanation

This comes from the derivative of $\frac{1}{2}(y - t)^2$:

$$\frac{d}{dy} \left[\frac{1}{2}(y - t)^2 \right] = \frac{1}{2} \cdot 2(y - t) \cdot 1 = y - t$$

The $\frac{1}{2}$ cancels with the 2 from the power rule.

Step 2: Gradient with respect to $z^{[2]}$

By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z^{[2]}} \quad (2.23)$$

$$= (y - t) \cdot \sigma'(z^{[2]}) \quad (2.24)$$

Notation Explanation

Derivative of Sigmoid:

The derivative of the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (2.25)$$

This beautiful formula says the derivative depends only on the sigmoid output itself!

If $y = \sigma(z)$, then $\sigma'(z) = y(1 - y)$.

Example: If $y = 0.8$, then $\sigma'(z) = 0.8(0.2) = 0.16$.

So:

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = (y - t) \cdot y(1 - y) \quad (2.26)$$

Step 3: Gradients for output layer weights

$$\frac{\partial \mathcal{L}}{\partial w_1^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w_1^{[2]}} \quad (2.27)$$

$$= \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot h_1 \quad (2.28)$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial w_2^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot h_2 \quad (2.29)$$

$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \quad (2.30)$$

Step 4: Propagate error to hidden layer

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial h_1} \quad (2.31)$$

$$= \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot w_1^{[2]} \quad (2.32)$$

Similarly:

$$\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot w_2^{[2]} \quad (2.33)$$

Step 5: Gradients for hidden layer weights

$$\frac{\partial \mathcal{L}}{\partial z_1^{[1]}} = \frac{\partial \mathcal{L}}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1^{[1]}} \quad (2.34)$$

$$= \frac{\partial \mathcal{L}}{\partial h_1} \cdot h_1(1 - h_1) \quad (2.35)$$

Then:

$$\frac{\partial \mathcal{L}}{\partial w_{11}^{[1]}} = \frac{\partial \mathcal{L}}{\partial z_1^{[1]}} \cdot x_1 \quad (2.36)$$

$$\frac{\partial \mathcal{L}}{\partial w_{12}^{[1]}} = \frac{\partial \mathcal{L}}{\partial z_1^{[1]}} \cdot x_2 \quad (2.37)$$

And similarly for $w_{21}^{[1]}, w_{22}^{[1]}$.

Worked Example**Numerical Backpropagation Example:**

Let's compute gradients for the XOR network we saw earlier.

Given:

- Input: $x_1 = 1, x_2 = 0$
- Target: $t = 1$
- Hidden activations (from forward pass): $h_1 = 0.62, h_2 = 0.38$
- Output: $y = 0.35$ (from forward pass)
- Weights: $w_1^{[2]} = 1.0, w_2^{[2]} = -2.0$

Backward pass:**Step 1: Output error**

$$\frac{\partial \mathcal{L}}{\partial y} = y - t = 0.35 - 1 = -0.65$$

Step 2: Gradient at $z^{[2]}$

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = (y - t) \cdot y(1 - y) = (-0.65) \cdot (0.35)(0.65) = -0.148$$

Step 3: Gradients for output weights

$$\frac{\partial \mathcal{L}}{\partial w_1^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot h_1 = (-0.148)(0.62) = -0.092$$

$$\frac{\partial \mathcal{L}}{\partial w_2^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \cdot h_2 = (-0.148)(0.38) = -0.056$$

Step 4: Update output weights (with $\eta = 0.5$)

$$w_1^{[2]} \leftarrow 1.0 - (0.5)(-0.092) = 1.0 + 0.046 = 1.046$$

$$w_2^{[2]} \leftarrow -2.0 - (0.5)(-0.056) = -2.0 + 0.028 = -1.972$$

These updates push the weights to reduce the error!

2.3.5 Mini-Batch Gradient Descent

In practice, we don't update weights after every single training example. Instead, we use **mini-batch gradient descent**:

1. Divide the training data into small batches (e.g., 32, 64, or 128 examples)
2. For each batch:
 - Compute forward pass for all examples in the batch
 - Compute gradients for all examples
 - Average the gradients
 - Update weights once using the average gradient
3. Repeat until convergence

Notation Explanation

Training Terminology:

- **Batch size**: Number of examples processed before updating weights
- **Epoch**: One complete pass through the entire training dataset
- **Iteration**: One weight update (processing one batch)
- If you have 1000 examples and batch size 100, one epoch = 10 iterations

2.4 Universal Approximation Theorem

Theorem 2.2 (Universal Approximation Theorem). *A neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact domain to arbitrary accuracy, provided the activation function is non-linear.*

Key Idea

This theorem is profound: it says neural networks are **universal function approximators**. Given enough hidden neurons, they can learn virtually any input-output mapping! However, the theorem doesn't say:

- How many neurons are needed (might be impractically large)
- How to find the right weights (training algorithm)
- Whether the network will generalize to new data

In practice, deep networks (many layers) often work better than wide networks (many neurons in one layer).

2.5 Practice Problems: Chapters 1-2

2.5.1 Problem 1: Perceptron Calculation

Question: A perceptron has 3 inputs with the following parameters:

- Weights: $w_1 = 0.5$, $w_2 = -0.3$, $w_3 = 0.8$
- Bias: $b = -0.2$
- Activation: Step function (output 1 if $z \geq 0$, else 0)

Given input $x = [1.0, 0.4, 0.6]$, what is the output?

Solution:

Step 1: Calculate weighted sum

$$\begin{aligned} z &= w_1x_1 + w_2x_2 + w_3x_3 + b \\ &= (0.5)(1.0) + (-0.3)(0.4) + (0.8)(0.6) + (-0.2) \\ &= 0.5 - 0.12 + 0.48 - 0.2 \\ &= 0.66 \end{aligned}$$

Step 2: Apply activation function

Since $z = 0.66 > 0$, the step function outputs:

$$y = 1$$

Answer: 1

2.5.2 Problem 2: Sigmoid Derivative

Question: If $\sigma(z) = 0.8$, what is $\sigma'(z)$?

Solution:

Recall the sigmoid derivative formula:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Substituting $\sigma(z) = 0.8$:

$$\begin{aligned} \sigma'(z) &= 0.8 \times (1 - 0.8) \\ &= 0.8 \times 0.2 \\ &= 0.16 \end{aligned}$$

Answer: 0.16

2.5.3 Problem 3: Backpropagation Gradient

Question: Consider a simple network:

- Input: $x = 2$
- Weight: $w = 0.5$
- Output: $\hat{y} = wx = 1.0$
- Target: $y = 3$
- Loss: $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

Calculate $\frac{\partial \mathcal{L}}{\partial w}$ and the new weight after one gradient descent step with learning rate $\eta = 0.1$.

Solution:

Step 1: Calculate loss

$$\begin{aligned}\mathcal{L} &= \frac{1}{2}(\hat{y} - y)^2 \\ &= \frac{1}{2}(1.0 - 3)^2 \\ &= \frac{1}{2}(4) = 2\end{aligned}$$

Step 2: Calculate gradient using chain rule

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} \\ &= (\hat{y} - y) \cdot x \\ &= (1.0 - 3) \cdot 2 \\ &= -4\end{aligned}$$

Step 3: Update weight

$$\begin{aligned}w_{\text{new}} &= w - \eta \frac{\partial \mathcal{L}}{\partial w} \\ &= 0.5 - (0.1)(-4) \\ &= 0.5 + 0.4 \\ &= 0.9\end{aligned}$$

Verification: New prediction would be $\hat{y} = 0.9 \times 2 = 1.8$ (closer to target 3!)

Answer: Gradient = -4, New weight = 0.9

2.5.4 Problem 4: ReLU vs Sigmoid

Question: For input values $z = [-2, -0.5, 0, 0.5, 2]$, compute the outputs for both ReLU and Sigmoid activation functions. Which activation has larger gradients for $z = 2$?

Solution:

ReLU outputs:

$$\begin{aligned}\text{ReLU}(-2) &= \max(0, -2) = 0 \\ \text{ReLU}(-0.5) &= \max(0, -0.5) = 0 \\ \text{ReLU}(0) &= \max(0, 0) = 0 \\ \text{ReLU}(0.5) &= \max(0, 0.5) = 0.5 \\ \text{ReLU}(2) &= \max(0, 2) = 2\end{aligned}$$

Sigmoid outputs:

$$\begin{aligned}\sigma(-2) &= \frac{1}{1 + e^2} \approx 0.12 \\ \sigma(-0.5) &= \frac{1}{1 + e^{0.5}} \approx 0.38 \\ \sigma(0) &= \frac{1}{1 + 1} = 0.5 \\ \sigma(0.5) &= \frac{1}{1 + e^{-0.5}} \approx 0.62 \\ \sigma(2) &= \frac{1}{1 + e^{-2}} \approx 0.88\end{aligned}$$

Gradients at $z = 2$:

ReLU gradient:

$$\text{ReLU}'(2) = 1 \quad (\text{constant gradient for } z > 0)$$

Sigmoid gradient:

$$\begin{aligned}\sigma'(2) &= \sigma(2)(1 - \sigma(2)) \\ &= 0.88 \times 0.12 \\ &= 0.106\end{aligned}$$

Answer: ReLU has much larger gradient (1.0 vs 0.106), which helps avoid vanishing gradients!

2.5.5 Problem 5: Binary Cross-Entropy Loss

Question: A binary classifier predicts $\hat{y} = 0.7$ for a positive example ($y = 1$) and $\hat{y} = 0.3$ for a negative example ($y = 0$). Calculate the total cross-entropy loss.

Solution:

Binary cross-entropy formula:

$$\mathcal{L} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Example 1: Positive ($y = 1, \hat{y} = 0.7$)

$$\begin{aligned}\mathcal{L}_1 &= -[1 \cdot \log(0.7) + 0 \cdot \log(0.3)] \\ &= -\log(0.7) \\ &= -(-0.357) \\ &= 0.357\end{aligned}$$

Example 2: Negative ($y = 0, \hat{y} = 0.3$)

$$\begin{aligned}\mathcal{L}_2 &= -[0 \cdot \log(0.3) + 1 \cdot \log(1 - 0.3)] \\ &= -\log(0.7) \\ &= 0.357\end{aligned}$$

Total loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_1 + \mathcal{L}_2 = 0.357 + 0.357 = 0.714$$

Answer: 0.714

Interpretation: Lower loss would mean better predictions. If model predicted 0.9 for positive and 0.1 for negative, loss would be lower!

2.6 Summary of Chapter 2

In this chapter, we learned:

- Multi-layer neural networks overcome the limitations of single perceptrons
- Activation functions introduce non-linearity, allowing networks to learn complex patterns
- Common activation functions: Sigmoid, Tanh, ReLU, and variants
- Loss functions measure prediction error: MSE for regression, Cross-Entropy for classification

- Gradient descent minimizes loss by moving in the direction of steepest descent
- Backpropagation efficiently computes gradients using the chain rule of calculus
- Mini-batch gradient descent balances efficiency and stability
- The Universal Approximation Theorem guarantees neural networks can approximate any function

In the next chapter, we'll apply these concepts to computer vision with Convolutional Neural Networks (CNNs).

Chapter 3

Convolutional Neural Networks for Computer Vision

3.1 Motivation: Why Special Networks for Images?

3.1.1 The Problem with Fully Connected Networks for Images

Imagine you want to recognize handwritten digits (0-9) from images. Each image might be 28×28 pixels, giving us 784 pixels. If each pixel is an input feature, a fully connected network would need:

- 784 input neurons (one per pixel)
- Say, 100 hidden neurons
- Number of weights = $784 \times 100 = 78,400$ weights just for the first layer!

For a color image of size 224×224 pixels:

- 3 color channels (Red, Green, Blue)
- Total inputs = $224 \times 224 \times 3 = 150,528$ inputs
- With 1000 hidden neurons: $150,528 \times 1000 = 150,528,000$ weights!

Key Idea

Fully connected networks for images have three major problems:

1. **Too many parameters:** Millions of weights lead to overfitting and slow training
2. **Ignore spatial structure:** Nearby pixels are treated the same as distant pixels
3. **Not translation invariant:** A cat in the top-left vs bottom-right looks completely different to the network

3.1.2 Key Ideas Behind CNNs

Convolutional Neural Networks (CNNs) address these problems using three key concepts:

1. **Local connectivity:** Each neuron only looks at a small region of the input (receptive field)
2. **Parameter sharing:** The same weights (filters) are used across the entire image
3. **Spatial hierarchy:** Stack layers to detect increasingly complex patterns

3.2 Convolution Operation

3.2.1 What is Convolution?

Convolution is a mathematical operation that slides a small matrix (called a **filter** or **kernel**) across the input image, computing element-wise multiplications and summing the results.

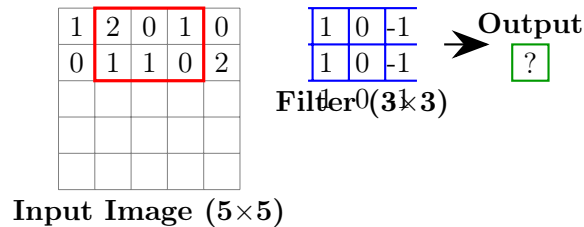


Figure 3.1: Convolution operation: sliding a filter over the input

3.2.2 Mathematical Definition

For a 2D image I and a filter K , the convolution at position (i, j) is:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (3.1)$$

Notation Explanation

Understanding the Convolution Formula:

- $I(i, j)$ = pixel value at row i , column j in the input image
- $K(m, n)$ = weight at position (m, n) in the filter
- The symbol $*$ denotes convolution (not multiplication!)
- $\sum_m \sum_n$ means sum over all positions in the filter
- For a 3×3 filter, m and n typically range from -1 to 1 (or 0 to 2)

The operation:

1. Place the filter on the image at position (i, j)
2. Multiply each filter weight with the corresponding image pixel
3. Sum all these products to get a single output value
4. Slide the filter to the next position and repeat

Worked Example

Numerical Convolution Example:

Input image (5×5):

$$I = \begin{bmatrix} 1 & 2 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 2 \\ 2 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 2 & 1 \\ 0 & 0 & 1 & 1 & 2 \end{bmatrix}$$

Vertical edge detection filter (3×3):

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

This filter detects vertical edges because it highlights regions where the left side is bright and the right side is dark.

Let's compute the output at position (1, 1) (top-left of the output):

Extract the 3×3 region from the input starting at (0, 0):

$$\text{Region} = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 2 & 0 & 1 \end{bmatrix}$$

Element-wise multiplication with filter:

$$\begin{bmatrix} 1 \cdot 1 & 2 \cdot 0 & 0 \cdot (-1) \\ 0 \cdot 1 & 1 \cdot 0 & 1 \cdot (-1) \\ 2 \cdot 1 & 0 \cdot 0 & 1 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 2 & 0 & -1 \end{bmatrix}$$

Sum all values:

$$\text{Output}(1, 1) = 1 + 0 + 0 + 0 + 0 + 0 + (-1) + 2 + 0 + (-1) = 1$$

Now compute at position (1, 2) (slide the filter one step right):

Extract region starting at (0, 1):

$$\text{Region} = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Element-wise multiplication:

$$\begin{bmatrix} 2 \cdot 1 & 0 \cdot 0 & 1 \cdot (-1) \\ 1 \cdot 1 & 1 \cdot 0 & 0 \cdot (-1) \\ 0 \cdot 1 & 1 \cdot 0 & 1 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 2 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

Sum:

$$\text{Output}(1, 2) = 2 + 0 + (-1) + 1 + 0 + 0 + 0 + 0 + (-1) = 1$$

Repeating this for all valid positions gives us the complete output feature map (3×3 in this case).

3.2.3 Receptive Field

Definition 3.1 (Receptive Field). The receptive field of a neuron is the region of the input image that affects that neuron's activation. In CNNs, each neuron in a convolutional layer has a local receptive field corresponding to the filter size.

Intuition

Think of the receptive field as the "field of view" of a neuron. A neuron with a 3×3 receptive field can "see" a 3×3 patch of the input image.

As we stack convolutional layers, the receptive field grows. A neuron in a deeper layer can "see" a larger region of the original input because it combines information from multiple neurons in the previous layer.

3.3 Building Blocks of CNNs

3.3.1 Convolutional Layer

A convolutional layer applies multiple filters to the input, each producing one **feature map** (also called **activation map**).

Notation Explanation

Convolutional Layer Notation:

- Input size: $H \times W \times C$ (Height $\times$ Width $\times$ Channels)
- For RGB images: $C = 3$ (Red, Green, Blue channels)
- For grayscale: $C = 1$
- Number of filters: F
- Filter size: $K \times K$ (usually 3×3 , 5×5 , or 7×7)
- Each filter has size: $K \times K \times C$ (must match input depth)
- Output size: $H' \times W' \times F$ (we'll compute H' and W' soon)

Example: Input $32 \times 32 \times 3$, apply 64 filters of size 5×5 :

- Each filter is $5 \times 5 \times 3$ (75 weights per filter)
- Total weights: $64 \times 75 = 4,800$ (plus 64 biases)
- Output: $28 \times 28 \times 64$ (64 feature maps)

3.3.2 Stride

Stride controls how far the filter moves at each step.

- Stride = 1: Move filter by 1 pixel at a time (overlapping regions)
- Stride = 2: Move filter by 2 pixels (skip pixels, smaller output)

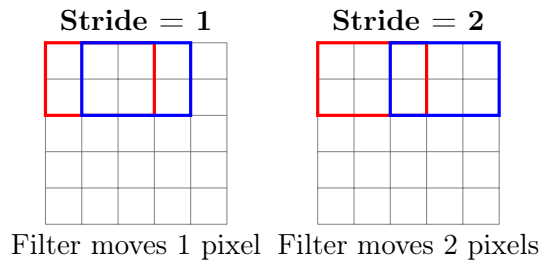


Figure 3.2: Stride determines the step size of the filter

3.3.3 Padding

Padding adds extra pixels (usually zeros) around the input to control the output size.

Notation Explanation

Padding Types:

- **Valid padding (no padding):** Don't add any pixels. Output is smaller than input.
- **Same padding:** Add enough zeros so output size equals input size (when stride = 1).
- Padding amount P = number of pixels added on each side

3.3.4 Output Size Formula

Given:

- Input size: $H \times W$
- Filter size: $K \times K$
- Stride: S
- Padding: P

Output size:

$$H' = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1 \quad (3.2)$$

$$W' = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \quad (3.3)$$

Notation Explanation

Understanding the Formula:

- $\lfloor x \rfloor$ is the **floor function** = largest integer $\leq x$
- Example: $\lfloor 3.7 \rfloor = 3$, $\lfloor 5.0 \rfloor = 5$
- $H - K + 2P$ = effective input size after padding and accounting for filter size
- Divide by stride S to get number of steps

- Add 1 because the first position counts

Worked Example

Output Size Examples:

Example 1: Input 32×32 , filter 5×5 , stride 1, no padding

$$\begin{aligned} H' &= \left\lfloor \frac{32 - 5 + 2(0)}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{27}{1} \right\rfloor + 1 \\ &= 27 + 1 = 28 \end{aligned}$$

Output: 28×28 (shrunk by 4 pixels on each dimension)

Example 2: Input 32×32 , filter 3×3 , stride 1, padding 1

$$\begin{aligned} H' &= \left\lfloor \frac{32 - 3 + 2(1)}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{31}{1} \right\rfloor + 1 \\ &= 31 + 1 = 32 \end{aligned}$$

Output: 32×32 (same size! This is "same padding")

Example 3: Input 64×64 , filter 3×3 , stride 2, padding 1

$$\begin{aligned} H' &= \left\lfloor \frac{64 - 3 + 2(1)}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{63}{2} \right\rfloor + 1 \\ &= \lfloor 31.5 \rfloor + 1 \\ &= 31 + 1 = 32 \end{aligned}$$

Output: 32×32 (halved the resolution due to stride 2)

3.3.5 Pooling Layers

Pooling layers reduce the spatial dimensions (height and width) while retaining important information.

Max Pooling

Max pooling takes the maximum value in each region.

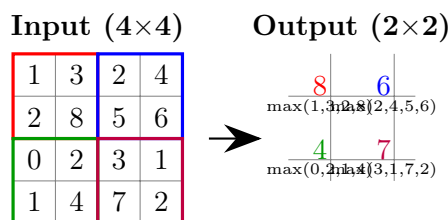


Figure 3.3: Max pooling with 2×2 filter and stride 2

Notation Explanation

Pooling Parameters:

- Pool size: Usually 2×2
- Stride: Usually equals pool size (non-overlapping regions)
- Max pooling: $\text{output}(i, j) = \max(\text{region})$
- Average pooling: $\text{output}(i, j) = \text{mean}(\text{region})$

Why pooling?

- Reduces computational cost (fewer parameters in next layer)
- Provides translation invariance (small shifts don't change max)
- Expands receptive field of subsequent layers
- Reduces overfitting by down-sampling

Average Pooling

Average pooling computes the mean of each region instead of the maximum.

$$\text{Average Pooling: } \text{output}(i, j) = \frac{1}{K^2} \sum_{m,n} \text{input}(i + m, j + n) \quad (3.4)$$

where K is the pool size.

3.3.6 Typical CNN Architecture Pattern

A typical CNN follows this pattern:

$$\boxed{\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{RELU} \rightarrow \text{POOL}] \times N \rightarrow \text{FC} \rightarrow \text{OUTPUT}} \quad (3.5)$$

Notation Explanation

Understanding the Pattern:

- INPUT: Raw image (e.g., $224 \times 224 \times 3$)
- CONV: Convolutional layer (learns features)
- RELU: Activation function (introduces non-linearity)
- POOL: Pooling layer (reduces spatial dimensions)
- $\times N$: Repeat this block N times
- FC: Fully connected layers (combine features for classification)
- OUTPUT: Final predictions (e.g., class probabilities)

As we go deeper:

- Spatial dimensions (height, width) decrease

- Number of channels (filters) increases
- Features become more abstract and complex

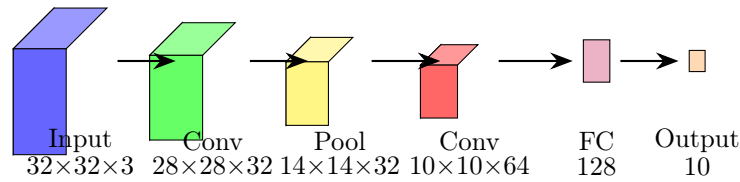


Figure 3.4: Typical CNN architecture: spatial dimensions decrease, depth increases

3.4 Famous CNN Architectures

3.4.1 LeNet-5 (1998)

One of the earliest CNNs, designed by Yann LeCun for digit recognition.

- Input: 32×32 grayscale images
- 2 convolutional layers
- 2 pooling layers (average pooling)
- 3 fully connected layers
- Total: 60,000 parameters

3.4.2 AlexNet (2012)

Won ImageNet competition, sparked the deep learning revolution.

- Input: $227 \times 227 \times 3$ RGB images
- 5 convolutional layers
- 3 max pooling layers
- 3 fully connected layers
- Used ReLU activation (first major architecture to do so)
- Dropout for regularization
- Total: 60 million parameters

Key innovations:

- ReLU instead of tanh/sigmoid (faster training)
- Data augmentation (flips, crops, color jittering)
- Dropout to prevent overfitting
- GPU acceleration

3.4.3 VGGNet (2014)

Known for its simplicity and depth.

- Uses only 3×3 convolutions throughout
- Very deep: 16-19 layers (VGG-16, VGG-19)
- Consistently doubles the number of filters after each pooling
- Architecture pattern: $[\text{CONV-CONV-POOL}] \times 5$
- Total: 138 million parameters (VGG-16)

Key Idea

VGG showed that depth matters: stacking many small 3×3 filters is better than using large filters.

Two 3×3 convolutions have the same receptive field as one 5×5 convolution, but:

- Fewer parameters: $2 \times (3^2) = 18$ vs $5^2 = 25$
- More non-linearity (two ReLU activations instead of one)
- Deeper network can learn more complex functions

3.4.4 ResNet (2015)

Introduced **residual connections** (skip connections), enabling very deep networks.

The Degradation Problem

Surprisingly, simply stacking more layers doesn't always improve performance. Very deep networks can have **higher training error** than shallower networks (not due to overfitting, but difficulty in optimization).

Residual Blocks

The solution: add **skip connections** that allow gradients to flow directly through the network.

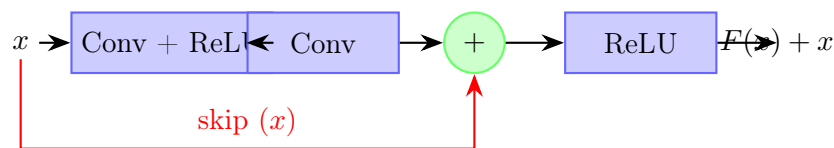


Figure 3.5: Residual block: learning the residual $F(x)$ instead of the full mapping

Notation Explanation

Residual Learning:

Traditional layer: Learn $H(x)$ directly.

Residual layer: Learn $F(x) = H(x) - x$, then compute $H(x) = F(x) + x$.

- x = input to the block
- $F(x)$ = residual (what the conv layers learn)

- $F(x) + x = \text{output}$ (adds input back)

Why this works:

- If the optimal function is close to identity, $F(x)$ can be nearly zero
- Easier to learn small adjustments than the full function
- Gradients can flow directly through skip connections (solves vanishing gradient)

ResNet variants:

- ResNet-50: 50 layers
- ResNet-101: 101 layers
- ResNet-152: 152 layers
- Even deeper variants with 1000+ layers have been trained!

3.4.5 Other Notable Architectures

- **GoogLeNet/Inception (2014):** Uses "inception modules" with multiple parallel convolutions of different sizes
- **DenseNet (2017):** Connects each layer to every other layer in a feed-forward fashion
- **EfficientNet (2019):** Systematically scales width, depth, and resolution
- **Vision Transformer (ViT, 2020):** Applies transformer architecture directly to images

3.5 Regularization Techniques for Deep Networks

3.5.1 The Overfitting Problem

Key Idea

As neural networks grow deeper and more complex, they can **overfit**: memorizing the training data instead of learning generalizable patterns.

Signs of overfitting:

- Training accuracy is very high (>95%)
- Validation/test accuracy is much lower
- Model fails on new, unseen data

Regularization techniques prevent overfitting by constraining the model during training.

3.5.2 Dropout

The Core Idea

Dropout (Srivastava et al., 2014) randomly "drops out" (sets to zero) a fraction of neurons during each training iteration.

Intuition

Think of dropout like training a team where random members are absent each day. This forces:

- Every member to learn to work independently
- The team not to rely too heavily on any single member
- Robust collaboration patterns

Similarly, dropout forces each neuron to learn useful features independently, preventing co-adaptation where neurons become too dependent on each other.

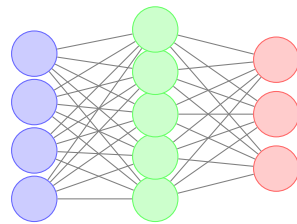
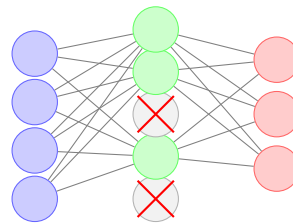
Standard Network**With Dropout**

Figure 3.6: Dropout: randomly drop neurons during training

Dropout Algorithm**During Training:**

For each training iteration and each layer with dropout rate p :

1. For each neuron, generate random number $r \sim \text{Uniform}(0, 1)$
2. If $r < p$: set neuron output to 0 (drop it)
3. If $r \geq p$: keep neuron active and scale by $\frac{1}{1-p}$

$$h_{\text{dropout}} = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{with probability } (1-p) \end{cases} \quad (3.6)$$

Notation Explanation**Why scale by $\frac{1}{1-p}$?**

Without scaling:

- During training: Expected output = $(1-p) \cdot h$ (some neurons dropped)
- During testing: Expected output = h (all neurons active)
- Mismatch between training and testing!

Scaling by $\frac{1}{1-p}$ makes expected outputs match:

- Training: $(1-p) \cdot \frac{h}{1-p} = h$
- Testing: h

This is called **inverted dropout**.

During Testing/Inference:

Use all neurons without dropout (no random dropping).

Worked Example**Dropout Example:**

Suppose we have a hidden layer with 4 neurons and dropout rate $p = 0.5$ (drop 50% of neurons).

Neuron activations before dropout:

$$h = [0.8, 1.2, 0.3, 0.9]$$

Random numbers generated:

$$r = [0.2, 0.7, 0.4, 0.1]$$

Apply dropout ($p = 0.5$):

- Neuron 1: $r = 0.2 < 0.5 \rightarrow \text{DROP} \rightarrow \text{output} = 0$
- Neuron 2: $r = 0.7 \geq 0.5 \rightarrow \text{KEEP} \rightarrow \text{output} = \frac{1.2}{1-0.5} = 2.4$
- Neuron 3: $r = 0.4 < 0.5 \rightarrow \text{DROP} \rightarrow \text{output} = 0$
- Neuron 4: $r = 0.1 < 0.5 \rightarrow \text{DROP} \rightarrow \text{output} = 0$

After dropout:

$$h_{\text{dropout}} = [0, 2.4, 0, 0]$$

At test time: Use original $h = [0.8, 1.2, 0.3, 0.9]$ (no dropout).

Why Dropout Works

1. **Ensemble effect:** Each training iteration uses a different subnetwork. It's like training many different networks and averaging them.
2. **Prevents co-adaptation:** Neurons can't rely on specific other neurons being present, so they learn more robust features.
3. **Sparsity:** Encourages sparse representations (only a subset of neurons active).

Typical dropout rates:

- Hidden layers: $p = 0.5$ (drop 50%)
- Input layer: $p = 0.2$ (drop 20%, if used at all)
- Output layer: usually no dropout

3.5.3 Batch Normalization**Internal Covariate Shift Problem**

As training progresses, the distribution of inputs to each layer changes because the parameters of previous layers change. This slows down training.

Key Idea

Batch Normalization (Ioffe & Szegedy, 2015) normalizes the inputs to each layer, making training faster and more stable.

Batch Normalization Algorithm

For a mini-batch of activations $\{x_1, x_2, \dots, x_m\}$:

Step 1: Compute batch statistics

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad (\text{batch mean}) \quad (3.7)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (\text{batch variance}) \quad (3.8)$$

Step 2: Normalize

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (3.9)$$

where ϵ (e.g., 10^{-5}) prevents division by zero.

Step 3: Scale and shift

$$y_i = \gamma \hat{x}_i + \beta \quad (3.10)$$

where γ (scale) and β (shift) are **learned parameters**.

Notation Explanation**Why scale and shift?**

Pure normalization forces mean=0, variance=1. But sometimes the network needs different distributions!

γ and β allow the network to learn the optimal scale and shift. If needed, the network can even undo the normalization by learning $\gamma = \sqrt{\sigma_B^2}$ and $\beta = \mu_B$.

Worked Example**Batch Norm Example:**

Mini-batch of 4 activations: $x = [2.0, 3.0, 2.5, 2.5]$

Learned parameters: $\gamma = 2.0$, $\beta = 1.0$

Step 1: Batch statistics

$$\begin{aligned} \mu_B &= \frac{2.0 + 3.0 + 2.5 + 2.5}{4} = 2.5 \\ \sigma_B^2 &= \frac{(2.0 - 2.5)^2 + (3.0 - 2.5)^2 + (2.5 - 2.5)^2 + (2.5 - 2.5)^2}{4} \\ &= \frac{0.25 + 0.25 + 0 + 0}{4} = 0.125 \end{aligned}$$

Step 2: Normalize (with $\epsilon = 0.001$)

$$\begin{aligned}\hat{x}_1 &= \frac{2.0 - 2.5}{\sqrt{0.125 + 0.001}} = \frac{-0.5}{\sqrt{0.126}} \approx -1.41 \\ \hat{x}_2 &= \frac{3.0 - 2.5}{\sqrt{0.126}} \approx 1.41 \\ \hat{x}_3 &= \frac{2.5 - 2.5}{\sqrt{0.126}} = 0 \\ \hat{x}_4 &= 0\end{aligned}$$

Step 3: Scale and shift

$$\begin{aligned}y_1 &= 2.0(-1.41) + 1.0 = -2.82 + 1.0 = -1.82 \\ y_2 &= 2.0(1.41) + 1.0 = 2.82 + 1.0 = 3.82 \\ y_3 &= 2.0(0) + 1.0 = 1.0 \\ y_4 &= 1.0\end{aligned}$$

Final outputs: $y = [-1.82, 3.82, 1.0, 1.0]$

Benefits of Batch Normalization

- **Faster training:** Can use higher learning rates
- **Reduces dependence on initialization:** More robust to weight initialization
- **Regularization effect:** Slight noise from batch statistics acts as regularizer
- **Reduces vanishing gradients:** Keeps activations in useful ranges

3.5.4 Layer Normalization

While Batch Norm normalizes across the batch dimension, **Layer Normalization** normalizes across the feature dimension for each example independently.

$$\text{Layer Norm: Normalize across features (for each sample separately)} \quad (3.11)$$

$$\text{Batch Norm: Normalize across batch (for each feature separately)} \quad (3.12)$$

When to use each:

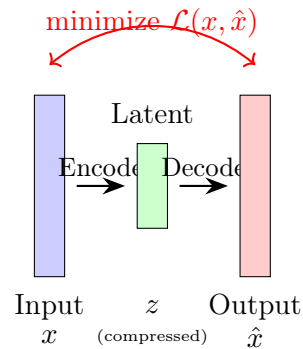
- **Batch Norm:** CNNs, fixed batch sizes, not sequential data
- **Layer Norm:** RNNs, Transformers, variable sequence lengths, small batches

3.6 Autoencoders

3.6.1 What is an Autoencoder?

Definition 3.2 (Autoencoder). An autoencoder is a neural network trained to reconstruct its input. It consists of two parts:

1. **Encoder:** Compresses input into a low-dimensional representation (bottleneck)
2. **Decoder:** Reconstructs the input from the compressed representation

Figure 3.7: Autoencoder architecture: input $\rightarrow$ compressed latent code $\rightarrow$ reconstruction**Key Idea****Why autoencoders?**

By forcing the network to compress and then reconstruct, the bottleneck learns a **compressed representation** that captures the most important features of the input.

Applications:

- Dimensionality reduction (like PCA, but non-linear)
- Denoising (remove noise from images)
- Anomaly detection (unusual inputs reconstruct poorly)
- Feature learning
- Data generation

3.6.2 Mathematical Formulation

Encoder:

$$z = f_{\text{enc}}(x; \theta_{\text{enc}}) \quad (3.13)$$

Decoder:

$$\hat{x} = f_{\text{dec}}(z; \theta_{\text{dec}}) \quad (3.14)$$

Loss function (reconstruction error):

$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2 = \sum_i (x_i - \hat{x}_i)^2 \quad (3.15)$$

For images, this is Mean Squared Error (MSE) or Binary Cross-Entropy (for pixel values in $[0,1]$).

Worked Example**Simple Autoencoder Example:**

Input: 784-dimensional image (28×28 MNIST digit)

Architecture:

- **Encoder:** $784 \rightarrow 128 \rightarrow 32$ (compress to 32 dimensions)
- **Latent code:** 32 dimensions
- **Decoder:** $32 \rightarrow 128 \rightarrow 784$ (reconstruct back)

Forward pass:Input: Image of digit "5" $\rightarrow x \in \mathbb{R}^{784}$

Encoder:

$$h_1 = \text{ReLU}(W_1 x + b_1) \quad (784 \rightarrow 128)$$

$$z = \text{ReLU}(W_2 h_1 + b_2) \quad (128 \rightarrow 32)$$

 z is the compressed 32-dimensional representation capturing the essence of "5".

Decoder:

$$h_2 = \text{ReLU}(W_3 z + b_3) \quad (32 \rightarrow 128)$$

$$\hat{x} = \sigma(W_4 h_2 + b_4) \quad (128 \rightarrow 784)$$

 $\hat{x}$ is the reconstructed image.**Loss:**

$$\mathcal{L} = \frac{1}{784} \sum_{i=1}^{784} (x_i - \hat{x}_i)^2$$

Training minimizes this reconstruction error, forcing z to capture essential information.**3.6.3 Convolutional Autoencoders**

For images, we use convolutional layers in both encoder and decoder.

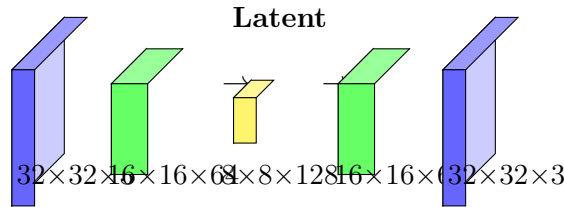
Encoder: Convolution + Pooling (downsample)**Decoder:** Transposed Convolution (upsample) + Convolution

Figure 3.8: Convolutional autoencoder: spatial dimensions decrease then increase

3.6.4 Regularized Autoencoders

Basic autoencoders can sometimes learn trivial solutions (e.g., copying input exactly). Regularization forces the model to learn meaningful representations.

1. Sparse Autoencoder

Add sparsity penalty to encourage most latent activations to be zero:

$$\mathcal{L} = \|x - \hat{x}\|^2 + \lambda \sum_j |z_j| \quad (3.16)$$

where λ controls sparsity strength.**Intuition**

Sparse representations are more interpretable: each latent dimension captures a specific feature, and most are inactive for any given input.

2. Denoising Autoencoder

Train the autoencoder to reconstruct clean input from corrupted input:

1. Add noise to input: $\tilde{x} = x + \epsilon$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$
2. Encoder processes noisy input: $z = f_{\text{enc}}(\tilde{x})$
3. Decoder reconstructs clean input: $\hat{x} = f_{\text{dec}}(z)$
4. Loss: $\mathcal{L} = \|x - \hat{x}\|^2$ (compare to *clean* input!)

Key Idea

Denoising autoencoders learn robust representations by recovering the clean signal from noise. This prevents overfitting and learns features that are resilient to perturbations. Applications: Image denoising, speech enhancement, missing data imputation.

3. Variational Autoencoder (VAE)

Instead of learning a deterministic encoding, VAE learns a probability distribution over latent codes.

$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x)) \quad (3.17)$$

The encoder outputs mean μ and variance σ^2 , then samples $z \sim \mathcal{N}(\mu, \sigma^2)$.
VAE loss combines reconstruction error with KL divergence regularization:

$$\mathcal{L}_{\text{VAE}} = \|x - \hat{x}\|^2 + \beta \cdot D_{\text{KL}}(q(z|x) \| p(z)) \quad (3.18)$$

where $p(z) = \mathcal{N}(0, I)$ is a standard normal prior.

Benefit: VAEs can generate new data by sampling from the latent space!

3.6.5 Class Autoencoder

A **class autoencoder** (also called **conditional autoencoder**) incorporates class labels into the encoding.

Notation Explanation

Architecture:

Encoder:

$$z = f_{\text{enc}}([x; y]) \quad (3.19)$$

where $[x; y]$ concatenates input x with class label y (one-hot encoded).

Decoder:

$$\hat{x} = f_{\text{dec}}([z; y]) \quad (3.20)$$

The decoder also receives the class label!

Key Idea

Why use class information?

1. **Better representations:** Latent code z can focus on capturing variations *within* a class, not between classes

- 2. **Conditional generation:** Can generate samples from specific classes
- 3. **Disentanglement:** Separates class-specific information (label) from class-agnostic variations (latent code)

Example 3.1. Class Autoencoder for MNIST:

Input: Digit image + label

For digit "7":

- Input x : Image of 7
- Label y : $[0,0,0,0,0,0,0,1,0,0]$ (one-hot for digit 7)
- Encoder receives: $[x; y]$
- Latent z : Captures writing style, thickness, slant (not which digit!)
- Decoder receives: $[z; y]$
- Output $\hat{x}$: Reconstructed 7

Benefit: Can generate different styles of "7" by varying z while keeping y fixed!**3.6.6 Application: Image Segmentation with Autoencoders**Autoencoders can be adapted for **semantic segmentation**: labeling each pixel with a class.**Architecture modification:**

- **Encoder:** Same as autoencoder (downsamples image)
- **Decoder:** Upsamples to original resolution
- **Output:** Class probabilities for each pixel (not reconstruction of input!)

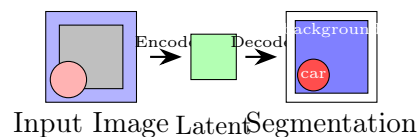
Loss: Cross-entropy between predicted and true segmentation masks.

Figure 3.9: Autoencoder-based segmentation: outputs class label per pixel

Famous architecture: U-Net (Ronneberger et al., 2015)

- Encoder path: Convolutions + pooling (downsampling)
- Decoder path: Transposed convolutions (upsampling)
- **Skip connections:** Copy features from encoder to decoder at same resolution
- Highly effective for medical image segmentation

3.7 Practice Problems: Chapter 3

3.7.1 Problem 1: Convolution Output Size

Question: An input image of size $32 \times 32 \times 3$ is passed through a convolutional layer with 64 filters of size 5×5 , stride $s = 1$, and padding $p = 2$. What is the output dimension?

Solution:

Step 1: Apply output size formula

For each spatial dimension:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - K}{s} \right\rfloor + 1$$

where:

- $H_{\text{in}} = 32$ (input height)
- $p = 2$ (padding)
- $K = 5$ (kernel size)
- $s = 1$ (stride)

Step 2: Calculate height

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{32 + 2(2) - 5}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{32 + 4 - 5}{1} \right\rfloor + 1 \\ &= \left\lfloor \frac{31}{1} \right\rfloor + 1 \\ &= 31 + 1 = 32 \end{aligned}$$

Step 3: Calculate width (same as height due to square input)

$$W_{\text{out}} = 32$$

Step 4: Determine depth

Depth = number of filters = 64

Answer: Output dimension is $32 \times 32 \times 64$

Interpretation: With padding=2 and stride=1, the spatial dimensions are preserved (still 32×32), but the depth increases to 64 (one feature map per filter).

3.7.2 Problem 2: Max Pooling Operation

Question: Apply 2×2 max pooling with stride 2 to the following 4×4 feature map:

$$\begin{bmatrix} 3 & 1 & 5 & 2 \\ 0 & 4 & 7 & 1 \\ 2 & 6 & 3 & 8 \\ 1 & 2 & 4 & 5 \end{bmatrix}$$

Solution:

Step 1: Divide input into non-overlapping 2×2 regions

Since stride = 2 and kernel size = 2×2 , we have non-overlapping regions:

Region 1 (top-left): $\begin{bmatrix} 3 & 1 \\ 0 & 4 \end{bmatrix}$

Region 2 (top-right): $\begin{bmatrix} 5 & 2 \\ 7 & 1 \end{bmatrix}$

Region 3 (bottom-left): $\begin{bmatrix} 2 & 6 \\ 1 & 2 \end{bmatrix}$

Region 4 (bottom-right): $\begin{bmatrix} 3 & 8 \\ 4 & 5 \end{bmatrix}$

Step 2: Take maximum in each region

- Region 1: $\max(3, 1, 0, 4) = 4$
- Region 2: $\max(5, 2, 7, 1) = 7$
- Region 3: $\max(2, 6, 1, 2) = 6$
- Region 4: $\max(3, 8, 4, 5) = 8$

Answer:

$$\text{Output} = \begin{bmatrix} 4 & 7 \\ 6 & 8 \end{bmatrix}$$

Interpretation: Max pooling reduces spatial dimensions by half (from 4×4 to 2×2) while keeping the most prominent features.

3.7.3 Problem 3: Dropout During Training

Question: During training with dropout probability $p = 0.5$, a hidden layer has activations $h = [2.0, 4.0, 1.0, 3.0]$. After applying dropout, neurons 2 and 4 are dropped. What are the activations after dropout (using inverted dropout)?

Solution:

Step 1: Understand inverted dropout formula

For each neuron i :

$$h_i^{\text{dropout}} = \begin{cases} 0 & \text{if neuron } i \text{ is dropped} \\ \frac{h_i}{1-p} & \text{if neuron } i \text{ is kept} \end{cases}$$

where $p = 0.5$ is the dropout probability.

Step 2: Apply dropout mask

Given: Neurons 2 and 4 are dropped (set to 0), neurons 1 and 3 are kept.

For kept neurons, scale by $\frac{1}{1-p} = \frac{1}{1-0.5} = \frac{1}{0.5} = 2$:

$$h_1^{\text{dropout}} = \frac{2.0}{0.5} = 4.0 \quad (\text{kept, scaled})$$

$$h_2^{\text{dropout}} = 0.0 \quad (\text{dropped})$$

$$h_3^{\text{dropout}} = \frac{1.0}{0.5} = 2.0 \quad (\text{kept, scaled})$$

$$h_4^{\text{dropout}} = 0.0 \quad (\text{dropped})$$

Answer: $h^{\text{dropout}} = [4.0, 0.0, 2.0, 0.0]$

Step 3: Verify expected value

Expected value of each neuron remains unchanged:

$$\mathbb{E}[h_1^{\text{dropout}}] = 0.5 \cdot 0 + 0.5 \cdot 4.0 = 2.0 = h_1 \quad \checkmark$$

$$\mathbb{E}[h_3^{\text{dropout}}] = 0.5 \cdot 0 + 0.5 \cdot 2.0 = 1.0 = h_3 \quad \checkmark$$

Interpretation: Inverted dropout scales kept neurons during training so that no scaling is needed at test time.

3.7.4 Problem 4: Batch Normalization

Question: A batch normalization layer receives a mini-batch of 4 activations: $x = [1.0, 3.0, 5.0, 7.0]$. Given learnable parameters $\gamma = 2.0$ and $\beta = 1.0$, and $\epsilon = 10^{-5}$, compute the normalized output.

Solution:

Step 1: Compute batch mean

$$\mu_B = \frac{1}{4}(1.0 + 3.0 + 5.0 + 7.0) = \frac{16.0}{4} = 4.0$$

Step 2: Compute batch variance

$$\begin{aligned}\sigma_B^2 &= \frac{1}{4} [(1 - 4)^2 + (3 - 4)^2 + (5 - 4)^2 + (7 - 4)^2] \\ &= \frac{1}{4} [(-3)^2 + (-1)^2 + (1)^2 + (3)^2] \\ &= \frac{1}{4} [9 + 1 + 1 + 9] = \frac{20}{4} = 5.0\end{aligned}$$

Step 3: Normalize each activation

For each x_i :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} = \frac{x_i - 4.0}{\sqrt{5.0 + 0.00001}} \approx \frac{x_i - 4.0}{2.236}$$

$$\hat{x}_1 = \frac{1.0 - 4.0}{2.236} = \frac{-3.0}{2.236} \approx -1.342$$

$$\hat{x}_2 = \frac{3.0 - 4.0}{2.236} = \frac{-1.0}{2.236} \approx -0.447$$

$$\hat{x}_3 = \frac{5.0 - 4.0}{2.236} = \frac{1.0}{2.236} \approx 0.447$$

$$\hat{x}_4 = \frac{7.0 - 4.0}{2.236} = \frac{3.0}{2.236} \approx 1.342$$

Step 4: Scale and shift

For each $\hat{x}_i$:

$$y_i = \gamma \hat{x}_i + \beta = 2.0 \cdot \hat{x}_i + 1.0$$

$$y_1 = 2.0(-1.342) + 1.0 = -2.684 + 1.0 = -1.684$$

$$y_2 = 2.0(-0.447) + 1.0 = -0.894 + 1.0 = 0.106$$

$$y_3 = 2.0(0.447) + 1.0 = 0.894 + 1.0 = 1.894$$

$$y_4 = 2.0(1.342) + 1.0 = 2.684 + 1.0 = 3.684$$

Answer: Output $y \approx [-1.684, 0.106, 1.894, 3.684]$

Interpretation: Batch normalization first normalizes to zero mean and unit variance, then allows the network to learn optimal scale (γ) and shift (β).

3.7.5 Problem 5: Autoencoder Reconstruction

Question: A simple autoencoder has:

- Encoder: $z = \text{ReLU}(W_e x + b_e)$ where $W_e = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix}$, $b_e = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$
- Decoder: $\hat{x} = W_d z + b_d$ where $W_d = \begin{bmatrix} 1.0 & 0.5 \\ 0.3 & 0.8 \end{bmatrix}$, $b_d = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Given input $x = \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix}$, compute the reconstruction $\hat{x}$.

Solution:

Step 1: Compute encoder pre-activation

$$\begin{aligned} W_e x + b_e &= \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.5(1.0) + 0.3(2.0) \\ 0.2(1.0) + 0.4(2.0) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 + 0.6 \\ 0.2 + 0.8 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \\ &= \begin{bmatrix} 1.1 \\ 1.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 0.9 \end{bmatrix} \end{aligned}$$

Step 2: Apply ReLU activation

$$z = \text{ReLU} \left(\begin{bmatrix} 1.2 \\ 0.9 \end{bmatrix} \right) = \begin{bmatrix} 1.2 \\ 0.9 \end{bmatrix}$$

(Both values are positive, so ReLU keeps them unchanged)

Step 3: Decode to reconstruction

$$\begin{aligned} \hat{x} &= W_d z + b_d \\ &= \begin{bmatrix} 1.0 & 0.5 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.9 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1.0(1.2) + 0.5(0.9) \\ 0.3(1.2) + 0.8(0.9) \end{bmatrix} \\ &= \begin{bmatrix} 1.2 + 0.45 \\ 0.36 + 0.72 \end{bmatrix} \\ &= \begin{bmatrix} 1.65 \\ 1.08 \end{bmatrix} \end{aligned}$$

Answer: Reconstruction $\hat{x} = \begin{bmatrix} 1.65 \\ 1.08 \end{bmatrix}$

Step 4: Compute reconstruction error

$$\begin{aligned} \text{MSE} &= \frac{1}{2} \|x - \hat{x}\|^2 \\ &= \frac{1}{2} [(1.0 - 1.65)^2 + (2.0 - 1.08)^2] \\ &= \frac{1}{2} [(-0.65)^2 + (0.92)^2] \\ &= \frac{1}{2} [0.4225 + 0.8464] \\ &= \frac{1.2689}{2} = 0.634 \end{aligned}$$

Interpretation: The autoencoder compresses input from 2D to 2D latent space through a bottleneck layer. Training would minimize this reconstruction error.

3.7.6 Problem 6: CNN Parameter Counting

Question: A CNN layer has:

- Input: $28 \times 28 \times 32$ ($H \times W \times C$)
- 64 filters of size 3×3
- Each filter has a bias term

How many trainable parameters does this layer have?

Solution:

Step 1: Parameters per filter

Each filter operates on all input channels. So one filter has:

$$\begin{aligned}\text{Weights per filter} &= (\text{kernel height}) \times (\text{kernel width}) \times (\text{input channels}) \\ &= 3 \times 3 \times 32 = 288\end{aligned}$$

Plus 1 bias term per filter.

Total per filter: $288 + 1 = 289$

Step 2: Total parameters for all filters

Number of filters = 64

$$\text{Total parameters} = 64 \times 289 = 18,496$$

Answer: 18,496 trainable parameters

Alternative calculation:

$$\begin{aligned}\text{Total} &= (\# \text{ filters}) \times (\text{kernel size}) \times (\text{input channels}) + (\# \text{ biases}) \\ &= 64 \times (3 \times 3) \times 32 + 64 \\ &= 64 \times 9 \times 32 + 64 \\ &= 18,432 + 64 = 18,496\end{aligned}$$

Key insight: Parameter count is independent of input spatial dimensions (28×28). It only depends on:

- Kernel size (3×3)
- Number of input channels (32)
- Number of output filters (64)

This is why CNNs are so efficient: parameter sharing across spatial locations!

3.8 Summary of Chapter 3

In this chapter, we learned:

- CNNs are specialized for image data, using local connectivity and parameter sharing
- Convolution slides filters over the input to detect local patterns
- Key hyperparameters: filter size, stride, padding
- Pooling reduces spatial dimensions while retaining important features
- Typical CNN architecture: stack CONV-RELU-POOL blocks
- Famous architectures:
 - LeNet-5: Early pioneering work

- AlexNet: Sparked deep learning revolution with ReLU and dropout
- VGGNet: Showed power of depth with simple 3×3 filters
- ResNet: Skip connections enable very deep networks (100+ layers)

- **Regularization techniques:**

- Dropout: Randomly drop neurons during training to prevent overfitting
- Batch Normalization: Normalize layer inputs for faster, more stable training
- Layer Normalization: Alternative to batch norm for sequential data

- **Autoencoders:**

- Compress input to latent representation, then reconstruct
- Regularized variants: Sparse, Denoising, Variational (VAE)
- Class autoencoder: Incorporates class labels for conditional generation
- Applications: Dimensionality reduction, denoising, image segmentation

Part II

Sequence Models and Natural Language Processing

Chapter 4

Recurrent Neural Networks and Sequence Modeling

4.1 Motivation: Sequential Data

4.1.1 What is Sequential Data?

Many real-world problems involve **sequences**: data where order matters.

Example 4.1. Examples of sequential data:

- **Text:** "The cat sat on the mat" (word order changes meaning)
- **Speech:** Audio waveforms over time
- **Time series:** Stock prices, weather data, sensor readings
- **Video:** Sequence of image frames
- **DNA sequences:** ATCGGCTA...

4.1.2 Why CNNs Don't Work Well for Sequences

While CNNs work great for images, they have limitations for sequences:

- **Fixed receptive field:** Can only look at a local window
- **No memory:** Can't remember what happened many steps ago
- **Variable length:** Sentences have different lengths; CNNs expect fixed-size inputs

Key Idea

We need networks that can:

1. Handle variable-length inputs
2. Remember information from earlier in the sequence
3. Share parameters across time steps

This is what **Recurrent Neural Networks (RNNs)** provide!

4.2 Traditional Approach: N-gram Language Models

Before neural networks, language modeling was done using **N-gram models**. Understanding these helps appreciate why neural models are so powerful.

4.2.1 What is a Language Model?

Definition 4.1 (Language Model). A language model assigns probabilities to sequences of words. Given a sequence of words $w_1, w_2, \dots, w_n$, it computes:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdots P(w_n|w_1, \dots, w_{n-1}) \quad (4.1)$$

This is called the **chain rule of probability**.

4.2.2 N-gram Models

Key Idea

The problem with the full chain rule: $P(w_n|w_1, \dots, w_{n-1})$ depends on ALL previous words. As sequences get longer, this becomes impossible to estimate.

N-gram assumption: Only the previous $N - 1$ words matter!

$$P(w_n|w_1, \dots, w_{n-1}) \approx P(w_n|w_{n-N+1}, \dots, w_{n-1}) \quad (4.2)$$

Types of N-grams

1. **Unigram (N=1):** Each word is independent

$$P(w_1, w_2, w_3) = P(w_1) \cdot P(w_2) \cdot P(w_3) \quad (4.3)$$

2. **Bigram (N=2):** Each word depends only on the previous word

$$P(w_1, w_2, w_3) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_2) \quad (4.4)$$

3. **Trigram (N=3):** Each word depends on the previous two words

$$P(w_1, w_2, w_3, w_4) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdot P(w_4|w_2, w_3) \quad (4.5)$$

4.2.3 Estimating N-gram Probabilities

We estimate probabilities from a training corpus using **Maximum Likelihood Estimation (MLE)**:

$$P(w_n|w_{n-1}) = \frac{\text{count}(w_{n-1}, w_n)}{\text{count}(w_{n-1})} \quad (4.6)$$

Notation Explanation

Understanding the Formula:

- Numerator: How many times do we see the word sequence (w_{n-1}, w_n) in the corpus?
- Denominator: How many times do we see the word w_{n-1} (followed by any word)?

- This gives the fraction of times w_n follows w_{n-1}

Worked Example

Bigram Example:

Training corpus:

"I like cats. I like dogs. I love cats."

Count bigrams:

- $\text{count}(\text{I, like}) = 2$
- $\text{count}(\text{I, love}) = 1$
- $\text{count}(\text{like, cats}) = 1$
- $\text{count}(\text{like, dogs}) = 1$
- $\text{count}(\text{love, cats}) = 1$
- $\text{count}(\text{cats, .}) = 2$
- $\text{count}(\text{dogs, .}) = 1$

Count unigrams:

- $\text{count}(\text{I}) = 3$
- $\text{count}(\text{like}) = 2$
- $\text{count}(\text{love}) = 1$
- $\text{count}(\text{cats}) = 2$
- $\text{count}(\text{dogs}) = 1$

Calculate probabilities:

$$P(\text{like}|\text{I}) = \frac{\text{count}(\text{I, like})}{\text{count}(\text{I})} = \frac{2}{3} \approx 0.67$$

$$P(\text{love}|\text{I}) = \frac{\text{count}(\text{I, love})}{\text{count}(\text{I})} = \frac{1}{3} \approx 0.33$$

$$P(\text{cats}|\text{like}) = \frac{\text{count}(\text{like, cats})}{\text{count}(\text{like})} = \frac{1}{2} = 0.5$$

$$P(\text{dogs}|\text{like}) = \frac{\text{count}(\text{like, dogs})}{\text{count}(\text{like})} = \frac{1}{2} = 0.5$$

Probability of sequence "I like cats":

$$\begin{aligned} P(\text{I like cats}) &= P(\text{I}) \cdot P(\text{like}|\text{I}) \cdot P(\text{cats}|\text{like}) \\ &= \frac{3}{14} \cdot \frac{2}{3} \cdot \frac{1}{2} \\ &= \frac{6}{84} = \frac{1}{14} \approx 0.071 \end{aligned}$$

(Note: $P(\text{I}) = \frac{3}{14}$ because there are 14 total words in the corpus)

4.2.4 Perplexity

Definition 4.2 (Perplexity). Perplexity (PPL) measures how well a language model predicts a test sequence. Lower perplexity = better model.

$$\text{PPL}(W) = P(w_1, w_2, \dots, w_N)^{-1/N} \quad (4.7)$$

Or equivalently:

$$\text{PPL}(W) = 2^{H(W)} \quad (4.8)$$

where $H(W) = -\frac{1}{N} \log_2 P(w_1, \dots, w_N)$ is the cross-entropy.

Intuition

What does perplexity mean?

Perplexity can be interpreted as the "weighted average branching factor" of the model.

- PPL = 10 means the model is as confused as if it had to choose uniformly from 10 possible next words
- PPL = 100 means the model is as confused as if it had to choose from 100 words
- Lower PPL = model is more confident and accurate

Perfect model: PPL = 1 (always predicts correctly)

Random uniform model over vocabulary of size V : PPL = V

Worked Example

Perplexity Example:

Test sequence: "I like cats"

Using our bigram model from before:

$$\begin{aligned} P(\text{I like cats}) &= P(\text{I}) \cdot P(\text{like}|\text{I}) \cdot P(\text{cats}|\text{like}) \\ &= \frac{3}{14} \cdot \frac{2}{3} \cdot \frac{1}{2} \\ &\approx 0.071 \end{aligned}$$

Perplexity:

$$\begin{aligned} \text{PPL} &= P(\text{I like cats})^{-1/3} \\ &= (0.071)^{-1/3} \\ &= 2.56 \end{aligned}$$

This means the model is as uncertain as having to choose from about 2.56 words on average at each position.

4.2.5 Problems with N-gram Models

1. **Data sparsity:** Many N-grams never appear in training, get zero probability

Example 4.2. If the corpus has "I like cats" but never "I like puppies", then:

$$P(\text{puppies}|\text{like}) = \frac{\text{count}(\text{like}, \text{puppies})}{\text{count}(\text{like})} = \frac{0}{2} = 0$$

This makes $P(\text{I like puppies}) = 0$, which is unreasonable!

2. **No generalization:** Model doesn't understand that "cats" and "dogs" are similar
3. **Storage:** Large N (like $N=5$) requires storing millions of N -gram counts
4. **Limited context:** Only considers previous $N-1$ words, ignores longer dependencies

4.2.6 Smoothing Techniques

To handle the zero probability problem, we use **smoothing**: redistributing probability mass from seen N -grams to unseen ones.

1. Add-k Smoothing (Laplace Smoothing)

Add a small constant k to all counts:

$$P_{\text{add-k}}(w_n|w_{n-1}) = \frac{\text{count}(w_{n-1}, w_n) + k}{\text{count}(w_{n-1}) + kV} \quad (4.9)$$

where V is the vocabulary size.

Notation Explanation

Understanding Add-k Smoothing:

- Numerator: Pretend we saw each N -gram k more times
- Denominator: Adjust total count (we added k to each of V possible words)
- Special case $k = 1$: **Laplace smoothing** (add-one smoothing)
- Larger k : More smoothing (more uniform distribution)

Worked Example

Add-1 Smoothing Example:

From our earlier example:

- $\text{count}(\text{like}, \text{cats}) = 1$
- $\text{count}(\text{like}, \text{dogs}) = 1$
- $\text{count}(\text{like}, \text{puppies}) = 0 \leftarrow$ unseen bigram!
- $\text{count}(\text{like}) = 2$
- Vocabulary size $V = 7$ (I, like, love, cats, dogs, puppies, .)

Without smoothing:

$$P(\text{puppies}|\text{like}) = \frac{0}{2} = 0$$

With add-1 smoothing:

$$\begin{aligned} P_{\text{add-1}}(\text{puppies}|\text{like}) &= \frac{0 + 1}{2 + 1 \times 7} \\ &= \frac{1}{9} \approx 0.11 \end{aligned}$$

For seen bigram:

$$\begin{aligned} P_{\text{add-1}}(\text{cats}|\text{like}) &= \frac{1+1}{2+7} \\ &= \frac{2}{9} \approx 0.22 \end{aligned}$$

Notice how probability mass was redistributed from "cats" ($0.5 \rightarrow 0.22$) to "puppies" ($0 \rightarrow 0.11$).

2. Backoff and Interpolation

Backoff: If an N-gram is unseen, fall back to (N-1)-gram:

$$P_{\text{backoff}}(w_n|w_{n-2}, w_{n-1}) = \begin{cases} P_{\text{trigram}}(w_n|w_{n-2}, w_{n-1}) & \text{if trigram seen} \\ \alpha \cdot P_{\text{bigram}}(w_n|w_{n-1}) & \text{otherwise} \end{cases} \quad (4.10)$$

Interpolation: Combine N-gram, (N-1)-gram, ..., unigram with weights:

$$P_{\text{interp}}(w_n|w_{n-1}) = \lambda_2 P_2(w_n|w_{n-1}) + \lambda_1 P_1(w_n) \quad (4.11)$$

where $\lambda_1 + \lambda_2 = 1$ and $\lambda_i \geq 0$.

Intuition

Backoff vs Interpolation:

- **Backoff:** Use highest-order N-gram if possible, otherwise fall back to lower order. Like asking experts first, then generalists.
- **Interpolation:** Always combine all orders. Like taking a weighted vote from experts and generalists.
- Interpolation is generally more robust

3. Kneser-Ney Smoothing

The most sophisticated and widely-used smoothing method, based on **absolute discounting** and **continuation probability**.

Key Idea

Kneser-Ney insight: For unseen bigrams, consider how "versatile" a word is (how many different contexts it appears in), not just how frequent it is.

Example:

- "San Francisco" is common, so unigram $P(\text{Francisco})$ is high
- But "Francisco" rarely appears in contexts other than after "San"
- So for a new context like "I saw Francisco", we shouldn't give it high probability

Kneser-Ney uses **continuation probability**: how many unique words precede a word?

The formula (simplified):

$$P_{\text{KN}}(w_n|w_{n-1}) = \frac{\max(\text{count}(w_{n-1}, w_n) - d, 0)}{\text{count}(w_{n-1})} + \lambda(w_{n-1})P_{\text{continuation}}(w_n) \quad (4.12)$$

where:

- d is a discount (typically 0.75)
- $\lambda(w_{n-1})$ is a normalization factor
- $P_{\text{continuation}}(w_n)$ is based on how many different contexts w_n appears in

4.3 Recurrent Neural Networks (RNNs)

4.3.1 The Core Idea: Recurrence

An RNN processes a sequence one element at a time, maintaining a **hidden state** that acts as memory.

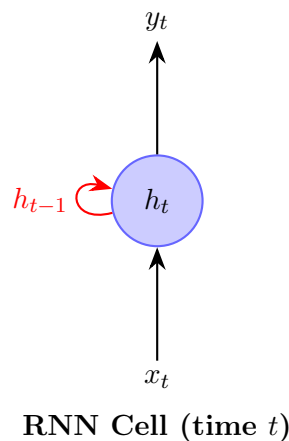


Figure 4.1: RNN cell: takes input x_t and previous hidden state h_{t-1} , produces output y_t and new hidden state h_t

4.3.2 RNN Mathematical Formulation

At each time step t , an RNN computes:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (4.13)$$

$$y_t = W_{hy}h_t + b_y \quad (4.14)$$

Notation Explanation

Understanding RNN Notation:

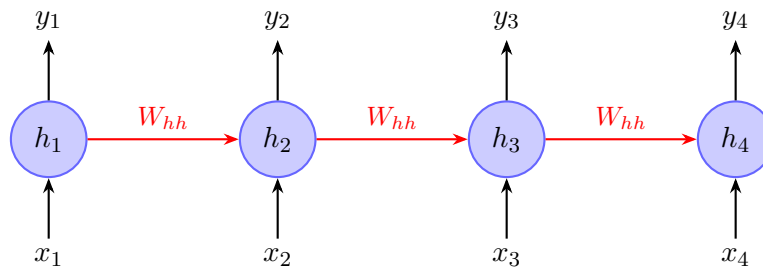
- t = time step (position in the sequence)
- x_t = input at time t (e.g., word embedding vector)
- h_t = hidden state at time t (the "memory")
- y_t = output at time t (e.g., predicted next word)

- W_{xh} = weight matrix for input-to-hidden (size: $H \times D$)
- W_{hh} = weight matrix for hidden-to-hidden (size: $H \times H$)
- W_{hy} = weight matrix for hidden-to-output (size: $O \times H$)
- b_h, b_y = bias vectors
- H = hidden size (dimension of hidden state)
- D = input dimension
- O = output dimension

Key insight: The same weights W_{xh}, W_{hh}, W_{hy} are used at *every* time step!

4.3.3 Unrolling an RNN Through Time

To understand RNNs better, we "unroll" them across time steps:



RNN unrolled through time

Same weights shared across all time steps

Figure 4.2: RNN unrolled: information flows from left to right through time

Worked Example

Numerical RNN Example:

Let's process a sequence of 3 inputs with a simple RNN.

Setup:

- Input dimension: $D = 2$
- Hidden size: $H = 3$
- Output dimension: $O = 1$
- Initial hidden state: $h_0 = [0, 0, 0]^T$

Weights (simplified):

$$W_{xh} = \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.4 \\ 0.1 & -0.3 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.2 & 0.1 & -0.1 \\ -0.1 & 0.3 & 0.2 \\ 0.15 & -0.2 & 0.25 \end{bmatrix}$$

$$W_{hy} = [0.4 \quad -0.3 \quad 0.5], \quad b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad b_y = 0$$

Input sequence:

$$x_1 = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0.8 \\ 1.2 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$$

Time step 1:

Compute h_1 :

$$\begin{aligned} z_1 &= W_{hh}h_0 + W_{xh}x_1 \\ &= W_{hh} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.4 \\ 0.1 & -0.3 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.65 \\ 0.0 \\ -0.05 \end{bmatrix} \\ &= \begin{bmatrix} 0.65 \\ 0.0 \\ -0.05 \end{bmatrix} \end{aligned}$$

Apply tanh:

$$h_1 = \tanh(z_1) \approx \begin{bmatrix} 0.57 \\ 0.0 \\ -0.05 \end{bmatrix}$$

Compute y_1 :

$$\begin{aligned} y_1 &= W_{hy}h_1 \\ &= \begin{bmatrix} 0.4 & -0.3 & 0.5 \end{bmatrix} \begin{bmatrix} 0.57 \\ 0.0 \\ -0.05 \end{bmatrix} \\ &= 0.4(0.57) + (-0.3)(0.0) + 0.5(-0.05) \\ &= 0.228 - 0.025 = 0.203 \end{aligned}$$

Time step 2:

Now we use h_1 (not h_0) as the previous hidden state:

$$\begin{aligned} z_2 &= W_{hh}h_1 + W_{xh}x_2 \\ &= \begin{bmatrix} 0.2 & 0.1 & -0.1 \\ -0.1 & 0.3 & 0.2 \\ 0.15 & -0.2 & 0.25 \end{bmatrix} \begin{bmatrix} 0.57 \\ 0.0 \\ -0.05 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.4 \\ 0.1 & -0.3 \end{bmatrix} \begin{bmatrix} 0.8 \\ 1.2 \end{bmatrix} \end{aligned}$$

Computing each piece:

$$\begin{aligned} W_{hh}h_1 &= \begin{bmatrix} 0.2(0.57) + 0.1(0.0) + (-0.1)(-0.05) \\ -0.1(0.57) + 0.3(0.0) + 0.2(-0.05) \\ 0.15(0.57) + (-0.2)(0.0) + 0.25(-0.05) \end{bmatrix} = \begin{bmatrix} 0.119 \\ -0.067 \\ 0.073 \end{bmatrix} \\ W_{xh}x_2 &= \begin{bmatrix} 0.5(0.8) + 0.3(1.2) \\ -0.2(0.8) + 0.4(1.2) \\ 0.1(0.8) + (-0.3)(1.2) \end{bmatrix} = \begin{bmatrix} 0.76 \\ 0.32 \\ -0.28 \end{bmatrix} \end{aligned}$$

Adding the two contributions:

$$z_2 = \begin{bmatrix} 0.119 \\ -0.067 \\ 0.073 \end{bmatrix} + \begin{bmatrix} 0.76 \\ 0.32 \\ -0.28 \end{bmatrix} = \begin{bmatrix} 0.879 \\ 0.253 \\ -0.207 \end{bmatrix}$$

Apply tanh element-wise:

$$h_2 = \tanh(z_2) \approx \begin{bmatrix} 0.706 \\ 0.248 \\ -0.204 \end{bmatrix}$$

Compute y_2 :

$$\begin{aligned} y_2 &= W_{hy} h_2 = \begin{bmatrix} 0.4 & -0.3 & 0.5 \end{bmatrix} \begin{bmatrix} 0.706 \\ 0.248 \\ -0.204 \end{bmatrix} \\ &= 0.4(0.706) + (-0.3)(0.248) + 0.5(-0.204) \\ &= 0.282 - 0.074 - 0.102 \approx 0.106 \end{aligned}$$

The key point: h_2 depends on both x_2 and h_1 , so it carries information from x_1 *through* h_1 . Time step 3 would proceed identically, using h_2 and x_3 to produce h_3 and y_3 .

4.3.4 Types of RNN Architectures

RNNs can be configured for different tasks:

1. **One-to-Many:** One input, sequence output
 - Example: Image captioning (image $\rightarrow$ sentence)
2. **Many-to-One:** Sequence input, one output
 - Example: Sentiment analysis (sentence $\rightarrow$ positive/negative)
3. **Many-to-Many (same length):** Sequence input, sequence output
 - Example: Video frame labeling (frames $\rightarrow$ labels per frame)
4. **Many-to-Many (different lengths):** Sequence input, different sequence output
 - Example: Machine translation (English sentence $\rightarrow$ French sentence)

4.4 Training RNNs: Backpropagation Through Time (BPTT)

4.4.1 The Challenge

RNNs are trained using **Backpropagation Through Time (BPTT)**, which is standard backpropagation applied to the unrolled network.

Notation Explanation

Loss for Sequence:

For a sequence of length T , the total loss is the sum of losses at each time step:

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t \quad (4.15)$$

where $\mathcal{L}_t$ is the loss at time t (e.g., cross-entropy between y_t and target t_t).
To update weights, we need:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}}, \quad \frac{\partial \mathcal{L}}{\partial W_{xh}}, \quad \frac{\partial \mathcal{L}}{\partial W_{hy}} \quad (4.16)$$

These gradients involve the chain rule across many time steps!

4.4.2 The Vanishing Gradient Problem

Key Idea

Vanishing Gradients: As we backpropagate through many time steps, gradients can become extremely small (approach zero). This makes it hard for the RNN to learn long-range dependencies.

Why does this happen?

When computing $\frac{\partial h_t}{\partial h_{t-k}}$ (how does changing h_{t-k} affect h_t), we have:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=t-k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \quad (4.17)$$

This is a product of many terms. If each term is less than 1, the product shrinks exponentially:

- If $\frac{\partial h_i}{\partial h_{i-1}} \approx 0.9$, then after 10 steps: $0.9^{10} \approx 0.35$
- After 100 steps: $0.9^{100} \approx 0.0000027$ (essentially zero!)

Result: The RNN forgets information from many steps ago.

4.4.3 The Exploding Gradient Problem

Conversely, if the gradients are larger than 1, they can **explode**:

- If $\frac{\partial h_i}{\partial h_{i-1}} \approx 1.1$, then after 100 steps: $1.1^{100} \approx 13,780$

Solution: Gradient Clipping

Clip gradients if their norm exceeds a threshold:

$$\text{if } \|\mathbf{g}\| > \text{threshold}, \quad \mathbf{g} \leftarrow \frac{\text{threshold}}{\|\mathbf{g}\|} \cdot \mathbf{g} \quad (4.18)$$

Notation Explanation

Gradient Norm:

$\|\mathbf{g}\|$ is the **magnitude** (length) of the gradient vector:

$$\|\mathbf{g}\| = \sqrt{g_1^2 + g_2^2 + \cdots + g_n^2} \quad (4.19)$$

If the norm is too large, we scale down the gradient to prevent instability.

4.5 Long Short-Term Memory (LSTM)

To solve the vanishing gradient problem, **LSTM** (Hochreiter & Schmidhuber, 1997) introduces a sophisticated memory mechanism.

4.5.1 LSTM Architecture

An LSTM cell has:

- **Cell state** c_t : Long-term memory that flows through time with minimal modification
- **Hidden state** h_t : Short-term memory (output of the cell)
- **Three gates**: Control information flow
 - **Forget gate** f_t : Decides what to forget from cell state
 - **Input gate** i_t : Decides what new information to add
 - **Output gate** o_t : Decides what to output

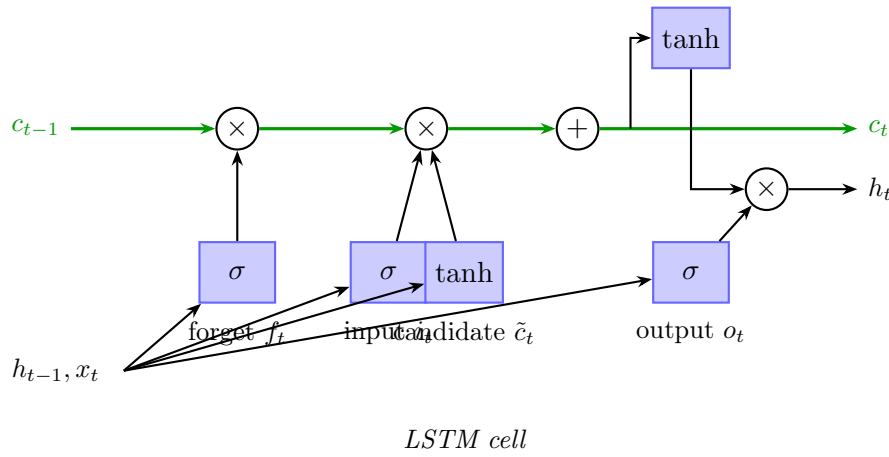


Figure 4.3: LSTM cell: the cell state $c_{t-1} \rightarrow c_t$ flows along the top with a forget multiply ($\times f_t$), an input update ($+ i_t \odot \tilde{c}_t$), then a tanh tap multiplied by the output gate to produce h_t .

4.5.2 LSTM Equations

At each time step, LSTM computes:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{forget gate}) \quad (4.20)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{input gate}) \quad (4.21)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{candidate cell state}) \quad (4.22)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{new cell state}) \quad (4.23)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{output gate}) \quad (4.24)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{new hidden state}) \quad (4.25)$$

Notation Explanation

LSTM Notation Explained:

- σ = sigmoid function (outputs 0 to 1, acts as a gate)

- $[h_{t-1}, x_t]$ = concatenation of vectors h_{t-1} and x_t
- $\odot$ = element-wise multiplication (Hadamard product)
- If $a = [1, 2, 3]$ and $b = [4, 5, 6]$, then $a \odot b = [4, 10, 18]$
- W_f, W_i, W_c, W_o = weight matrices for each gate
- b_f, b_i, b_c, b_o = bias vectors

How gates work:

- Sigmoid outputs close to 0: gate is closed (block information)
- Sigmoid outputs close to 1: gate is open (allow information through)
- $f_t \odot c_{t-1}$: selectively forget parts of old cell state
- $i_t \odot \tilde{c}_t$: selectively add new information

4.5.3 Why LSTM Solves Vanishing Gradients**Key Idea**

The cell state c_t flows through time with only minor linear interactions (additions). Gradients can flow backward through this "highway" without being multiplied many times. In vanilla RNN: $h_t = \tanh(W_{hh}h_{t-1} + \dots)$ (many matrix multiplications)
 In LSTM: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (mostly addition!)
 This allows LSTM to remember information for 100+ time steps.

4.5.4 Deep LSTM

Deep LSTM stacks multiple LSTM layers on top of each other, allowing the network to learn hierarchical representations.

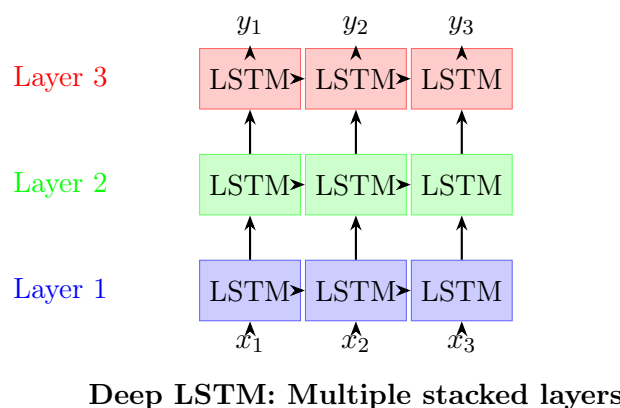


Figure 4.4: Deep LSTM with 3 layers: each layer's output becomes the next layer's input

Notation Explanation**Deep LSTM Architecture:**

For L layers:

Layer 1:

$$h_t^{[1]} = \text{LSTM}^{[1]}(x_t, h_{t-1}^{[1]}, c_{t-1}^{[1]}) \quad (4.26)$$

Layer 2:

$$h_t^{[2]} = \text{LSTM}^{[2]}(h_t^{[1]}, h_{t-1}^{[2]}, c_{t-1}^{[2]}) \quad (4.27)$$

Layer l (for $l = 2, \dots, L$):

$$h_t^{[l]} = \text{LSTM}^{[l]}(h_t^{[l-1]}, h_{t-1}^{[l]}, c_{t-1}^{[l]}) \quad (4.28)$$

Key points:

- Each layer has its own LSTM parameters
- Hidden state from layer l at time t becomes input to layer $l + 1$ at same time t
- Each layer also maintains its own temporal hidden states across time
- Bottom layer processes raw inputs, top layer produces final outputs

Key Idea

Why Deep LSTMs?

- **Hierarchical features:** Lower layers learn low-level patterns (words, phrases), higher layers learn high-level patterns (semantics, concepts)
- **Better representations:** Multiple layers of abstraction improve performance
- **More expressiveness:** Deep networks can represent more complex functions

Typical depths:

- Machine translation: 2-4 layers
- Speech recognition: 3-7 layers
- Too deep (>8 layers) can be hard to train and slow

Example 4.3. Deep LSTM for Machine Translation:

Input: "The cat sat on the mat"

Layer 1: Learns word-level patterns

- Recognizes "the" is an article
- "cat" is a noun
- "sat" is a past tense verb

Layer 2: Learns phrase-level patterns

- "The cat" is a noun phrase (subject)
- "sat on the mat" is a verb phrase
- Subject-verb-object structure

Layer 3: Learns sentence-level semantics

- Complete action: something sitting somewhere
- Temporal aspect: past tense
- Overall meaning representation

This hierarchical understanding improves translation quality!

4.6 Gated Recurrent Unit (GRU)

GRU (Cho et al., 2014) is a simpler alternative to LSTM with fewer gates.

4.6.1 GRU Equations

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (\text{update gate}) \quad (4.29)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (\text{reset gate}) \quad (4.30)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (\text{candidate hidden state}) \quad (4.31)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{new hidden state}) \quad (4.32)$$

Notation Explanation

GRU vs LSTM:

- **GRU:** 2 gates (update, reset), no separate cell state
- **LSTM:** 3 gates (forget, input, output), separate cell state c_t
- GRU has fewer parameters $\rightarrow$ faster training, less memory
- LSTM is more expressive $\rightarrow$ can model more complex patterns
- In practice: both work well, GRU often preferred for efficiency

4.7 Bidirectional RNNs

In many tasks, we want to look at future context as well as past context.

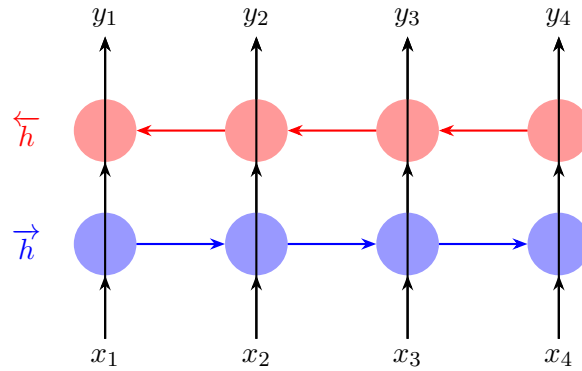
Intuition

Consider the sentence: "I grew up in France, so I speak fluent ____."
To predict the blank, looking only at past words is helpful, but looking ahead to see there's nothing after would also be useful. In offline processing (when we have the full sentence), we can process it in both directions!

4.7.1 Bidirectional RNN Architecture

A **bidirectional RNN** runs two RNNs:

- **Forward RNN:** Processes sequence left-to-right
- **Backward RNN:** Processes sequence right-to-left
- Outputs are concatenated: $h_t = [\vec{h}_t; \overleftarrow{h}_t]$



Bidirectional RNN

Figure 4.5: Bidirectional RNN: combines forward and backward contexts

4.8 Practice Problems: Chapter 4

4.8.1 Problem 1: Bigram Probability Calculation

Question: Given the following corpus:

"I like cats. I like dogs. I love cats."

Using a bigram model, calculate $P(\text{"like"}|\text{"I"})$ and $P(\text{"cats"}|\text{"like"})$ using Maximum Likelihood Estimation (MLE).

Solution:

Step 1: Tokenize and count bigrams

After tokenization: `<s> I like cats . </s> <s> I like dogs . </s> <s> I love cats . </s>`

Count occurrences:

- $\text{count}(\text{"I"}) = 3$
- $\text{count}(\text{"I"}, \text{"like"}) = 2$
- $\text{count}(\text{"I"}, \text{"love"}) = 1$
- $\text{count}(\text{"like"}) = 2$
- $\text{count}(\text{"like"}, \text{"cats"}) = 1$
- $\text{count}(\text{"like"}, \text{"dogs"}) = 1$

Step 2: Calculate $P(\text{"like"}|\text{"I"})$

Using MLE formula:

$$\begin{aligned} P(\text{"like"}|\text{"I"}) &= \frac{\text{count}(\text{"I"}, \text{"like"})}{\text{count}(\text{"I"})} \\ &= \frac{2}{3} \approx 0.667 \end{aligned}$$

Step 3: Calculate $P(\text{"cats"}|\text{"like"})$

$$\begin{aligned}
 P(\text{"cats"}|\text{"like"}) &= \frac{\text{count}(\text{"like"}, \text{"cats"})}{\text{count}(\text{"like"})} \\
 &= \frac{1}{2} = 0.5
 \end{aligned}$$

Answers:

- $P(\text{"like"}|\text{"I"}) = 0.667$
- $P(\text{"cats"}|\text{"like"}) = 0.5$

Interpretation: Given "I", the word "like" appears 2 out of 3 times in the corpus. Given "like", the words "cats" and "dogs" each appear once (50% probability each).

4.8.2 Problem 2: Perplexity Calculation

Question: A bigram language model assigns the following probabilities to a test sentence "I like cats":

- $P(\text{"I"}|\langle s \rangle) = 0.8$
- $P(\text{"like"}|\text{"I"}) = 0.6$
- $P(\text{"cats"}|\text{"like"}) = 0.4$
- $P(\langle s \rangle|\text{"cats"}) = 0.9$

Calculate the perplexity of this sentence.

Solution:

Step 1: Calculate sentence probability

For a bigram model:

$$\begin{aligned}
 P(\text{sentence}) &= P(w_1|\langle s \rangle) \cdot P(w_2|w_1) \cdot P(w_3|w_2) \cdot P(\langle s \rangle|w_3) \\
 &= 0.8 \times 0.6 \times 0.4 \times 0.9 \\
 &= 0.1728
 \end{aligned}$$

Step 2: Count total words (including sentence markers)

Total words $N = 4$ (I, like, cats, $\langle s \rangle$)

Step 3: Apply perplexity formula

$$\begin{aligned}
 \text{PPL} &= P(\text{sentence})^{-1/N} \\
 &= (0.1728)^{-1/4} \\
 &= (0.1728)^{-0.25}
 \end{aligned}$$

To calculate $(0.1728)^{-0.25}$:

$$\begin{aligned}
 (0.1728)^{-0.25} &= \frac{1}{(0.1728)^{0.25}} \\
 &= \frac{1}{0.647} \approx 1.545
 \end{aligned}$$

Answer: Perplexity ≈ 1.545

Interpretation: On average, the model is as "surprised" as if it had to choose uniformly from about 1.5 words at each step. Lower perplexity means better model fit.

4.8.3 Problem 3: Add-k Smoothing

Question: Consider a bigram model trained on a small corpus. For the bigram "dog barks":

- $\text{count}(\text{"dog"}) = 5$
- $\text{count}(\text{"dog"}, \text{"barks"}) = 0$ (unseen bigram!)
- Vocabulary size $V = 100$

Calculate $P(\text{"barks"}|\text{"dog"})$ using:

- Raw MLE (no smoothing)
- Add-1 smoothing (Laplace)

Solution:

Part (a): Raw MLE

$$P_{\text{MLE}}(\text{"barks"}|\text{"dog"}) = \frac{\text{count}(\text{"dog"}, \text{"barks"})}{\text{count}(\text{"dog"})} = \frac{0}{5} = 0$$

Problem: Zero probability for unseen bigram! Model assigns impossibility to a perfectly valid sentence.

Part (b): Add-1 Smoothing

Formula:

$$P_{\text{add-1}}(w_n|w_{n-1}) = \frac{\text{count}(w_{n-1}, w_n) + 1}{\text{count}(w_{n-1}) + V}$$

Applying:

$$\begin{aligned} P_{\text{add-1}}(\text{"barks"}|\text{"dog"}) &= \frac{0 + 1}{5 + 100} \\ &= \frac{1}{105} \approx 0.0095 \end{aligned}$$

Answers:

- (a) MLE: $P = 0$ (problematic!)
- (b) Add-1: $P \approx 0.0095$

Interpretation: Smoothing prevents zero probabilities by "stealing" probability mass from seen events and redistributing to unseen events. This allows the model to handle novel word combinations.

4.8.4 Problem 4: LSTM Forward Pass

Question: An LSTM cell at time t has:

- Input: $x_t = 0.5$
- Previous hidden state: $h_{t-1} = 0.6$
- Previous cell state: $c_{t-1} = 0.4$
- All weight matrices simplified to scalars: $W = 1.0$, $U = 0.5$
- All biases: $b = 0$

Calculate the forget gate f_t (assume forget gate weights/biases sum to 0.8 before sigmoid).

Solution:

Step 1: Recall forget gate formula

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

where σ is the sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$

Step 2: Compute pre-activation

Given that the combination results in 0.8:

$$z_f = W_f x_t + U_f h_{t-1} + b_f = 0.8$$

Step 3: Apply sigmoid

$$\begin{aligned} f_t &= \sigma(0.8) = \frac{1}{1 + e^{-0.8}} \\ &= \frac{1}{1 + 0.449} \\ &= \frac{1}{1.449} \approx 0.690 \end{aligned}$$

Answer: $f_t \approx 0.69$

Interpretation: The forget gate value of 0.69 means the LSTM will retain about 69% of the previous cell state c_{t-1} . This partial forgetting allows the network to balance between remembering long-term information and incorporating new inputs.

Step 4: Cell state update (for context)

If we also had input gate $i_t = 0.5$ and candidate $\tilde{c}_t = 0.3$:

$$\begin{aligned} c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ &= 0.69 \times 0.4 + 0.5 \times 0.3 \\ &= 0.276 + 0.15 = 0.426 \end{aligned}$$

4.8.5 Problem 5: Vanishing Gradient in RNN

Question: A vanilla RNN has recurrent weight $W_h = 0.8$ and uses tanh activation. When backpropagating through 10 time steps, estimate the gradient magnitude relative to the final time step.

Solution:

Step 1: Recall gradient flow in RNN

For vanilla RNN:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

When backpropagating from time T to time t :

$$\frac{\partial \mathcal{L}}{\partial h_t} \propto \prod_{k=t+1}^T W_h \cdot \tanh'(z_k)$$

Step 2: Bound tanh derivative

For tanh activation:

$$\tanh'(z) = 1 - \tanh^2(z) \leq 1$$

Maximum value is 1 (when $z = 0$), typically around 0.25-0.5 for most activations.

Let's assume average $\tanh'(z) \approx 0.5$.

Step 3: Calculate gradient attenuation

Over 10 time steps ($T - t = 10$):

$$\begin{aligned}\text{Gradient magnitude} &\propto (W_h \cdot \tanh')^{10} \\ &\approx (0.8 \times 0.5)^{10} \\ &= (0.4)^{10} \\ &\approx 0.0001048576\end{aligned}$$

Answer: Gradient reduces to approximately 10^{-4} (0.01%) of original magnitude

Interpretation: After backpropagating through just 10 time steps, the gradient has decayed to nearly zero. This is the **vanishing gradient problem**: gradients become too small for the network to learn long-term dependencies.

Why LSTM solves this:

LSTM's cell state provides an additive path:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Gradient flows through addition (not repeated multiplication):

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

If forget gates $f_t \approx 1$, gradients flow with minimal decay!

4.8.6 Problem 6: Sequence Probability with RNN

Question: An RNN language model outputs the following probability distributions at each time step for the sentence "the cat sat":

Time 1: $P(\cdot|\langle s \rangle)$: {the: 0.7, cat: 0.2, sat: 0.1}

Time 2: $P(\cdot|\text{the})$: {the: 0.1, cat: 0.6, sat: 0.3}

Time 3: $P(\cdot|\text{the cat})$: {the: 0.05, cat: 0.15, sat: 0.8}

Calculate the probability the model assigns to this sentence.

Solution:

Step 1: Recall RNN language model probability

Unlike bigrams which use only immediate context, RNN uses full history:

$$P(w_1, \dots, w_n) = \prod_{t=1}^n P(w_t | w_1, \dots, w_{t-1})$$

Step 2: Extract relevant probabilities

From the distributions:

- $P(\text{"the"}|\langle s \rangle) = 0.7$
- $P(\text{"cat"}|\text{"the"}) = 0.6$
- $P(\text{"sat"}|\text{"the cat"}) = 0.8$

Step 3: Calculate sentence probability

$$\begin{aligned}P(\text{"the cat sat"}) &= P(\text{"the"}|\langle s \rangle) \times P(\text{"cat"}|\text{"the"}) \\ &\quad \times P(\text{"sat"}|\text{"the cat"}) \\ &= 0.7 \times 0.6 \times 0.8 \\ &= 0.336\end{aligned}$$

Answer: $P(\text{"the cat sat"}) = 0.336$ or **33.6%**

Step 4: Compare with bigram model

A bigram model would calculate:

$$P_{\text{bigram}} = P(\text{"the"}) \times P(\text{"cat"}|\text{"the"}) \times P(\text{"sat"}|\text{"cat"})$$

Note: At time 3, bigram only uses "cat", but RNN uses full context "the cat".

Advantage of RNN: Captures long-range dependencies beyond fixed n-gram window.

4.9 Summary of Chapter 4

In this chapter, we learned:

- **Traditional N-gram language models:**

- N-grams approximate language probability using limited context
- Perplexity measures model quality (lower is better)
- Smoothing techniques handle unseen N-grams: Add-k, Backoff/Interpolation, Kneser-Ney
- Limitations: sparsity, no generalization, limited context

- **Recurrent Neural Networks:**

- RNNs process sequences one element at a time, maintaining hidden state
- Vanilla RNNs suffer from vanishing/exploding gradient problems
- LSTM introduces gating mechanisms to remember long-term dependencies
- Deep LSTM stacks multiple LSTM layers for hierarchical representations
- GRU is a simpler alternative to LSTM with fewer parameters
- Bidirectional RNNs process sequences in both directions

- Applications: language modeling, machine translation, sentiment analysis, speech recognition

In the next chapter, we'll see how **attention mechanisms** revolutionize sequence modeling by allowing the model to focus on relevant parts of the input.

Chapter 5

Attention Mechanisms

5.1 Motivation: The Bottleneck Problem

5.1.1 Seq2Seq Models

For tasks like machine translation, we use **sequence-to-sequence (Seq2Seq)** models:

- **Encoder:** RNN that reads the input sequence and produces a fixed-size vector
- **Decoder:** RNN that generates the output sequence from the encoder's vector

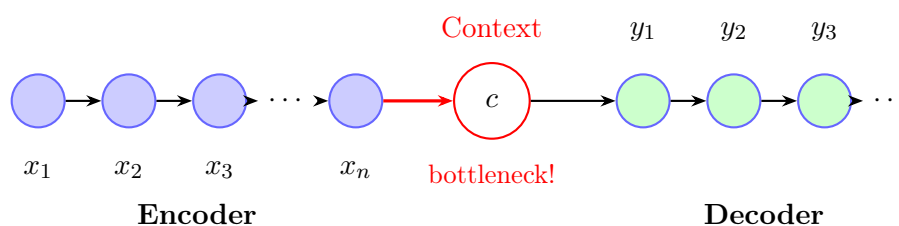


Figure 5.1: Seq2Seq model: entire input must pass through a single context vector

Key Idea

The Bottleneck Problem:

The encoder must compress the entire input sequence into a single fixed-size vector c . For long sequences:

- Information from early in the sequence gets "lost"
- The decoder has no direct access to specific input words
- Performance degrades significantly for long sentences

Solution: Let the decoder "attend" to different parts of the input at each decoding step!

5.2 Attention Mechanism (Bahdanau Attention)

5.2.1 The Core Idea

Instead of using a single context vector, we create a **different context vector for each decoder step** by taking a weighted combination of all encoder hidden states.

Intuition

Think of reading a difficult text. You don't read it once and memorize everything. Instead, when writing a summary, you constantly *refer back* to relevant parts of the original text. Attention allows the decoder to "look back" at the entire input and focus on relevant parts.

5.2.2 Attention Mathematical Formulation

At each decoder time step t , we compute:

Step 1: Compute attention scores

For each encoder hidden state h_i (where $i = 1, \dots, n$):

$$e_{t,i} = \text{score}(s_{t-1}, h_i) \quad (5.1)$$

Notation Explanation**Notation:**

- s_{t-1} = previous decoder hidden state
- h_i = encoder hidden state at position i
- $e_{t,i}$ = attention score (how relevant is h_i for generating y_t ?)
- score function examples:
 - Dot product: $\text{score}(s, h) = s^T h$
 - Additive: $\text{score}(s, h) = v^T \tanh(W_1 s + W_2 h)$

Step 2: Compute attention weights

Apply softmax to normalize scores into a probability distribution:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})} \quad (5.2)$$

Notation Explanation**Understanding Softmax:**

Softmax converts scores into probabilities that sum to 1:

- $\alpha_{t,i} \in [0, 1]$ (between 0 and 1)
- $\sum_{i=1}^n \alpha_{t,i} = 1$ (sums to 1)
- Higher scores $e_{t,i}$ lead to higher probabilities $\alpha_{t,i}$
- $\alpha_{t,i}$ represents "how much attention to pay to h_i "

Step 3: Compute context vector

Take a weighted sum of encoder hidden states:

$$c_t = \sum_{i=1}^n \alpha_{t,i} h_i \quad (5.3)$$

Step 4: Generate output

Use c_t along with previous decoder state to generate output:

$$s_t = \text{RNN}(s_{t-1}, [y_{t-1}; c_t]) \quad (5.4)$$

$$y_t = \text{softmax}(W_s s_t + W_c c_t) \quad (5.5)$$

5.2.3 Attention Visualization

Attention weights can be visualized as a heatmap showing which input words the model focuses on when generating each output word.

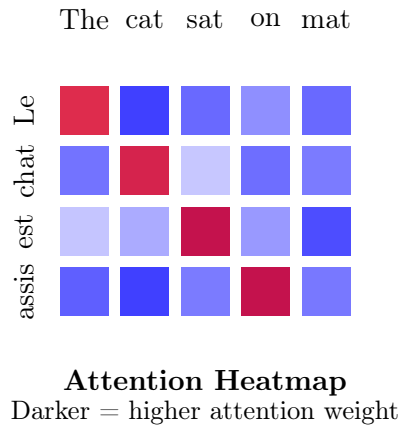


Figure 5.2: Attention weights show which input words are relevant for each output word

Worked Example

Numerical Attention Example:

Suppose we're translating "The cat sat" to French, and we're at the step of generating the second French word.

Encoder hidden states (simplified to 2D):

$$h_1 = [0.5, 0.3]^T \quad (\text{for "The"})$$

$$h_2 = [0.8, 0.6]^T \quad (\text{for "cat"})$$

$$h_3 = [0.2, 0.7]^T \quad (\text{for "sat"})$$

Previous decoder state:

$$s_1 = [0.6, 0.5]^T \quad (\text{after generating "Le"})$$

Step 1: Compute attention scores using dot product

$$e_{2,1} = s_1^T h_1 = [0.6, 0.5] \cdot [0.5, 0.3] = 0.6(0.5) + 0.5(0.3) = 0.45$$

$$e_{2,2} = s_1^T h_2 = [0.6, 0.5] \cdot [0.8, 0.6] = 0.6(0.8) + 0.5(0.6) = 0.78$$

$$e_{2,3} = s_1^T h_3 = [0.6, 0.5] \cdot [0.2, 0.7] = 0.6(0.2) + 0.5(0.7) = 0.47$$

Step 2: Apply softmax

First compute exponentials:

$$\exp(e_{2,1}) = \exp(0.45) \approx 1.57$$

$$\exp(e_{2,2}) = \exp(0.78) \approx 2.18$$

$$\exp(e_{2,3}) = \exp(0.47) \approx 1.60$$

Sum: $1.57 + 2.18 + 1.60 = 5.35$

Attention weights:

$$\begin{aligned}\alpha_{2,1} &= \frac{1.57}{5.35} \approx 0.29 \quad (29\% \text{ attention to "The"}) \\ \alpha_{2,2} &= \frac{2.18}{5.35} \approx 0.41 \quad (41\% \text{ attention to "cat"}) \\ \alpha_{2,3} &= \frac{1.60}{5.35} \approx 0.30 \quad (30\% \text{ attention to "sat"})\end{aligned}$$

Step 3: Compute context vector

$$\begin{aligned}c_2 &= \alpha_{2,1}h_1 + \alpha_{2,2}h_2 + \alpha_{2,3}h_3 \\ &= 0.29 \begin{bmatrix} 0.5 \\ 0.3 \end{bmatrix} + 0.41 \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix} + 0.30 \begin{bmatrix} 0.2 \\ 0.7 \end{bmatrix} \\ &= \begin{bmatrix} 0.145 \\ 0.087 \end{bmatrix} + \begin{bmatrix} 0.328 \\ 0.246 \end{bmatrix} + \begin{bmatrix} 0.06 \\ 0.21 \end{bmatrix} \\ &= \begin{bmatrix} 0.533 \\ 0.543 \end{bmatrix}\end{aligned}$$

The context vector c_2 is now a weighted combination, focusing most on "cat" (41%) since we're likely generating "chat" (French for cat).

5.3 Self-Attention

5.3.1 From Encoder-Decoder Attention to Self-Attention

In encoder-decoder attention, the decoder attends to the encoder. In **self-attention**, a sequence attends to *itself*.

Intuition

When reading the word "it" in a sentence, we look back to earlier words to determine what "it" refers to.

Example: "The cat sat on the mat. It was comfortable."

To understand "It", we need to pay attention to "cat" (not "mat") from earlier in the sentence.

5.3.2 Self-Attention Mechanism

For each position i in the sequence, self-attention computes a weighted sum of all positions.

Notation Explanation

Query, Key, Value (QKV) Framework:

Self-attention uses three learned projections of each input:

- **Query** (Q): "What am I looking for?"
- **Key** (K): "What do I contain?"
- **Value** (V): "What information do I actually provide?"

For input x_i :

$$q_i = W_Q x_i \quad (\text{query}) \quad (5.6)$$

$$k_i = W_K x_i \quad (\text{key}) \quad (5.7)$$

$$v_i = W_V x_i \quad (\text{value}) \quad (5.8)$$

where W_Q, W_K, W_V are learned weight matrices.

5.3.3 Scaled Dot-Product Attention

The attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5.9)$$

Notation Explanation

Breaking Down the Formula:

- Q = query matrix (all queries stacked)
- K = key matrix (all keys stacked)
- V = value matrix (all values stacked)
- QK^T = matrix of dot products between all query-key pairs
- d_k = dimension of keys (size of key vectors)
- $\sqrt{d_k}$ = scaling factor to prevent large dot products
- Softmax is applied row-wise (each query gets a probability distribution over keys)

Step-by-step:

1. Compute QK^T : similarity scores between queries and keys
2. Scale by $\frac{1}{\sqrt{d_k}}$: prevents gradients from vanishing
3. Apply softmax: converts scores to probabilities
4. Multiply by V : weighted sum of values

Worked Example

Numerical Self-Attention Example:

Suppose we have a sequence of 3 words with embedding dimension 4. For simplicity, let's use $d_k = 2$ (key dimension = 2).

Input embeddings:

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \quad \begin{array}{l} (\text{word 1}) \\ (\text{word 2}) \\ (\text{word 3}) \end{array}$$

Weight matrices (simplified):

$$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Step 1: Compute Q, K, V

$$Q = XW_Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 2 & 3 \end{bmatrix}$$

$$K = XW_K = \begin{bmatrix} 1 & 2 \\ 3 & 0 \\ 3 & 2 \end{bmatrix}, \quad V = XW_V = \begin{bmatrix} 2 & 0 \\ 0 & 3 \\ 2 & 2 \end{bmatrix}$$

Step 2: Compute attention scores QK^T

$$QK^T = \begin{bmatrix} 2 & 1 \\ 0 & 3 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 3 & 3 \\ 2 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 6 & 8 \\ 6 & 0 & 6 \\ 8 & 6 & 12 \end{bmatrix}$$

Each entry (i, j) represents how much position i should attend to position j .

Step 3: Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.4142$

$$\frac{QK^T}{\sqrt{d_k}} \approx \begin{bmatrix} 2.83 & 4.24 & 5.66 \\ 4.24 & 0.00 & 4.24 \\ 5.66 & 4.24 & 8.49 \end{bmatrix}$$

Step 4: Apply softmax (row-wise)

For the first row: $[2.83, 4.24, 5.66]$

Exponentials: $[\exp(2.83), \exp(4.24), \exp(5.66)] \approx [16.95, 69.41, 286.75]$

Sum: $16.95 + 69.41 + 286.75 \approx 373.11$

Softmax: $[0.05, 0.19, 0.77]$ (rows sum to 1.00 before rounding).

This means position 1 attends mostly to position 3 ($\approx 77\%$).

Computing softmax for all rows:

$$\text{Attention weights} \approx \begin{bmatrix} 0.05 & 0.19 & 0.77 \\ 0.50 & 0.01 & 0.50 \\ 0.06 & 0.01 & 0.93 \end{bmatrix}$$

(Row 2: scores $[4.24, 0, 4.24]$ are symmetric in positions 1 and 3, so attention splits roughly 50–50 between them, with negligible weight on position 2.)

Step 5: Multiply by V

$$\text{Output} = \begin{bmatrix} 0.05 & 0.19 & 0.77 \\ 0.50 & 0.01 & 0.50 \\ 0.06 & 0.01 & 0.93 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 3 \\ 2 & 2 \end{bmatrix} \approx \begin{bmatrix} 1.63 & 2.10 \\ 1.99 & 1.01 \\ 1.97 & 1.90 \end{bmatrix}$$

Each output row is a context-aware representation of that position!

5.3.4 Why Scaling by $\sqrt{d_k}$?

Key Idea

When d_k is large, dot products $q \cdot k$ can become very large in magnitude.

Example: If q and k are random vectors of dimension $d_k = 512$:

- Expected magnitude of $q \cdot k \propto \sqrt{d_k} \approx 22.6$
- Softmax of large values becomes close to one-hot (extremely peaky)
- Gradients become very small (saturation)

Dividing by $\sqrt{d_k}$ normalizes the dot products to have unit variance, keeping the softmax in a good range for gradients.

5.4 Multi-Head Attention

Instead of performing attention once, we perform it multiple times in parallel with different learned projections.

5.4.1 Multi-Head Attention Formula

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (5.10)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (5.11)$$

Notation Explanation

Understanding Multi-Head Attention:

- h = number of attention heads (e.g., 8 or 16)
- Each head has its own W_i^Q, W_i^K, W_i^V matrices
- Each head learns to focus on different aspects of the input
- Outputs are concatenated and linearly transformed by W^O

Why multiple heads?

- Different heads can attend to different types of relationships
- Example: One head might focus on syntactic relationships, another on semantic similarity
- Provides richer representations than single-head attention

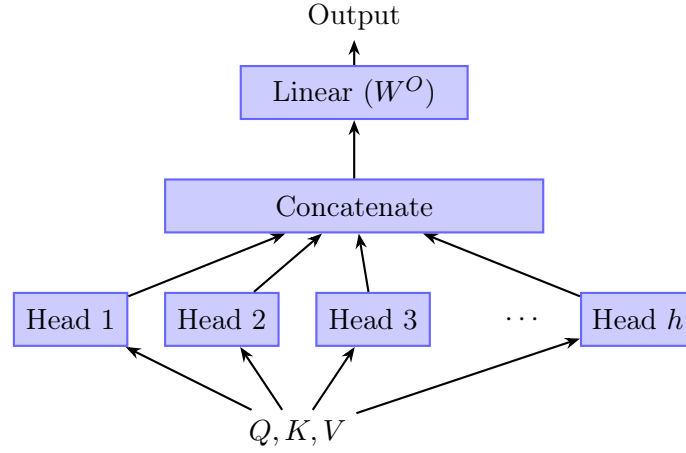


Figure 5.3: Multi-head attention: multiple attention operations in parallel

5.5 Practice Problems: Chapter 5

5.5.1 Problem 1: Attention Score Calculation

Question: In a Bahdanau attention mechanism, the decoder hidden state is $s_t = [0.5, 0.3]$ and encoder hidden states are:

- $h_1 = [0.2, 0.8]$
- $h_2 = [0.6, 0.4]$
- $h_3 = [0.1, 0.7]$

Using a simplified scoring function $e_{ij} = s_t^T h_j$ (dot product), calculate the attention scores and attention weights using softmax.

Solution:

Step 1: Calculate raw attention scores (energies)

$$\begin{aligned} e_1 &= s_t^T h_1 = [0.5, 0.3] \cdot [0.2, 0.8] \\ &= 0.5(0.2) + 0.3(0.8) = 0.1 + 0.24 = 0.34 \end{aligned}$$

$$\begin{aligned} e_2 &= s_t^T h_2 = [0.5, 0.3] \cdot [0.6, 0.4] \\ &= 0.5(0.6) + 0.3(0.4) = 0.3 + 0.12 = 0.42 \end{aligned}$$

$$\begin{aligned} e_3 &= s_t^T h_3 = [0.5, 0.3] \cdot [0.1, 0.7] \\ &= 0.5(0.1) + 0.3(0.7) = 0.05 + 0.21 = 0.26 \end{aligned}$$

Step 2: Apply softmax to get attention weights

$$\alpha_j = \frac{\exp(e_j)}{\sum_{k=1}^3 \exp(e_k)}$$

Calculate exponentials:

$$\exp(e_1) = \exp(0.34) \approx 1.405$$

$$\exp(e_2) = \exp(0.42) \approx 1.522$$

$$\exp(e_3) = \exp(0.26) \approx 1.297$$

Sum: $\sum \exp(e_k) = 1.405 + 1.522 + 1.297 = 4.224$

Attention weights:

$$\begin{aligned}\alpha_1 &= \frac{1.405}{4.224} \approx 0.333 \\ \alpha_2 &= \frac{1.522}{4.224} \approx 0.360 \\ \alpha_3 &= \frac{1.297}{4.224} \approx 0.307\end{aligned}$$

Answers:

- Attention scores: $e = [0.34, 0.42, 0.26]$
- Attention weights: $\alpha \approx [0.333, 0.360, 0.307]$

Verification: $0.333 + 0.360 + 0.307 = 1.0 \checkmark$

Interpretation: The decoder pays most attention to h_2 (36%), which had the highest raw score. All positions receive non-zero weight, allowing soft alignment.

5.5.2 Problem 2: Context Vector Computation

Question: Using the attention weights from Problem 1, compute the context vector c_t as a weighted sum of encoder hidden states.

Solution:

Step 1: Recall context vector formula

$$c_t = \sum_{j=1}^3 \alpha_j h_j$$

Step 2: Calculate weighted sum

$$\begin{aligned}c_t &= \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 \\ &= 0.333 \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix} + 0.360 \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} + 0.307 \begin{bmatrix} 0.1 \\ 0.7 \end{bmatrix}\end{aligned}$$

First dimension:

$$\begin{aligned}c_t[0] &= 0.333(0.2) + 0.360(0.6) + 0.307(0.1) \\ &= 0.0666 + 0.216 + 0.0307 = 0.313\end{aligned}$$

Second dimension:

$$\begin{aligned}c_t[1] &= 0.333(0.8) + 0.360(0.4) + 0.307(0.7) \\ &= 0.2664 + 0.144 + 0.2149 = 0.625\end{aligned}$$

Answer: Context vector $c_t \approx [0.313, 0.625]$

Interpretation: The context vector is a weighted mixture of all encoder hidden states, emphasizing the most relevant information for the current decoding step.

5.5.3 Problem 3: Scaled Dot-Product Attention

Question: In Transformer-style scaled dot-product attention, given:

- Query: $q = [1, 2, 0]$
- Keys: $K = \begin{bmatrix} 1 & 0 & 1 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix}$ (3 keys as rows)
- Dimension: $d_k = 3$

Calculate the attention scores using the scaled dot-product formula.

Solution:

Step 1: Recall scaled dot-product formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

We'll focus on computing $\frac{qK^T}{\sqrt{d_k}}$.

Step 2: Compute qK^T (dot products)

$$qK^T[0] = q \cdot K_0 = [1, 2, 0] \cdot [1, 0, 1] = 1(1) + 2(0) + 0(1) = 1$$

$$qK^T[1] = q \cdot K_1 = [1, 2, 0] \cdot [2, 1, 0] = 1(2) + 2(1) + 0(0) = 4$$

$$qK^T[2] = q \cdot K_2 = [1, 2, 0] \cdot [0, 2, 1] = 1(0) + 2(2) + 0(1) = 4$$

So $qK^T = [1, 4, 4]$

Step 3: Scale by $\sqrt{d_k}$

$$\sqrt{d_k} = \sqrt{3} \approx 1.732$$

$$\begin{aligned} \text{Scaled scores} &= \frac{qK^T}{\sqrt{d_k}} = \frac{[1, 4, 4]}{1.732} \\ &\approx [0.577, 2.309, 2.309] \end{aligned}$$

Step 4: Apply softmax

$$\exp(0.577) \approx 1.781$$

$$\exp(2.309) \approx 10.064$$

$$\exp(2.309) \approx 10.064$$

$$\text{Sum: } 1.781 + 10.064 + 10.064 = 21.909$$

$$\begin{aligned} \alpha_0 &= \frac{1.781}{21.909} \approx 0.081 \\ \alpha_1 &= \frac{10.064}{21.909} \approx 0.459 \\ \alpha_2 &= \frac{10.064}{21.909} \approx 0.459 \end{aligned}$$

Answer: Attention weights $\approx [0.081, 0.459, 0.459]$

Why scaling matters: Without scaling (dividing by $\sqrt{d_k}$), dot products grow large for high dimensions, pushing softmax into regions with tiny gradients. Scaling stabilizes training.

5.5.4 Problem 4: Multi-Head Attention Dimensions

Question: A multi-head attention layer has:

- Model dimension: $d_{\text{model}} = 512$
- Number of heads: $h = 8$

Calculate:

- Dimension per head d_k
- Total number of parameters in the query, key, value projection matrices

Solution:

Part (a): Dimension per head

$$d_k = d_v = \frac{d_{\text{model}}}{h} = \frac{512}{8} = 64$$

Answer (a): Each head operates in 64 dimensions

Part (b): Parameter count

Each head needs three projection matrices:

- W^Q : maps $d_{\text{model}} \rightarrow d_k$, size 512×64
- W^K : maps $d_{\text{model}} \rightarrow d_k$, size 512×64
- W^V : maps $d_{\text{model}} \rightarrow d_v$, size 512×64

Parameters per head:

$$3 \times (512 \times 64) = 3 \times 32,768 = 98,304$$

Total for all 8 heads:

$$8 \times 98,304 = 786,432$$

Plus final output projection W^O : $512 \times 512 = 262,144$

Total parameters:

$$786,432 + 262,144 = 1,048,576$$

Answer (b): 1,048,576 parameters

Note: We're not counting biases here. If biases are included, add $4 \times 512 = 2,048$ more parameters.

5.5.5 Problem 5: Self-Attention vs Cross-Attention

Question: Explain the difference between self-attention and cross-attention with a concrete example from machine translation.

Solution:

Self-Attention:

Definition: Query, Key, and Value all come from the *same* sequence.

Example - Encoder Self-Attention:

Input sentence (English): "The cat sat"

Each word attends to *all words in the same sentence*:

- "sat" queries: How relevant is each word to "sat"?
- Might attend strongly to "cat" (subject) and "the" (modifier)

- $Q = K = V =$ encoder representations

Purpose: Capture intra-sentence relationships and context.

Cross-Attention (Encoder-Decoder Attention):

Definition: Query comes from one sequence, Keys and Values from another.

Example - Decoder attending to Encoder:

When generating French word "assis" (sat):

- **Query:** Current decoder state for "assis"
- **Keys & Values:** Encoder representations of "The cat sat"
- Decoder asks: "Which English words should I focus on for this French word?"
- Might attend strongly to "sat"

Mathematical Formulation:

Self-Attention (Encoder):

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

where X is input sequence.

Cross-Attention (Decoder):

$$Q = YW^Q \quad (\text{from decoder})$$

$$K = XW^K \quad (\text{from encoder})$$

$$V = XW^V \quad (\text{from encoder})$$

where Y is decoder sequence, X is encoder output.

Key Difference:

- **Self-attention:** Q, K, V from same source $\rightarrow$ models relationships *within* a sequence
- **Cross-attention:** Q from target, K/V from source $\rightarrow$ models relationships *between* sequences

Visual Summary:

Self-Attention: English $\rightarrow$ English relationships

Cross-Attention: French (being generated) $\rightarrow$ English (source) relationships

5.6 Summary of Chapter 5

In this chapter, we learned:

- Seq2Seq models have a bottleneck: entire input compressed into one vector
- Attention allows the model to focus on relevant parts of the input
- Bahdanau attention: decoder attends to encoder hidden states
- Self-attention: sequence attends to itself
- Query-Key-Value framework: flexible attention mechanism

- Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$
- Scaling by $\sqrt{d_k}$ prevents gradient saturation
- Multi-head attention: parallel attention with different learned projections

Attention mechanisms are the foundation for **Transformers**, which we'll study in the next chapter!

Chapter 6

The Transformer Architecture

6.1 Motivation: Getting Rid of Recurrence

6.1.1 Limitations of RNNs

While RNNs with attention work well, they have fundamental limitations:

- **Sequential processing:** Must process tokens one at a time (can't parallelize)
- **Long-range dependencies:** Even with LSTM/GRU, very long sequences are challenging
- **Training time:** Sequential nature makes training slow on modern hardware (GPUs/TPUs)

Key Idea

The Transformer Insight:

"What if we get rid of recurrence entirely and rely solely on attention?"

The Transformer ("Attention Is All You Need", Vaswani et al., 2017) does exactly this:

- No RNNs or CNNs
- Pure attention-based architecture
- Fully parallelizable
- Achieves state-of-the-art results

6.2 Transformer Architecture Overview

The Transformer has an encoder-decoder structure, but both use only attention and feed-forward layers.

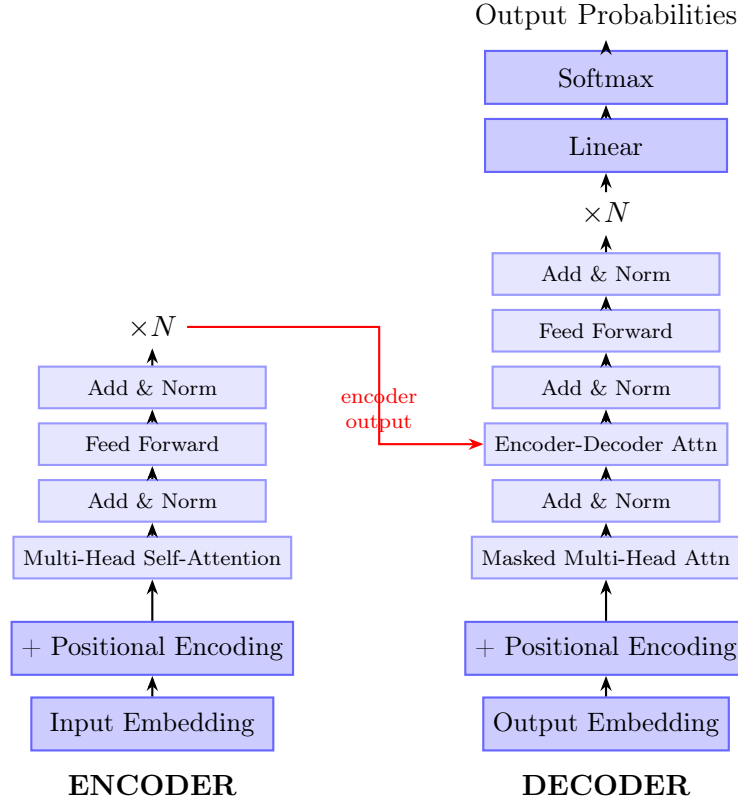


Figure 6.1: Transformer architecture: encoder–decoder with stacked attention and feed-forward sub-layers, each followed by Add & Norm. The decoder also attends to the encoder output via encoder–decoder (cross) attention.

6.3 Positional Encoding

6.3.1 The Problem: No Sequential Information

Unlike RNNs, the Transformer processes all tokens simultaneously. But this means it has **no notion of order!**

Example 6.1. Without positional information:

- "The cat sat on the mat"
- "Mat the on sat cat the"

These would be treated identically because self-attention is permutation-invariant.

6.3.2 The Solution: Positional Encodings

We add a **positional encoding** to each input embedding that encodes the position information.

Definition 6.1 (Positional Encoding). For position pos and dimension i :

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (6.1)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (6.2)$$

where:

- pos = position in sequence (0, 1, 2, ...)
- i = dimension index (0, 1, 2, ..., $d_{model}/2 - 1$)
- d_{model} = embedding dimension (e.g., 512)
- Even dimensions use sine, odd dimensions use cosine

Notation Explanation

Understanding the Formula:

- The formula uses sinusoidal functions at different frequencies
- Lower dimensions (small i) have high frequency (change quickly)
- Higher dimensions (large i) have low frequency (change slowly)
- This creates a unique "fingerprint" for each position
- The pattern allows the model to learn relative positions

Why sinusoids?

- They're bounded (between -1 and 1)
- The model can learn to attend to relative positions using linear transformations
- Can extrapolate to sequence lengths longer than seen during training

Worked Example

Numerical Positional Encoding Example:

Let's compute positional encodings for $d_{model} = 4$ (simplified).

For position $pos = 0$ (first word):

$$PE_{(0,0)} = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$

$$PE_{(0,1)} = \cos\left(\frac{0}{10000^{0/4}}\right) = \cos(0) = 1$$

$$PE_{(0,2)} = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$

$$PE_{(0,3)} = \cos\left(\frac{0}{10000^{2/4}}\right) = \cos(0) = 1$$

So $PE_0 = [0, 1, 0, 1]^T$

For position $pos = 1$ (second word):

$$PE_{(1,0)} = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) \approx 0.84$$

$$PE_{(1,1)} = \cos\left(\frac{1}{10000^{0/4}}\right) = \cos(1) \approx 0.54$$

$$PE_{(1,2)} = \sin\left(\frac{1}{10000^{1/2}}\right) = \sin(0.01) \approx 0.01$$

$$PE_{(1,3)} = \cos\left(\frac{1}{10000^{1/2}}\right) = \cos(0.01) \approx 1.00$$

So $PE_1 \approx [0.84, 0.54, 0.01, 1.00]^T$

These encodings are added to the word embeddings:

$$\text{input}_i = \text{word_embedding}_i + PE_i$$

6.4 Encoder Layer

Each encoder layer has two sub-layers:

6.4.1 1. Multi-Head Self-Attention

The input sequence attends to itself.

$$\text{Attention output} = \text{MultiHead}(X, X, X) \quad (6.3)$$

where X is the input (queries, keys, and values all come from the same source).

6.4.2 2. Position-wise Feed-Forward Network

After attention, each position goes through the same feed-forward network independently:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (6.4)$$

Notation Explanation

Feed-Forward Network:

- Two linear transformations with ReLU in between
- Applied to each position independently (same weights for all positions)
- Typical dimensions: input/output $d_{\text{model}} = 512$, hidden layer $d_{\text{ff}} = 2048$
- Adds non-linearity and allows the model to transform representations

6.4.3 Residual Connections and Layer Normalization

Each sub-layer has a **residual connection** followed by **layer normalization**:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x)) \quad (6.5)$$

Notation Explanation

Residual Connections:

- Add the input x to the output of the sub-layer: $x + \text{Sublayer}(x)$
- Allows gradients to flow directly through the network
- Same idea as ResNet skip connections

Layer Normalization:

Normalizes activations across the feature dimension:

$$\text{LayerNorm}(x) = \gamma \frac{x - \mu}{\sigma} + \beta \quad (6.6)$$

where:

- μ = mean of x across features
- σ = standard deviation of x across features
- γ, β = learned scale and shift parameters

This stabilizes training and allows for deeper networks.

6.5 Decoder Layer

The decoder is similar to the encoder but with an additional attention layer.

6.5.1 1. Masked Multi-Head Self-Attention

The decoder uses **masked self-attention** to prevent attending to future positions.

Key Idea

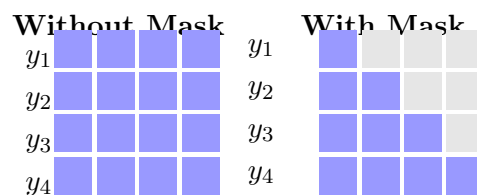
Why masking?

During training, we have the full target sequence. But when generating, we only have access to previously generated tokens.

Masking ensures the model learns to generate autoregressively:

$$P(y_1, y_2, \dots, y_n) = P(y_1) \cdot P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdots \quad (6.7)$$

At position i , we mask out (set to $-\infty$) all positions $j > i$ before the softmax.



Gray = masked (cannot attend)

Figure 6.2: Masking in decoder self-attention: can only attend to previous positions

6.5.2 2. Encoder-Decoder Attention

The decoder attends to the encoder output:

- **Queries** come from the decoder
- **Keys and Values** come from the encoder output

$$\text{Attn output} = \text{MultiHead}(Q_{\text{decoder}}, K_{\text{encoder}}, V_{\text{encoder}}) \quad (6.8)$$

This allows each decoder position to attend to all encoder positions (the entire input sequence).

6.5.3 3. Position-wise Feed-Forward Network

Same as in the encoder.

6.6 Complete Transformer Forward Pass

Let's trace through a complete example:

Worked Example

Translation Example: "The cat" $\rightarrow$ "Le chat"

ENCODER:

Step 1: Input embeddings + positional encoding

- "The" $\rightarrow$ embedding e_1 , add PE_0
- "cat" $\rightarrow$ embedding e_2 , add PE_1

Input matrix: $X_{enc} = [e_1 + PE_0, e_2 + PE_1]$

Step 2: Encoder layer 1

1. Multi-head self-attention: each word attends to both words
2. Add & Norm: $X_1 = \text{LayerNorm}(X_{enc} + \text{Attn}(X_{enc}))$
3. Feed-forward: independently process each position
4. Add & Norm: $X'_1 = \text{LayerNorm}(X_1 + \text{FFN}(X_1))$

Step 3: Encoder layers 2 through N

Repeat the same structure, output is X_{enc_out}

DECODER:

Step 1: Start with [START] token

- Input: [START]
- Embedding + positional encoding: y_0

Step 2: Decoder layer 1

1. Masked self-attention: [START] attends only to itself
2. Add & Norm
3. Encoder-decoder attention: [START] attends to "The" and "cat" from encoder
4. Add & Norm
5. Feed-forward
6. Add & Norm

Output: y'_1

Step 3: Decoder layers 2 through N

Repeat, final output: y_{dec_out}

Step 4: Linear + Softmax

$$\text{logits} = y_{dec_out} W_{vocab}$$

where W_{vocab} maps to vocabulary size.

$$P(\text{next word}) = \text{softmax}(\text{logits})$$

Highest probability: "Le"

Step 5: Generate next word

Now input is [START, "Le"], and we repeat the decoder process to predict "chat".

This continues until we generate an [END] token.

6.7 Training the Transformer

6.7.1 Teacher Forcing

During training, we use **teacher forcing**:

- Feed the true target sequence to the decoder (not the generated one)
- Compute loss at each position simultaneously
- This allows parallel training (huge speedup over RNNs!)

6.7.2 Loss Function

Cross-entropy loss at each position:

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t^* | y_1^*, \dots, y_{t-1}^*, X) \quad (6.9)$$

where y_t^* is the true target token at position t .

6.8 Why Transformers Work So Well

Key Idea

Key Advantages of Transformers:

1. **Parallelization:** All positions processed simultaneously (unlike RNNs)
2. **Long-range dependencies:** Direct connections between any two positions (path length = 1)
3. **Flexibility:** Attention can learn arbitrary dependencies
4. **Scalability:** Architecture scales to billions of parameters
5. **Transfer learning:** Pre-trained transformers transfer well to new tasks

Computational Complexity:

- Self-attention: $O(n^2 \cdot d)$ where n = sequence length, d = dimension
- RNN: $O(n \cdot d^2)$ but sequential (can't parallelize)
- For short sequences and high dimensionality, Transformers are much faster

6.9 Transformer Hyperparameters

Typical configuration (Transformer Base):

Hyperparameter	Value
Number of encoder layers	6
Number of decoder layers	6
Model dimension (d_{model})	512
Feed-forward dimension (d_{ff})	2048
Number of attention heads	8
Dimension per head ($d_k = d_v$)	64
Dropout rate	0.1
Total parameters	~65 million

Table 6.1: Transformer Base configuration (Vaswani et al., 2017)

Transformer Big uses $d_{model} = 1024$, $d_{ff} = 4096$, 16 heads, and has ~213 million parameters.

6.10 Practice Problems: Chapter 6

6.10.1 Problem 1: Positional Encoding

Question: Calculate the positional encoding for position $\text{pos} = 1$ (second token) at dimensions $i = 0$ and $i = 1$, given $d_{model} = 512$.

Solution:

Step 1: Recall positional encoding formula

For even dimensions (i even):

$$PE_{(\text{pos}, i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{model}}}\right)$$

For odd dimensions (i odd):

$$PE_{(\text{pos}, i)} = \cos\left(\frac{\text{pos}}{10000^{(2\lfloor i/2 \rfloor)/d_{model}}}\right)$$

Step 2: Calculate for dimension $i = 0$ (even)

$$\begin{aligned}
 PE_{(1,0)} &= \sin\left(\frac{1}{10000^{2(0)/512}}\right) \\
 &= \sin\left(\frac{1}{10000^0}\right) \\
 &= \sin\left(\frac{1}{1}\right) \\
 &= \sin(1) \approx 0.8415
 \end{aligned}$$

Step 3: Calculate for dimension $i = 1$ (odd)

For $i = 1$, we use $\lfloor 1/2 \rfloor = 0$:

$$\begin{aligned} PE_{(1,1)} &= \cos\left(\frac{1}{10000^{2(0)/512}}\right) \\ &= \cos\left(\frac{1}{1}\right) \\ &= \cos(1) \approx 0.5403 \end{aligned}$$

Answers:

- $PE_{(1,0)} \approx 0.8415$
- $PE_{(1,1)} \approx 0.5403$

Step 4: Calculate for dimension $i = 2$ (for comparison)

$$\begin{aligned} PE_{(1,2)} &= \sin\left(\frac{1}{10000^{2(1)/512}}\right) \\ &= \sin\left(\frac{1}{10000^{2/512}}\right) \\ &= \sin\left(\frac{1}{10000^{0.00391}}\right) \\ &= \sin\left(\frac{1}{1.0363}\right) \approx \sin(0.965) \approx 0.8242 \end{aligned}$$

Interpretation: Different dimensions use different frequencies. Lower dimensions (like $i = 0, 1$) use high frequencies that change rapidly across positions. Higher dimensions use lower frequencies that change slowly, capturing longer-range position information.

6.10.2 Problem 2: Feed-Forward Network Computation

Question: A Transformer feed-forward network has:

- Input dimension: $d_{\text{model}} = 4$
- Hidden dimension: $d_{ff} = 8$
- Input vector: $x = [0.5, 0.2, 0.8, 0.1]$

Given simplified weights:

- $W_1 = 4 \times 8$ matrix (for illustration, assume first row is $[1, 0, 1, 0, 1, 0, 1, 0]$)
- $b_1 = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$
- ReLU activation

Calculate the first element of the hidden layer output.

Solution:**Step 1: Recall FFN formula**

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

We'll focus on the first step: computing the first hidden unit.

Step 2: Compute pre-activation for first hidden unit

Given first row of W_1 : $[1, 0, 1, 0, 1, 0, 1, 0]$

Actually, let's clarify: x has dimension 4, W_1 should be 4×8 . The first *column* of W_1 determines the first hidden unit.

Let's say first column of $W_1 = [1, 0.5, 0.2, 0.3]^T$:

$$\begin{aligned} z_1 &= x \cdot W_1[:, 0] + b_1[0] \\ &= [0.5, 0.2, 0.8, 0.1] \cdot [1, 0.5, 0.2, 0.3] + 0.1 \\ &= 0.5(1) + 0.2(0.5) + 0.8(0.2) + 0.1(0.3) + 0.1 \\ &= 0.5 + 0.1 + 0.16 + 0.03 + 0.1 \\ &= 0.89 \end{aligned}$$

Step 3: Apply ReLU

$$h_1 = \text{ReLU}(0.89) = \max(0, 0.89) = 0.89$$

Answer: First hidden unit output = 0.89

Key insight about FFN:

The feed-forward network in Transformers:

- Processes each position *independently* (no interaction between positions)
- Expands to higher dimension ($d_{ff} = 4 \times d_{\text{model}}$ typically)
- Projects back down to d_{model}
- Acts as a position-wise fully connected layer

6.10.3 Problem 3: Masked Self-Attention

Question: In a decoder self-attention layer, we have 3 tokens with attention scores (before softmax):

$$\text{Scores} = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \\ 3 & 2 & 5 \end{bmatrix}$$

where element (i, j) is the attention from token i to token j .

Apply causal masking (tokens can't attend to future positions) and compute attention weights for token 2 (middle row).

Solution:

Step 1: Understand causal masking

Token i can only attend to positions $j \leq i$ (itself and previous tokens).

Set future positions to $-\infty$ before softmax:

$$\text{Masked Scores} = \begin{bmatrix} 2 & -\infty & -\infty \\ 1 & 4 & -\infty \\ 3 & 2 & 5 \end{bmatrix}$$

Step 2: Extract row for token 2

Token 2 (third row, index 2): $[3, 2, 5]$ (can attend to all previous + itself)

Step 3: Apply softmax

$$\exp(3) \approx 20.09$$

$$\exp(2) \approx 7.39$$

$$\exp(5) \approx 148.41$$

$$\text{Sum: } 20.09 + 7.39 + 148.41 = 175.89$$

$$\alpha_{2,0} = \frac{20.09}{175.89} \approx 0.114$$

$$\alpha_{2,1} = \frac{7.39}{175.89} \approx 0.042$$

$$\alpha_{2,2} = \frac{148.41}{175.89} \approx 0.844$$

Answer: Attention weights for token 2: [0.114, 0.042, 0.844]

Step 4: Verify token 1 (for understanding)

Token 1 (second row): [1, 4, $-\infty$]

Only attends to positions 0 and 1:

$$\exp(1) \approx 2.72$$

$$\exp(4) \approx 54.60$$

$$\text{Sum} = 57.32$$

$$\alpha_{1,0} = \frac{2.72}{57.32} \approx 0.047$$

$$\alpha_{1,1} = \frac{54.60}{57.32} \approx 0.953$$

$$\alpha_{1,2} = 0 \quad (\text{masked})$$

Purpose of masking: Ensures autoregressive property during training—when predicting token i , the model cannot "cheat" by looking at future tokens.

6.10.4 Problem 4: Transformer Parameter Count

Question: Calculate the number of parameters in one Transformer encoder layer with:

- $d_{\text{model}} = 512$
- $h = 8$ heads
- $d_{\text{ff}} = 2048$

Include: multi-head attention, feed-forward network, and layer normalization parameters.

Solution:

Step 1: Multi-head attention parameters

Q, K, V projections:

- Each: $d_{\text{model}} \times d_{\text{model}} = 512 \times 512 = 262,144$
- Three matrices: $3 \times 262,144 = 786,432$
- Biases: $3 \times 512 = 1,536$

Output projection W^O :

- Weight: $512 \times 512 = 262,144$
- Bias: 512

Attention total: $786,432 + 1,536 + 262,144 + 512 = 1,050,624$

Step 2: Feed-forward network parameters

First linear layer:

- Weight W_1 : $512 \times 2048 = 1,048,576$
- Bias b_1 : 2048

Second linear layer:

- Weight W_2 : $2048 \times 512 = 1,048,576$
- Bias b_2 : 512

FFN total: $1,048,576 + 2048 + 1,048,576 + 512 = 2,099,712$

Step 3: Layer normalization parameters

Two layer norms (after attention and after FFN):

- Each has: scale (γ) and shift (β) of size $d_{\text{model}} = 512$
- Total: $2 \times 2 \times 512 = 2,048$

Step 4: Sum all parameters

$$\begin{aligned} \text{Total} &= \text{Attention} + \text{FFN} + \text{LayerNorm} \\ &= 1,050,624 + 2,099,712 + 2,048 \\ &= 3,152,384 \end{aligned}$$

Answer: One encoder layer has approximately 3.15 million parameters

For full Transformer Base (6 encoder + 6 decoder layers):

Encoder layers: $6 \times 3,152,384 = 18,914,304$

Decoder layers are similar (with extra cross-attention): ~ 20 million per layer $\times 6 \approx 120$ million

Plus embedding layers: vocab size $\times d_{\text{model}}$

Total: ~ 65 million parameters for Transformer Base

6.10.5 Problem 5: Comparing Transformer to RNN Complexity

Question: Compare the computational complexity per layer for:

- RNN processing a sequence of length n with dimension d
- Transformer self-attention on the same sequence

Assume $n = 1000$ and $d = 512$.

Solution:

Part (a): RNN Complexity

For each time step t :

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Operations per time step:

- $W_h h_{t-1}$: $d \times d$ multiply $\rightarrow O(d^2)$
- $W_x x_t$: $d \times d$ multiply $\rightarrow O(d^2)$
- Total per step: $O(d^2)$

For sequence of length n :

$$\text{RNN complexity} = O(n \cdot d^2)$$

Sequential dependency: Cannot parallelize across time steps!

With $n = 1000$, $d = 512$:

$$\text{Operations} \sim 1000 \times 512^2 = 1000 \times 262,144 = 262,144,000$$

Part (b): Self-Attention Complexity

Step 1: Compute QK^T

Q and K are both $n \times d$. Matrix multiplication QK^T produces $n \times n$:

$$\text{Operations} = O(n^2 \cdot d)$$

Step 2: Softmax

$O(n^2)$ for $n \times n$ matrix

Step 3: Multiply with V

$(n \times n) \times (n \times d) = n \times d$ output:

$$\text{Operations} = O(n^2 \cdot d)$$

Total self-attention complexity:

$$O(n^2 \cdot d)$$

Fully parallelizable! All positions computed simultaneously.

With $n = 1000$, $d = 512$:

$$\text{Operations} \sim 1000^2 \times 512 = 1,000,000 \times 512 = 512,000,000$$

Comparison:

Model	Complexity	Parallelizable?
RNN	$O(n \cdot d^2)$	No (sequential)
Self-Attention	$O(n^2 \cdot d)$	Yes (fully parallel)

Trade-off:

- For short sequences ($n < d$): Self-attention faster
- For long sequences ($n \gg d$): Self-attention becomes expensive (n^2 scaling!)
- RNN: Linear in n , but sequential bottleneck
- Transformer: Quadratic in n , but massively parallel

Answer:

- RNN: $\sim 262\text{M}$ operations, sequential
- Transformer: $\sim 512\text{M}$ operations, parallel

Due to parallelization on GPUs, Transformers are typically **much faster in practice** despite higher operation count!

6.11 Summary of Chapter 6

In this chapter, we learned:

- Transformers eliminate recurrence, using only attention
- Positional encodings inject sequence order information
- Encoder: multi-head self-attention + feed-forward
- Decoder: masked self-attention + encoder-decoder attention + feed-forward
- Residual connections and layer normalization stabilize deep networks
- Masking prevents decoder from seeing future tokens
- Teacher forcing enables parallel training
- Transformers excel at long-range dependencies and parallelize well

The Transformer architecture is the foundation for all modern LLMs, which we'll study next!

Part III

Modern Large Language Models and Generative AI

Chapter 7

Modern Language Model Architectures

7.1 From Transformers to Large Language Models

7.1.1 What is a Language Model?

Definition 7.1 (Language Model). A language model assigns probabilities to sequences of words (or tokens). It models:

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdots P(w_n|w_1, \dots, w_{n-1}) \quad (7.1)$$

This allows the model to:

- Generate text
- Evaluate how likely a sequence is
- Perform tasks like translation, summarization, question answering

7.1.2 Pre-training and Fine-tuning Paradigm

Modern NLP follows a two-stage approach:

1. **Pre-training:** Train on massive unlabeled text (learn general language understanding)
2. **Fine-tuning:** Adapt to specific tasks with smaller labeled datasets

Key Idea

Why this paradigm works:

- Pre-training learns general linguistic knowledge: grammar, facts, reasoning patterns
- Fine-tuning specializes this knowledge for specific tasks
- Much more data-efficient than training from scratch
- Same pre-trained model can be fine-tuned for many tasks

7.2 BERT: Bidirectional Encoder Representations from Transformers

7.2.1 Key Idea

BERT uses the **encoder-only** part of the Transformer, trained with **bidirectional context**.

Intuition

Traditional language models are unidirectional (left-to-right): predicting next words using only previous context.

BERT is bidirectional: uses both left and right context simultaneously. This gives much richer representations!

Example: "The bank of the river"

- Unidirectional sees: "The bank" (could be financial institution)
- Bidirectional sees: "The bank of the river" (clearly geographical)

7.2.2 BERT Pre-training Objectives

BERT is pre-trained with two objectives:

1. Masked Language Modeling (MLM)

- Randomly mask 15% of input tokens
- Model must predict the masked tokens using surrounding context

Example 7.1. Original: "The cat sat on the mat"

Masked: "The [MASK] sat on the mat"

BERT predicts: [MASK] = "cat"

Notation Explanation

Masking Strategy:

For each selected token (15% of all tokens):

- 80% of the time: replace with [MASK]
- 10% of the time: replace with a random word
- 10% of the time: keep original word

This prevents the model from learning that [MASK] only appears during training.

2. Next Sentence Prediction (NSP)

- Given two sentences A and B
- Predict whether B actually follows A in the text

Example 7.2. Positive pair (label = 1):

Sentence A: "The man went to the store."

Sentence B: "He bought a gallon of milk."

Negative pair (label = 0):

Sentence A: "The man went to the store."
Sentence B: "Penguins are flightless birds."

7.2.3 BERT Architecture Details

Input Representation:

Each token gets three embeddings summed together:

1. **Token embedding:** Word piece embedding
2. **Segment embedding:** Indicates which sentence (A or B)
3. **Position embedding:** Position in the sequence (learned, not sinusoidal)

Special tokens:

- [CLS]: Classification token (first token, used for sequence-level tasks)
- [SEP]: Separator between sentences
- [MASK]: Masked token

Model sizes:

- BERT-Base: 12 layers, 768 hidden, 12 heads, 110M parameters
- BERT-Large: 24 layers, 1024 hidden, 16 heads, 340M parameters

7.2.4 Fine-tuning BERT

For downstream tasks, we add a task-specific layer on top of BERT and fine-tune:

- **Classification:** Use [CLS] token representation
- **Question Answering:** Predict start and end positions in context
- **Named Entity Recognition:** Classify each token

7.3 GPT: Generative Pre-trained Transformer

7.3.1 Key Idea

GPT uses the **decoder-only** part of the Transformer, trained autoregressively (left-to-right).

Key Idea

GPT vs BERT:

- **BERT:** Encoder-only, bidirectional, fill-in-the-blank style
- **GPT:** Decoder-only, unidirectional, predict-next-token style
- BERT is better for understanding tasks (classification, QA)
- GPT is better for generation tasks (text completion, creative writing)

7.3.2 GPT Training Objective

GPT is trained to maximize the likelihood of the next token:

$$\mathcal{L} = \sum_{i=1}^n \log P(w_i | w_1, \dots, w_{i-1}) \quad (7.2)$$

This is standard language modeling: predict the next word given all previous words.

7.3.3 GPT Architecture

- Stack of decoder blocks (without encoder-decoder attention)
- Each block has:
 - Masked self-attention (causal mask: can only attend to past)
 - Feed-forward network
 - Layer normalization and residual connections
- Final layer: linear + softmax over vocabulary

7.3.4 GPT Evolution

1. **GPT-1 (2018)**: 117M parameters, demonstrated transfer learning for NLP
2. **GPT-2 (2019)**: 1.5B parameters, showed impressive zero-shot performance
3. **GPT-3 (2020)**: 175B parameters, few-shot learning, emergent abilities
4. **GPT-4 (2023)**: Multimodal, rumored 1.7T parameters, further improvements

Key Idea

Scaling Laws:

Larger models consistently perform better:

- More parameters $\rightarrow$ better performance (diminishing returns but still improving)
- More training data $\rightarrow$ better performance
- More compute $\rightarrow$ better performance

This observation led to the era of "Large Language Models" (LLMs).

7.4 T5: Text-to-Text Transfer Transformer

7.4.1 Unified Text-to-Text Framework

T5 treats **every NLP task as text-to-text**:

Example 7.3. Translation:

Input: "translate English to German: The house is wonderful."

Output: "Das Haus ist wunderbar."

Summarization:

Input: "summarize: [long article]"

Output: [summary]

Classification (sentiment):

Input: "sentiment: This movie is great!"

Output: "positive"

7.4.2 Architecture

T5 uses the full encoder-decoder Transformer (like the original):

- Encoder processes input
- Decoder generates output

7.4.3 Pre-training

T5 uses **span corruption**:

- Mask random spans (consecutive tokens)
- Model predicts the masked content

Example 7.4. Original: "Thank you for inviting me to your party last week"

Masked: "Thank you [X] me to your party [Y] week"

Target: "[X] for inviting [Y] last"

7.5 BART: Bidirectional and Autoregressive Transformer

7.5.1 Combining BERT and GPT

BART combines ideas from both BERT and GPT:

- Encoder (bidirectional like BERT)
- Decoder (autoregressive like GPT)

7.5.2 Denoising Pre-training

BART is trained to reconstruct original text from corrupted input:

- Token masking (like BERT)
- Token deletion
- Text infilling (mask spans like T5)
- Sentence permutation (shuffle sentences)
- Document rotation (rotate to start from random token)

This diverse corruption makes BART robust and good at generation.

7.6 Retrieval-Augmented Generation (RAG)

7.6.1 The Problem: Hallucination

LLMs sometimes generate plausible-sounding but incorrect information (**hallucination**).

Example 7.5. Question: "When did Neil Armstrong land on Mars?"

LLM (hallucinating): "Neil Armstrong landed on Mars in 1969."

(Wrong! Armstrong landed on the Moon, not Mars, and no human has been to Mars)

7.6.2 RAG Solution

Definition 7.2 (Retrieval-Augmented Generation). RAG combines a language model with a retrieval system:

1. **Retrieve:** Find relevant documents from a knowledge base
2. **Augment:** Provide retrieved documents as context to the LLM
3. **Generate:** LLM generates answer conditioned on retrieved context

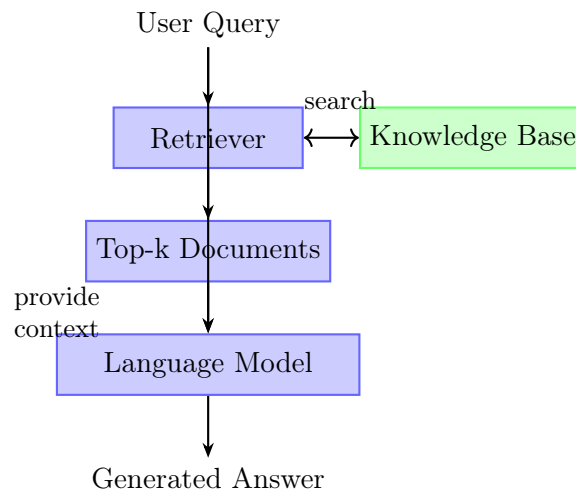


Figure 7.1: RAG architecture: retrieve relevant documents, then generate answer

7.6.3 RAG Benefits

- **Reduces hallucination:** Grounds generation in retrieved facts
- **Up-to-date information:** Knowledge base can be updated without retraining LLM
- **Verifiable:** Can trace answer back to source documents
- **Domain-specific:** Can use domain-specific knowledge bases

7.7 Decoding Strategies

When generating text, we need to select tokens from the probability distribution. Different strategies give different results.

7.7.1 Greedy Decoding

Always pick the highest probability token:

$$w_t = \underset{w}{\operatorname{argmax}} P(w|w_1, \dots, w_{t-1}) \quad (7.3)$$

- **Pros:** Fast, deterministic
- **Cons:** Can miss better overall sequences, repetitive, boring

7.7.2 Beam Search

Keep top- k most likely sequences at each step:

Notation Explanation

Beam Search Algorithm:

1. Start with k beams (initially just [START] token)
2. At each step:
 - For each beam, consider all possible next tokens
 - Compute score for each extended sequence
 - Keep top- k sequences by score
3. Continue until all beams produce [END] or max length
4. Return highest-scoring complete sequence

Score: Usually log probability

$$\text{score}(w_1, \dots, w_n) = \sum_{i=1}^n \log P(w_i | w_1, \dots, w_{i-1}) \quad (7.4)$$

Common beam sizes: $k = 5$ or $k = 10$

- **Pros:** Finds better sequences than greedy, commonly used
- **Cons:** Still can be repetitive, computationally expensive, less diverse

7.7.3 Sampling Methods

Sample from the probability distribution instead of taking max:

1. Temperature Sampling

Adjust the probability distribution with temperature τ :

$$P_{\tau}(w|\text{context}) = \frac{\exp(\text{logit}_w/\tau)}{\sum_{w'} \exp(\text{logit}_{w'}/\tau)} \quad (7.5)$$

Notation Explanation

Temperature Effects:

- $\tau = 1$: Original distribution
- $\tau < 1$ (e.g., 0.7): "Cooler", more confident, less random
 - High-probability words become even more likely
 - More focused, factual generation
- $\tau > 1$ (e.g., 1.5): "Hotter", less confident, more random
 - Distribution becomes flatter
 - More diverse, creative generation

- $\tau \rightarrow 0$: Approaches greedy (always pick max)
- $\tau \rightarrow \infty$: Uniform distribution (completely random)

2. Top-k Sampling

Sample from only the top- k most likely words:

- Filter to top- k words
- Renormalize probabilities
- Sample from this reduced distribution

Common values: $k = 40$ or $k = 50$

3. Top-p (Nucleus) Sampling

Sample from the smallest set of words whose cumulative probability exceeds p :

1. Sort words by probability
2. Keep adding words until cumulative probability $\geq p$
3. Sample from this set

Example 7.6. If $p = 0.9$:

- "the": 40%
- "a": 25%
- "this": 15%
- "that": 12%
- ...

Cumulative: $40\% + 25\% + 15\% + 12\% = 92\% \geq 90\%$

Sample from {"the", "a", "this", "that"}

Common value: $p = 0.9$ or $p = 0.95$

- **Pros:** Adapts to distribution shape, good diversity
- **Cons:** Can include too many low-probability words in flat distributions

7.8 Practice Problems: Chapter 7

7.8.1 Problem 1: BERT Masked Language Modeling

Question: In BERT pre-training, the sentence "The cat sat on the mat" is prepared for MLM. Token "sat" is replaced with [MASK]. If BERT's output distribution at the [MASK] position is:

- sat: 0.65
- was: 0.15

- is: 0.10
- ran: 0.05
- stood: 0.05

- (a) What is the MLM loss for this example?
- (b) What would happen if we masked "the" instead? Why is this challenging?

Solution:

Part (a): MLM Loss Calculation

Step 1: Recall MLM loss

Cross-entropy loss for masked token:

$$\mathcal{L}_{\text{MLM}} = -\log P(\text{true token}|\text{context})$$

Step 2: Calculate loss

True token is "sat" with predicted probability 0.65:

$$\begin{aligned}\mathcal{L}_{\text{MLM}} &= -\log(0.65) \\ &= -\log(0.65) \approx -(-0.431) = 0.431\end{aligned}$$

Answer (a): Loss ≈ 0.431

Interpretation: Lower loss means better prediction. If model predicted probability 1.0 for "sat", loss would be 0 (perfect). Probability 0.5 gives loss ≈ 0.693 .

Part (b): Masking "the"

If we mask: "[MASK] cat sat on the mat"

Challenges:

1. **Multiple valid answers:** "A", "The", "This", "My", etc. are all grammatically correct
2. **Function words:** "The" is a function word with less semantic content than content words like "sat"
3. **Context-dependent:** Harder to predict without semantic clues
4. BERT would likely give more uniform distribution (higher perplexity) because multiple determiners fit

Answer (b): Masking function words like "the" is harder because:

- Multiple valid replacements exist
- Less semantic information to leverage
- Model uncertainty increases (more uniform probability distribution)

This is why BERT learns both syntactic (function words) and semantic (content words) understanding.

7.8.2 Problem 2: GPT Autoregressive Generation

Question: GPT-2 is generating text with the prompt "The weather is". At each step, it produces probability distributions:

Step 1: $P(\cdot \mid \text{"The weather is"})$: {sunny: 0.4, rainy: 0.3, cold: 0.2, hot: 0.1}

Step 2: $P(\cdot \mid \text{"The weather is sunny"})$: {and: 0.5, .: 0.25, today: 0.15, ,: 0.10}

Using greedy decoding, generate the next 2 tokens. Then calculate the sequence probability.

Solution:

Step 1: Greedy decoding at step 1

Choose token with highest probability:

$$\arg \max P(\cdot \mid \text{"The weather is"}) = \text{"sunny"} \quad (0.4)$$

Step 2: Greedy decoding at step 2

Given "The weather is sunny", choose:

$$\arg \max P(\cdot \mid \text{"The weather is sunny"}) = \text{"and"} \quad (0.5)$$

Answer: Generated sequence = "sunny and"

Step 3: Calculate sequence probability

For GPT (autoregressive):

$$\begin{aligned} P(\text{"sunny and"} \mid \text{prompt}) &= P(\text{"sunny"} \mid \text{prompt}) \\ &\quad \times P(\text{"and"} \mid \text{prompt} + \text{"sunny"}) \\ &= 0.4 \times 0.5 = 0.2 \end{aligned}$$

Answer: Sequence probability = 0.2 or 20%

Limitation of greedy decoding:

Greedy might miss better overall sequences. For example:

- Greedy: "sunny" (0.4) $\rightarrow$ "and" (0.5) = 0.20 total
- Alternative: "rainy" (0.3) $\rightarrow$ "." (0.4 hypothetically) = 0.12 total

But locally optimal choices don't guarantee globally optimal sequences! Beam search explores multiple paths.

7.8.3 Problem 3: Temperature Sampling

Question: A language model outputs logits [2.0, 1.0, 0.5] for three candidate tokens. Calculate the probability distribution after applying:

- Temperature $T = 1.0$ (standard softmax)
- Temperature $T = 0.5$ (low temperature, more deterministic)
- Temperature $T = 2.0$ (high temperature, more random)

Solution:

Temperature-scaled softmax:

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Part (a): $T = 1.0$

$$\exp(2.0) \approx 7.389$$

$$\exp(1.0) \approx 2.718$$

$$\exp(0.5) \approx 1.649$$

$$\text{Sum: } 7.389 + 2.718 + 1.649 = 11.756$$

$$P_1 = \frac{7.389}{11.756} \approx 0.628$$

$$P_2 = \frac{2.718}{11.756} \approx 0.231$$

$$P_3 = \frac{1.649}{11.756} \approx 0.140$$

Answer (a): $[0.628, 0.231, 0.140]$

Part (b): $T = 0.5$ (sharper distribution)

$$\exp(2.0/0.5) = \exp(4.0) \approx 54.598$$

$$\exp(1.0/0.5) = \exp(2.0) \approx 7.389$$

$$\exp(0.5/0.5) = \exp(1.0) \approx 2.718$$

$$\text{Sum: } 54.598 + 7.389 + 2.718 = 64.705$$

$$P_1 = \frac{54.598}{64.705} \approx 0.844$$

$$P_2 = \frac{7.389}{64.705} \approx 0.114$$

$$P_3 = \frac{2.718}{64.705} \approx 0.042$$

Answer (b): $[0.844, 0.114, 0.042]$

Part (c): $T = 2.0$ (flatter distribution)

$$\exp(2.0/2.0) = \exp(1.0) \approx 2.718$$

$$\exp(1.0/2.0) = \exp(0.5) \approx 1.649$$

$$\exp(0.5/2.0) = \exp(0.25) \approx 1.284$$

$$\text{Sum: } 2.718 + 1.649 + 1.284 = 5.651$$

$$P_1 = \frac{2.718}{5.651} \approx 0.481$$

$$P_2 = \frac{1.649}{5.651} \approx 0.292$$

$$P_3 = \frac{1.284}{5.651} \approx 0.227$$

Answer (c): $[0.481, 0.292, 0.227]$

Comparison:

Key observations:

Temperature	Token 1	Token 2	Token 3
$T = 0.5$ (low)	0.844	0.114	0.042
$T = 1.0$ (standard)	0.628	0.231	0.140
$T = 2.0$ (high)	0.481	0.292	0.227

- Low T : Sharpens distribution, more deterministic (token 1 dominates)
- High T : Flattens distribution, more diverse (all tokens closer in probability)
- $T \rightarrow 0$: Approaches greedy (argmax)
- $T \rightarrow \infty$: Approaches uniform distribution

7.8.4 Problem 4: Top-k Sampling

Question: Given a probability distribution over 6 tokens:

{a: 0.35, b: 0.25, c: 0.15, d: 0.10, e: 0.08, f: 0.07}

Apply top-k sampling with $k = 3$. What are the renormalized probabilities for sampling?

Solution:

Step 1: Select top-k tokens

Top 3 by probability:

- a: 0.35
- b: 0.25
- c: 0.15

Set others to zero: d, e, f $\rightarrow$ probability 0

Step 2: Renormalize

Sum of top-k probabilities: $0.35 + 0.25 + 0.15 = 0.75$

Renormalized probabilities:

$$P'(a) = \frac{0.35}{0.75} \approx 0.467$$

$$P'(b) = \frac{0.25}{0.75} \approx 0.333$$

$$P'(c) = \frac{0.15}{0.75} = 0.200$$

Answer: Renormalized distribution for sampling:

- a: 0.467
- b: 0.333
- c: 0.200
- d, e, f: 0 (excluded)

Verification: $0.467 + 0.333 + 0.200 = 1.0$ ✓

Effect: Tokens d, e, f are never sampled, preventing low-quality outputs while maintaining diversity among top candidates.

7.8.5 Problem 5: Nucleus (Top-p) Sampling

Question: Using the same distribution from Problem 4, apply nucleus sampling with $p = 0.75$. Which tokens are included?

Distribution: {a: 0.35, b: 0.25, c: 0.15, d: 0.10, e: 0.08, f: 0.07}

Solution:

Step 1: Sort tokens by probability (already sorted)

Step 2: Accumulate probabilities until sum $\geq p$

After token a: $0.35 < 0.75$ (include a, continue)

After token b: $0.35 + 0.25 = 0.60 < 0.75$ (include b, continue)

After token c: $0.60 + 0.15 = 0.75 \geq 0.75$ (include c, STOP)

Step 3: Nucleus set

Included tokens: {a, b, c}

Step 4: Renormalize

Sum: $0.35 + 0.25 + 0.15 = 0.75$

$$P'(a) = \frac{0.35}{0.75} \approx 0.467$$

$$P'(b) = \frac{0.25}{0.75} \approx 0.333$$

$$P'(c) = \frac{0.15}{0.75} = 0.200$$

Answer: Nucleus sampling includes {a, b, c} with probabilities [0.467, 0.333, 0.200]

Comparison with Top-k:

For this distribution, top-k ($k = 3$) and top-p ($p = 0.75$) give the same result! But consider a different distribution:

Peaked distribution: {a: 0.90, b: 0.05, c: 0.03, d: 0.02}

- Top-k ($k = 3$): includes a, b, c (total prob 0.98) — unnecessarily includes low-prob c
- Top-p ($p = 0.90$): includes only a (90%) — more focused

Flat distribution: {a: 0.20, b: 0.19, c: 0.18, d: 0.17, e: 0.16, f: 0.10}

- Top-k ($k = 3$): includes a, b, c (total 0.57) — cuts off too early
- Top-p ($p = 0.75$): includes a, b, c, d (total 0.74) — more diverse

Advantage of nucleus: Adapts to distribution shape!

7.8.6 Problem 6: BERT vs GPT Architecture

Question: Compare BERT and GPT-2 on the following dimensions:

- Architecture (encoder/decoder)
- Pre-training objective
- Contextual information
- Suitable tasks

Dimension	BERT	GPT-2
Architecture	Encoder-only (stack of encoder layers with self-attention)	Decoder-only (stack of decoder layers with masked self-attention)
Pre-training Objective	(1) Masked Language Modeling (MLM): predict [MASK] tokens (2) Next Sentence Prediction (NSP)	Causal Language Modeling: predict next token autoregressively
Contextual Info	Bidirectional: Sees entire sentence in both directions (past + future context)	Unidirectional: Sees only left context (past tokens), future masked
Suitable Tasks	Understanding tasks: Classification, NER, Question Answering, Sentence similarity	Generation tasks: Text generation, dialogue, completion, creative writing
Output	Contextual embeddings for each token (no inherent generation capability)	Probability distribution over vocabulary for next token

Solution:

Detailed Comparison:

(a) Architecture:

- **BERT:** 12-24 encoder layers, bidirectional self-attention
- **GPT:** 12-48 decoder layers, causal (masked) self-attention

(b) Pre-training:

- **BERT MLM:** "The [MASK] is red" → predict "cat" using context from both sides
- **GPT Causal LM:** "The cat" → predict "is" using only left context

(c) Contextual Information:

- **BERT:** For "bank" in "river bank", sees both "river" (left) and context after (right)
- **GPT:** For "bank", only sees "river" (left context), not what comes after

(d) Task Examples:

- **BERT excels:** Sentiment classification ("This movie is [MASK]" → "excellent"), Named Entity Recognition
- **GPT excels:** Story generation ("Once upon a time" → continue story), Code completion

Key Insight: BERT trades off generation capability for better bidirectional understanding. GPT maintains autoregressive structure for generation but only sees left context.

7.9 Summary of Chapter 7

In this chapter, we learned:

- Pre-training and fine-tuning paradigm for modern NLP
- BERT: Bidirectional encoder, masked language modeling, next sentence prediction
- GPT: Autoregressive decoder, predict next token, scales to 100B+ parameters

- T5: Unified text-to-text framework for all NLP tasks
- BART: Combines bidirectional encoder with autoregressive decoder
- RAG: Retrieval-augmented generation reduces hallucination
- Decoding strategies: Greedy, beam search, temperature sampling, top-k, top-p

Next, we'll explore how to effectively use these models through prompting and instruction tuning!

Chapter 8

Prompting and Instruction Tuning

8.1 The Power of Prompting

8.1.1 What is Prompting?

Definition 8.1 (Prompt). A prompt is the input text given to a language model to elicit a desired response. It provides context, instructions, or examples for the task.

Key Idea

With large enough pre-trained models (like GPT-3), we can solve tasks **without fine-tuning** just by designing good prompts. This is called **in-context learning**.

8.2 Types of Prompting

8.2.1 Zero-Shot Prompting

Give the model a task with just instructions, no examples:

Example 8.1. Prompt:

"Classify the sentiment of this review as positive or negative.

Review: This movie was absolutely terrible and a waste of time.

Sentiment:"

Model output:

"negative"

- **Pros:** No examples needed, quick
- **Cons:** Performance can be inconsistent, model might not understand the task format

8.2.2 Few-Shot Prompting

Provide a few examples before the actual query:

Example 8.2. Prompt:

"Classify the sentiment of movie reviews.

Review: I loved every minute! The acting was superb. Sentiment: positive

Review: Boring and predictable. Don't waste your money. Sentiment: negative

Review: One of the best films I've seen this year! Sentiment: positive

Review: The plot made no sense and the dialogue was awful. Sentiment:"

Model output:

"negative"

- **Pros:** Better performance than zero-shot, model learns format from examples
- **Cons:** Requires good examples, uses more tokens (costs more)

8.2.3 One-Shot Prompting

Special case of few-shot with exactly one example:

Example 8.3. Prompt:

"Translate English to French.

English: Hello, how are you? French: Bonjour, comment allez-vous?

English: The weather is nice today. French:"

Model output:

"Il fait beau aujourd'hui."

8.3 Chain-of-Thought (CoT) Prompting

8.3.1 The Idea

For complex reasoning tasks, prompt the model to show its reasoning steps.

Key Idea

Adding "Let's think step by step" or showing reasoning examples dramatically improves performance on math, logic, and reasoning tasks.

8.3.2 Zero-Shot CoT

Simply add "Let's think step by step" to the prompt:

Example 8.4. Without CoT:

"Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: 11"

(Wrong! Correct answer is 11, but let's see with CoT...)

With Zero-Shot CoT:

"Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Let's think step by step.

Roger starts with 5 tennis balls. He buys 2 cans. Each can has 3 balls. So he buys $2 \times 3 = 6$ tennis balls. In total: $5 + 6 = 11$ tennis balls.

The answer is 11."

8.3.3 Few-Shot CoT

Provide examples with reasoning steps:

Example 8.5. Prompt:

"Q: There are 15 trees in the grove. Grove workers will plant trees in the grove today. After they are done, there will be 21 trees. How many did they plant?

A: There are 15 trees originally. Then there were 21 trees after some more were planted. So they must have planted $21 - 15 = 6$ trees. The answer is 6.

Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?

A: There are originally 3 cars. 2 more cars arrive. $3 + 2 = 5$. The answer is 5.

Q: Jason had 20 lollipops. He gave some to Denny. Now Jason has 12 lollipops. How many lollipops did Jason give to Denny?

A:"

Model output:

"Jason had 20 lollipops originally. Now he has 12. So he gave away $20 - 12 = 8$ lollipops. The answer is 8."

Notation Explanation

Why CoT Works:

- Forces the model to break down complex problems
- Intermediate steps reduce the "reasoning gap"
- Model can check its own logic along the way
- More interpretable (we can see the reasoning)
- Particularly effective for:
 - Math word problems
 - Multi-step reasoning
 - Common sense reasoning
 - Logical deduction

8.4 Advanced Prompting Techniques

8.4.1 Self-Consistency

1. Generate multiple reasoning paths (using sampling)
2. Take the majority vote of final answers

Example 8.6. Generate 5 answers with different reasoning:

- Path 1 → Answer: 24
- Path 2 → Answer: 24
- Path 3 → Answer: 18
- Path 4 → Answer: 24
- Path 5 → Answer: 24

Majority vote: 24 (appears 4 out of 5 times)

8.4.2 Tree of Thoughts

Explore multiple reasoning branches like a search tree:

1. Generate multiple possible next steps
2. Evaluate each step

3. Expand the most promising paths
4. Backtrack if needed

More systematic than self-consistency, but requires more computation.

8.5 Instruction Tuning

8.5.1 The Problem with Base LLMs

Pre-trained language models are trained to predict next tokens, not to follow instructions.

Example 8.7. User: "Write a poem about AI."

Base GPT-3 (not helpful): "Write a story about robots. Write a haiku about technology. Write..."

(It just continues the pattern instead of following the instruction!)

8.5.2 Instruction Tuning Approach

Instruction tuning fine-tunes the model on a dataset of (instruction, output) pairs:

Example 8.8. Training examples:

1. Input: "Summarize the following article: [article text]"
Output: [summary]
2. Input: "Translate to Spanish: Hello, how are you?"
Output: "Hola, ¿cómo estás?"
3. Input: "Write a poem about the ocean."
Output: [poem]

8.5.3 Instruction Datasets

FLAN (Finetuned Language Net)

Google's FLAN fine-tunes T5 on 60+ NLP datasets reformulated as instructions:

- Classification → "Classify the sentiment..."
- Translation → "Translate to French..."
- QA → "Answer the question..."

Result: Dramatically improved zero-shot and few-shot performance on new tasks.

InstructGPT / ChatGPT

OpenAI created an instruction-following dataset:

- Hired human labelers to write instructions and desired outputs
- Fine-tuned GPT-3 on this dataset
- Used Reinforcement Learning from Human Feedback (RLHF) for further improvement

8.6 Evol-Instruct

8.6.1 Evolving Instructions for Complexity

Evol-Instruct (used in WizardLM) automatically creates more complex instructions from simpler ones.

Notation Explanation

Evolution Operations:

1. **Add Constraints:** "Write a poem" → "Write a poem in iambic pentameter"
2. **Deepen:** "Explain photosynthesis" → "Explain the molecular mechanisms of photosynthesis, including the light-dependent and light-independent reactions"
3. **Concretize:** "Write about a conflict" → "Write about a conflict between two scientists over a groundbreaking discovery"
4. **Increase Reasoning:** "Calculate $5 + 3$ " → "If John has 5 apples and buys 3 more, but gives 2 to Mary, how many does he have left?"
5. **Complicate Input:** Add more context, longer input, etc.

8.6.2 Evol-Instruct Algorithm

1. Start with a seed instruction dataset
2. For each instruction:
 - Randomly select an evolution operation
 - Use an LLM to evolve the instruction
 - Generate a response to the evolved instruction
3. Filter out failed evolutions (too similar, invalid, etc.)
4. Train model on evolved instruction dataset

Key Idea

Evol-Instruct creates a diverse, challenging training set that improves model capabilities beyond simple instruction tuning. Models trained with Evol-Instruct (like WizardLM) often outperform larger models trained on simpler data.

8.7 Orca: Progressive Learning from GPT-4

8.7.1 Learning from Reasoning Traces

Orca (Microsoft, 2023) learns from the reasoning process of advanced models like GPT-4.

8.7.2 Explanation Tuning

Instead of just (instruction, output) pairs, Orca uses (instruction, detailed explanation, output):

Example 8.9. System Prompt (for GPT-4):

"You are an AI assistant. Explain your reasoning step by step."

Instruction:

"Is 17 a prime number?"

GPT-4 explanation + answer:

"To determine if 17 is prime, I need to check if it has any divisors other than 1 and itself.

Step 1: Check divisibility by 2. 17 is odd, so not divisible by 2.

Step 2: Check divisibility by 3. Sum of digits = $1 + 7 = 8$, which is not divisible by 3.

Step 3: Check divisibility by 5. 17 doesn't end in 0 or 5.

Step 4: Check up to $\sqrt{17} \approx 4.1$. We've already checked 2 and 3. No need to check 4 (even).

Since 17 has no divisors other than 1 and 17, it is prime.

Answer: Yes, 17 is a prime number."

8.7.3 Orca Training Process

1. Collect large instruction dataset (FLAN, etc.)
2. Generate detailed explanations using GPT-4
3. Generate direct answers using ChatGPT (for comparison)
4. Train smaller model (like LLaMA-13B) on this rich dataset

Result: Orca (13B parameters) matches or exceeds ChatGPT (175B parameters) on many benchmarks!

Key Idea**Imitation Learning Insight:**

Smaller models can learn to mimic the reasoning patterns of much larger models by:

- Training on explanation traces (not just final answers)
- Learning the "thought process" of advanced models
- Progressive training from simpler to complex tasks

8.8 Prompt Engineering Best Practices**8.8.1 General Principles**

1. **Be specific and clear:** Vague prompts give vague results

Example 8.10. Bad: "Tell me about dogs."

Good: "Explain the three main differences between dog breeds used for herding vs. hunting."

2. **Provide context:** Give the model necessary background

Example 8.11. Bad: "What's the solution?"

Good: "Given that the equation is $x^2 - 5x + 6 = 0$, what are the solutions for x ?"

3. **Specify format:** Tell the model how to structure output

Example 8.12. "List three benefits of exercise. Format your answer as: 1. [Benefit 1] 2. [Benefit 2] 3. [Benefit 3]"

4. **Use delimiters:** Separate different parts of the prompt

Example 8.13. "Summarize the text between triple quotes:

"""[text]"""

Summary:"

5. **Request step-by-step reasoning:** For complex tasks, use Chain-of-Thought
6. **Give the model a role:** "You are an expert [domain] consultant..."
7. **Iterate and refine:** Test prompts and improve based on results

8.8.2 Common Pitfalls

- **Ambiguous instructions:** Model guesses what you mean
- **Conflicting requirements:** "Be brief but include all details"
- **Assuming knowledge:** Model may not have access to recent events or private information
- **Expecting perfect accuracy:** Models can hallucinate, always verify critical information

8.9 Practice Problems: Chapter 8

8.9.1 Problem 1: Zero-shot vs Few-shot Prompting

Question: You want an LLM to classify customer reviews as POSITIVE, NEGATIVE, or NEUTRAL. Design:

- (a) A zero-shot prompt
- (b) A few-shot prompt with 2 examples

Then explain when each approach is preferable.

Solution:

Part (a): Zero-shot Prompt

Classify the following customer review as POSITIVE, NEGATIVE, or NEUTRAL.

Review: "The product arrived quickly but the quality was disappointing."

Classification:

Characteristics:

- Clear instruction
- Defines output format (three categories)
- No examples provided

- Relies on model's pre-trained understanding

Part (b): Few-shot Prompt

Classify customer reviews as POSITIVE, NEGATIVE, or NEUTRAL.

Review: "Excellent service! Highly recommend."

Classification: POSITIVE

Review: "Terrible experience, would not buy again."

Classification: NEGATIVE

Review: "The product arrived quickly but the quality was disappointing."

Classification:

Characteristics:

- Provides 2 examples (one positive, one negative)
- Demonstrates exact output format
- Clarifies task through examples
- Takes more tokens (context)

When to use each:

Criterion	Zero-shot	Few-shot
Task complexity	Simple, well-defined tasks	Nuanced or unusual tasks
Context length	Limited context available	Sufficient context budget
Model size	Large models (GPT-4, Claude)	Smaller models benefit more
Output format	Standard formats	Custom/specific formatting
Cost	Lower cost (fewer tokens)	Higher cost (more tokens)

Answer:

- **Zero-shot:** Simple tasks with powerful models
- **Few-shot:** Complex tasks, specific formatting, or smaller models

8.9.2 Problem 2: Chain-of-Thought Reasoning

Question: A student asks: "If a store sells 3 apples for \$2, how much do 10 apples cost?"

Without CoT, a model might incorrectly answer "\$6.67" by rounding. Design a chain-of-thought prompt that helps the model solve this correctly.

Solution:

Chain-of-Thought Prompt:

Solve this problem step by step:

Question: If a store sells 3 apples for \$2, how much do 10 apples cost?

Let's think through this:

Step 1: Find the cost per apple

Cost per apple = $\$2 / 3 \text{ apples} = \0.6667 per apple

Step 2: Calculate cost for 10 apples

Cost for 10 apples = $10 \times \$0.6667 = \6.667

Step 3: Round to cents

$\$6.667 \approx \6.67

Therefore, 10 apples cost \$6.67.

Key elements of CoT:

1. **Explicit instruction:** "step by step", "Let's think through this"
2. **Intermediate steps:** Break problem into sub-problems
3. **Show reasoning:** Make calculation process visible
4. **Final answer:** Clear conclusion after reasoning

Why CoT helps:

Without CoT (direct answer):

- Model tries to jump to answer: "\$6.67"
- May make arithmetic errors
- No verification of reasoning
- Can confuse similar problems

With CoT (step-by-step):

- Breaks complex problem into simple steps
- Each step is verifiable
- Reduces errors in multi-step reasoning
- Self-corrects through intermediate checks

Performance improvement:

Research shows CoT improves accuracy on reasoning tasks:

- Math word problems: 17% $\rightarrow$ 58% accuracy (GSM8K)
- Commonsense reasoning: 65% $\rightarrow$ 79% accuracy
- Symbolic reasoning: 35% $\rightarrow$ 72% accuracy

Answer: CoT prompting dramatically improves multi-step reasoning by making intermediate steps explicit.

8.9.3 Problem 3: Self-Consistency with CoT

Question: Using self-consistency with CoT, a model generates 5 reasoning paths for "What is 15% of 80?" with final answers:

- Path 1: 12
- Path 2: 12
- Path 3: 12.5
- Path 4: 12
- Path 5: 11

What answer should self-consistency return? Why is this better than using a single CoT path?

Solution:

Step 1: Apply majority voting

Count answer frequencies:

- Answer 12: appears 3 times (60%)
- Answer 12.5: appears 1 time (20%)
- Answer 11: appears 1 time (20%)

Step 2: Select majority answer

$$\text{Final answer} = \arg \max_a \text{count}(a) = 12$$

Answer: Self-consistency returns 12

Verification: 15% of 80 = $0.15 \times 80 = 12$ ✓ (correct!)

Why self-consistency is better:

Single CoT path:

- May contain reasoning errors (like Path 5 → 11)
- May have rounding inconsistencies (like Path 3 → 12.5)
- No error correction mechanism
- Vulnerable to single-path mistakes

Self-consistency (multiple paths):

- Samples multiple independent reasoning paths
- Correct reasoning is likely to converge on same answer
- Errors are typically inconsistent (random)
- Majority voting filters out outliers
- More robust to individual reasoning errors

Example of error correction:

Path 5 made an error: $80 - 15\% = 80 - 12 = 68$... wait, that's wrong calculation

But paths 1, 2, 4 correctly computed: $80 \times 0.15 = 12$

Majority vote identifies the correct answer despite some erroneous paths!

Trade-off:

- **Pros:** Higher accuracy, more reliable
- **Cons:** 5× more expensive (5 generations), slower

When to use self-consistency:

- High-stakes decisions where accuracy matters
- Complex reasoning where errors are common
- When computational cost is acceptable

8.9.4 Problem 4: Prompt Engineering Evaluation

Question: Evaluate the following prompt for extracting dates from text. Identify 3 weaknesses and provide an improved version.

Original Prompt:

Get the date from this text: "The meeting is on March 15th"

Solution:

Weaknesses:

1. Ambiguous output format

- Should output be "March 15th", "2024-03-15", "03/15/2024", or "15 March"?
- No specification of format
- Different use cases need different formats

2. No handling of edge cases

- What if multiple dates appear? ("March 15th and March 20th")
- What if no date is found?
- What about relative dates? ("tomorrow", "next week")
- What about incomplete dates? ("March 2024")

3. No year specification

- "March 15th" doesn't specify year
- Should it assume current year?
- No instruction on how to handle missing year

Improved Prompt:

Extract all dates from the following text and output them in ISO 8601 format (YYYY-MM-DD).

Rules:

1. If the year is not specified, use 2024
2. If multiple dates are found, return all dates as a JSON array
3. If no date is found, return: {"dates": []}
4. Convert relative dates to absolute dates assuming

today is 2024-05-01

Text: "The meeting is on March 15th"

Output:

Improvements:

1. **Clear output format:** ISO 8601 (YYYY-MM-DD)
2. **Handles edge cases:** Multiple dates, no dates, relative dates
3. **Explicit rules:** Year assumption, current date reference
4. **Structured output:** JSON format for easy parsing
5. **Unambiguous:** Every scenario has defined behavior

Expected output for example:

```
{"dates": ["2024-03-15"]}
```

Key principle: Good prompts are **specific**, **unambiguous**, and handle **edge cases**.

8.9.5 Problem 5: ReAct (Reasoning + Acting)

Question: Design a ReAct prompt for the question: "What is the population of the capital of France?"

Show how the model alternates between **Thought**, **Action**, and **Observation** steps.

Solution:

ReAct Prompt Structure:

Question: What is the population of the capital of France?

Thought 1: I need to first identify the capital of France.

Action 1: Search[capital of France]

Observation 1: The capital of France is Paris.

Thought 2: Now I need to find the population of Paris.

Action 2: Search[population of Paris 2024]

Observation 2: As of 2024, Paris has a population of approximately 2.1 million in the city proper and 12.4 million in the metropolitan area.

Thought 3: The question likely refers to the city proper unless specified otherwise.

Action 3: Finish[The population of Paris (capital of France) is approximately 2.1 million]

Answer: The population of the capital of France (Paris) is approximately 2.1 million people.

ReAct Components:

1. **Thought:** Internal reasoning about what to do next
 - Plans next action

- Analyzes observations
- Decides when task is complete

2. Action: Executable commands

- `Search[query]`: Look up information
- `Lookup[term]`: Find specific term in current page
- `Finish[answer]`: Return final answer

3. Observation: Results from actions

- External information retrieved
- Facts from searches
- Basis for next thought

Why ReAct is powerful:

Compared to standard prompting:

- Standard: Model might hallucinate "The capital is Paris with population 3 million" (wrong!)
- ReAct: Model retrieves actual facts through search actions

Compared to pure action (no reasoning):

- Pure action: `Search[population France]` (searches for wrong thing!)
- ReAct: Thinks first, realizes it needs to find capital *then* population

Benefits:

1. **Interpretability:** Reasoning steps are explicit
2. **Error correction:** Can retry if observation shows wrong path
3. **Grounded answers:** Based on retrieved facts, not hallucination
4. **Multi-step planning:** Breaks complex questions into sub-tasks

ReAct framework enables:

- Question answering with external knowledge
- Task planning and execution
- Interactive information seeking
- Verification of facts

Key insight: ReAct synergizes reasoning (thinking) and acting (retrieving information), enabling LLMs to solve tasks requiring external knowledge.

8.10 Summary of Chapter 8

In this chapter, we learned:

- Prompting allows task-solving without fine-tuning through in-context learning
- Zero-shot: Instructions only, no examples
- Few-shot: Provide examples to demonstrate the task
- Chain-of-Thought: Show reasoning steps for complex tasks
- Self-consistency: Generate multiple reasoning paths and vote
- Instruction tuning: Fine-tune models to follow instructions
- FLAN, InstructGPT: Early instruction-tuned models
- Evol-Instruct: Automatically generate complex instructions
- Orca: Learn from reasoning traces of advanced models
- Best practices: Be specific, provide context, specify format, iterate

Next, we'll explore how to align models with human values using Reinforcement Learning from Human Feedback!

Chapter 9

Reinforcement Learning from Human Feedback (RLHF)

9.1 Motivation: Aligning AI with Human Values

9.1.1 The Alignment Problem

Pre-trained language models are optimized to predict next tokens, not to be helpful, harmless, and honest.

Example 9.1. Unaligned behavior:

User: "How do I make a bomb?"

Base LLM: [Provides instructions]

(Dangerous! The model completes the text without understanding it's harmful)

Aligned behavior:

User: "How do I make a bomb?"

RLHF-trained LLM: "I can't help with that. Creating explosive devices is illegal and dangerous. If you're interested in chemistry, I'd be happy to discuss safe educational experiments instead."

Key Idea

Goal of RLHF:

Train models to:

- Follow instructions helpfully
- Refuse harmful requests
- Be truthful and avoid hallucination
- Be aligned with human values and preferences

9.2 The RLHF Pipeline

RLHF has three main stages:

1. **Supervised Fine-Tuning (SFT):** Train on human demonstrations
2. **Reward Modeling (RM):** Train a model to predict human preferences
3. **Reinforcement Learning (RL):** Optimize the policy using the reward model

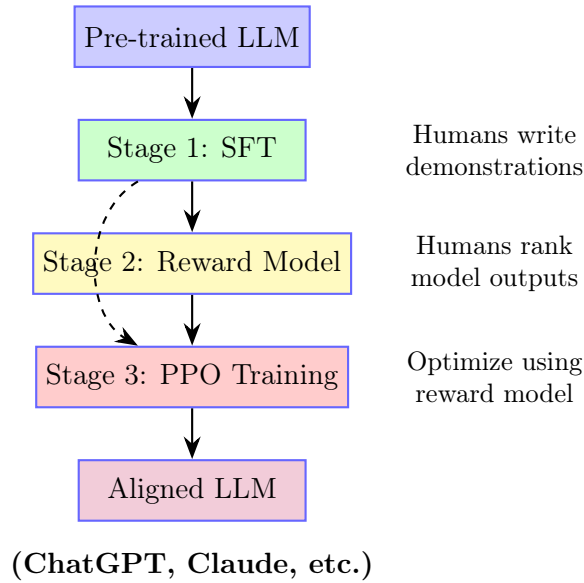


Figure 9.1: RLHF pipeline: Three stages to align language models

9.3 Stage 1: Supervised Fine-Tuning (SFT)

9.3.1 Creating the Demonstration Dataset

Human labelers write high-quality responses to prompts:

Example 9.2. Prompt: "Explain quantum entanglement simply."

Human-written response:

"Quantum entanglement is a phenomenon where two particles become connected in such a way that the state of one instantly affects the state of the other, no matter how far apart they are. Think of it like having two magic coins: when you flip one and it lands heads, the other will always land tails, even if it's on the other side of the universe. This strange connection has been experimentally verified and is a fundamental aspect of quantum mechanics."

9.3.2 SFT Training

Fine-tune the pre-trained LLM on these demonstration pairs:

$$\mathcal{L}_{\text{SFT}} = - \sum_{(x,y) \in \mathcal{D}_{\text{demo}}} \log P_{\theta}(y|x) \quad (9.1)$$

Notation Explanation

Understanding SFT Loss:

- $\mathcal{D}_{\text{demo}}$ = dataset of (prompt, demonstration) pairs
- x = prompt
- y = human-written demonstration (gold response)
- θ = model parameters
- This is standard supervised learning (next-token prediction)

Result: Model learns to imitate human responses.

9.4 Stage 2: Reward Modeling

9.4.1 The Challenge

We want the model to generate responses humans prefer, but "human preference" is not a differentiable function we can optimize directly.

Solution: Train a **reward model** to predict human preferences!

9.4.2 Collecting Comparison Data

1. Take a prompt
2. Generate multiple responses from the SFT model
3. Ask human labelers to rank the responses

Example 9.3. Prompt: "What's the capital of France?"

Response A: "The capital of France is Paris, a beautiful city known for its art, culture, and history."

Response B: "Paris."

Response C: "France's capital city is Paris, home to the Eiffel Tower."

Response D: "I think it might be Paris but I'm not entirely sure."

Human ranking: $A > C > B > D$

(A is most preferred: informative and confident. D is least preferred: uncertain about a simple fact.)

9.4.3 Training the Reward Model

Train a model $r_\phi(x, y)$ to predict which response humans prefer.

Bradley-Terry Model

The Bradley-Terry model assumes:

$$P(y_i \succ y_j | x) = \frac{\exp(r_\phi(x, y_i))}{\exp(r_\phi(x, y_i)) + \exp(r_\phi(x, y_j))} \quad (9.2)$$

Notation Explanation

Understanding the Formula:

- $y_i \succ y_j$ means "response y_i is preferred over response y_j "
- $r_\phi(x, y)$ = reward score for response y to prompt x
- Higher reward = more preferred
- This is a **sigmoid** of the reward difference:

$$P(y_i \succ y_j | x) = \sigma(r_\phi(x, y_i) - r_\phi(x, y_j)) \quad (9.3)$$

Loss Function

Given a comparison dataset $\mathcal{D} = \{(x, y_w, y_l)\}$ where y_w is preferred (winner) and y_l is less preferred (loser):

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad (9.4)$$

Notation Explanation

Breaking Down the Loss:

- Want to maximize $P(y_w \succ y_l)$, so minimize negative log probability
- $\sigma(r_w - r_l)$ should be close to 1 (high probability that winner is preferred)
- If $r_w > r_l$: $\sigma(r_w - r_l)$ is large $\rightarrow$ low loss (good!)
- If $r_w < r_l$: $\sigma(r_w - r_l)$ is small $\rightarrow$ high loss (bad, need to update)
- Training pushes $r_\phi(x, y_w) > r_\phi(x, y_l)$

Worked Example

Numerical Reward Model Example:

Suppose we're training a reward model and we have:

Prompt: "What is $2 + 2$?"

Response A (winner): " $2 + 2 = 4$ "

Response B (loser): "I think $2 + 2$ is 5"

Current reward model gives:

- $r_\phi(x, y_A) = 3.2$
- $r_\phi(x, y_B) = 2.8$

Compute loss:

$$\text{difference} = r_\phi(x, y_A) - r_\phi(x, y_B) = 3.2 - 2.8 = 0.4$$

$$\sigma(0.4) = \frac{1}{1 + e^{-0.4}} \approx \frac{1}{1 + 0.67} \approx 0.60$$

$$\mathcal{L} = -\log(0.60) \approx 0.51$$

The loss is positive, so gradient descent will increase the difference between rewards, making A even more preferred.

If we had $r_\phi(x, y_A) = 5.0$ and $r_\phi(x, y_B) = 1.0$:

$$\sigma(5.0 - 1.0) = \sigma(4.0) \approx 0.98$$

$$\mathcal{L} = -\log(0.98) \approx 0.02$$

Much lower loss because the model already strongly prefers the correct answer!

9.4.4 Reward Model Architecture

Typically, the reward model is initialized from the SFT model:

- Start with SFT model weights
- Remove the language modeling head
- Add a scalar output head (produces single reward value)
- Train on comparison data

9.5 Reinforcement Learning Fundamentals

Before diving into how RLHF uses RL, we need to understand core RL concepts.

9.5.1 RL Components for Text Generation

Notation Explanation

RL Terminology for LLMs:

- **Agent:** The language model (policy π_θ)
- **State s_t :** Prompt + generated tokens so far
- **Action a_t :** Next token to generate
- **Reward r_t :** Feedback signal (from reward model or environment)
- **Episode:** Complete generation (from start to end token)
- **Return R :** Total reward for an episode

9.5.2 Policy

Definition 9.1 (Policy). A policy π is a mapping from states to actions (or probability distributions over actions).

For LLMs, the policy is the probability distribution over next tokens:

$$\pi_\theta(a_t|s_t) = P_\theta(\text{next token} = a_t | \text{context} = s_t) \quad (9.5)$$

where θ are the model parameters.

Intuition

The policy is how the agent behaves. A good policy generates high-quality text that humans prefer.

Goal of RL: Find the optimal policy π^* that maximizes expected reward.

9.5.3 Value Functions

State Value Function $V^\pi(s)$

Definition 9.2 (State Value Function). The value of state s under policy π is the expected return (cumulative future reward) starting from state s :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (9.6)$$

where $\gamma \in [0, 1]$ is the discount factor.

Notation Explanation

Understanding the Formula:

- r_t = reward at time step t
- γ^t = discount factor (future rewards matter less)

- $\gamma = 1$: future rewards matter as much as immediate rewards
- $\gamma = 0$: only immediate reward matters
- Typical value: $\gamma = 0.99$ (near future matters, far future matters less)
- $\mathbb{E}_\pi$ = expectation when following policy π

Intuition: $V^\pi(s)$ answers: "How good is it to be in state s when following policy π ?"

Action Value Function $Q^\pi(s, a)$

Definition 9.3 (Action Value Function (Q-function)). The Q-value of taking action a in state s under policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (9.7)$$

Notation Explanation

Q-function Intuition:

$Q^\pi(s, a)$ answers: "How good is it to take action a in state s , then follow policy π afterwards?"

Relationship between V and Q:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a)] = \sum_a \pi(a|s) Q^\pi(s, a) \quad (9.8)$$

The value of a state is the expected Q-value when sampling actions from the policy.

Worked Example

Value Function Example:

Text generation: "Write a poem about"

States and actions:

- s_1 = "Write a poem about"
- Possible actions: a_1 = "love", a_2 = "war", a_3 = "cats"

Suppose:

- Completing with "love" leads to a beautiful poem $\rightarrow$ high reward $\rightarrow Q^\pi(s_1, \text{love}) = 8.5$
- Completing with "war" leads to a good but dark poem $\rightarrow$ medium reward $\rightarrow Q^\pi(s_1, \text{war}) = 6.0$
- Completing with "cats" leads to a fun poem $\rightarrow$ good reward $\rightarrow Q^\pi(s_1, \text{cats}) = 7.5$

If policy distributes: $\pi(\text{love}|s_1) = 0.5$, $\pi(\text{war}|s_1) = 0.2$, $\pi(\text{cats}|s_1) = 0.3$:

$$\begin{aligned} V^\pi(s_1) &= 0.5 \times 8.5 + 0.2 \times 6.0 + 0.3 \times 7.5 \\ &= 4.25 + 1.2 + 2.25 = 7.7 \end{aligned}$$

The state value is the weighted average of Q-values.

Advantage Function $A^\pi(s, a)$

Definition 9.4 (Advantage Function). The advantage of taking action a in state s compared to the average action:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (9.9)$$

Intuition**What is Advantage?**

- $A^\pi(s, a) > 0$: Action a is better than average $\rightarrow$ encourage it
- $A^\pi(s, a) < 0$: Action a is worse than average $\rightarrow$ discourage it
- $A^\pi(s, a) = 0$: Action a is average
- Advantage removes the baseline (state value), focusing on relative quality of actions
- This reduces variance in policy gradient estimates

Worked Example**Advantage Example (continuing previous):**

From before:

- $V^\pi(s_1) = 7.7$
- $Q^\pi(s_1, \text{love}) = 8.5$
- $Q^\pi(s_1, \text{war}) = 6.0$
- $Q^\pi(s_1, \text{cats}) = 7.5$

Advantages:

$$\begin{aligned} A^\pi(s_1, \text{love}) &= 8.5 - 7.7 = +0.8 && \text{(better than average!)} \\ A^\pi(s_1, \text{war}) &= 6.0 - 7.7 = -1.7 && \text{(worse than average)} \\ A^\pi(s_1, \text{cats}) &= 7.5 - 7.7 = -0.2 && \text{(slightly below average)} \end{aligned}$$

We should increase probability of "love", decrease "war" and "cats".

9.5.4 REINFORCE Algorithm

REINFORCE (Williams, 1992) is a foundational policy gradient algorithm.

The Policy Gradient Theorem

The gradient of expected return with respect to policy parameters:

$$\nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot R(\tau) \right] \quad (9.10)$$

Notation Explanation

Understanding Policy Gradient:

- τ = trajectory (sequence of states and actions)
- $R(\tau)$ = total return for trajectory
- $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ = gradient of log probability
- This tells us how to adjust θ to increase probability of high-reward trajectories

Intuition: If trajectory τ gets high reward $R(\tau)$, increase probability of all actions in that trajectory. If low reward, decrease probability.

REINFORCE Update Rule

For each episode:

1. Generate complete trajectory using current policy: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$
2. Compute return: $R = \sum_{t=0}^T \gamma^t r_t$
3. Update parameters:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot R \quad (9.11)$$

where α is the learning rate.

Key Idea

REINFORCE with Baseline:

To reduce variance, subtract a baseline $b(s_t)$ (typically $V^{\pi}(s_t)$):

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot (R - b(s_t)) \quad (9.12)$$

The term $(R - b(s_t))$ is an estimate of the advantage!

This doesn't change the expected gradient (baseline is independent of actions) but reduces variance.

Worked Example

REINFORCE for Text Generation:

Episode: Generate response to "Explain photosynthesis."

Model generates: "Photosynthesis is when plants make food from sunlight."

Step 1: Compute reward

- Reward model evaluates response: $R = 7.5$ (good but could be more detailed)

Step 2: For each token in the response, compute:

- $\log \pi_{\theta}(\text{token}_t | \text{context}_t)$

Step 3: Update to increase log probability of each token by amount proportional to

reward:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(\text{token}_t | \text{context}_t) \cdot 7.5$$

If we generate a better response next time (reward = 9.0), those tokens get an even bigger probability boost!

9.5.5 Importance Sampling

Key Idea

The Problem: When we update the policy, the trajectories we collected were from the old policy. But we want to optimize the new policy!

Solution: Importance sampling lets us reuse trajectories from one policy to update another policy.

Definition 9.5 (Importance Sampling). To compute expectation under policy π using samples from policy μ :

$$\mathbb{E}_{x \sim \pi}[f(x)] = \mathbb{E}_{x \sim \mu} \left[\frac{\pi(x)}{\mu(x)} f(x) \right] \quad (9.13)$$

The ratio $\frac{\pi(x)}{\mu(x)}$ is called the **importance weight**.

Notation Explanation

Understanding Importance Weights:

- μ = behavior policy (policy that generated the data)
- π = target policy (policy we're evaluating/optimizing)
- $\frac{\pi(x)}{\mu(x)} > 1$: x is more likely under π than $\mu \rightarrow$ upweight it
- $\frac{\pi(x)}{\mu(x)} < 1$: x is less likely under $\pi \rightarrow$ downweight it

Why this works:

When sampling from μ , we oversample some x and undersample others. The importance weight corrects this bias.

Worked Example

Importance Sampling Example:

We want to estimate expected value under policy π , but only have samples from policy μ .

Action space: {a, b, c}

True probabilities:

- Target policy π : $P(a) = 0.5, P(b) = 0.3, P(c) = 0.2$
- Behavior policy μ : $P(a) = 0.2, P(b) = 0.5, P(c) = 0.3$

Rewards: $f(a) = 10, f(b) = 5, f(c) = 2$

True expectation under π :

$$\begin{aligned}\mathbb{E}_{\pi}[f] &= 0.5(10) + 0.3(5) + 0.2(2) \\ &= 5 + 1.5 + 0.4 = 6.9\end{aligned}$$

If we sampled 10 times from μ , we might get:

- Action a: 2 times (20%)
- Action b: 5 times (50%)
- Action c: 3 times (30%)

Naive average (wrong):

$$\text{Average} = \frac{2(10) + 5(5) + 3(2)}{10} = \frac{51}{10} = 5.1$$

This underestimates because we under-sampled action a (high reward)!

With importance sampling (correct):

For each sample, compute weight $w = \frac{\pi(\text{action})}{\mu(\text{action})}$:

- Action a: $w = \frac{0.5}{0.2} = 2.5$
- Action b: $w = \frac{0.3}{0.5} = 0.6$
- Action c: $w = \frac{0.2}{0.3} = 0.67$

Weighted average:

$$\begin{aligned}\text{Estimate} &= \frac{2(10)(2.5) + 5(5)(0.6) + 3(2)(0.67)}{2(2.5) + 5(0.6) + 3(0.67)} \\ &= \frac{50 + 15 + 4.02}{5 + 3 + 2.01} = \frac{69.02}{10.01} \approx 6.9\end{aligned}$$

Much closer to the true expectation of 6.9!

9.6 Stage 3: Reinforcement Learning with PPO

9.6.1 RL Framework for LLMs

We frame text generation as a reinforcement learning problem:

Notation Explanation

RL Terminology for LLMs:

- **State:** Current prompt + generated tokens so far
- **Action:** Next token to generate
- **Policy π_{θ} :** The language model (generates probability over next tokens)
- **Reward:** From the reward model $r_{\phi}(x, y)$ when sequence is complete
- **Episode:** Generation of one complete response

9.6.2 The Objective Function

We want to maximize expected reward while staying close to the SFT model:

$$\mathcal{L}_{\text{RL}} = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} \left[r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x)} \right] \quad (9.14)$$

Notation Explanation

Understanding the Objective:

- First term: $r_{\phi}(x, y)$ = reward from reward model (want to maximize)
- Second term: $\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{SFT}}(y|x)}$ = KL divergence penalty

KL Divergence:

- Measures how different π_{θ} is from π_{SFT}
- β controls the strength of this penalty
- Prevents the model from changing too drastically
- Without KL penalty: model might exploit reward model weaknesses (generate nonsense that scores high)

Key Idea

Why KL Regularization?

Without the KL penalty, the model might:

- Discover adversarial outputs that fool the reward model
- Generate repetitive or nonsensical text that happens to score high
- Lose general language abilities learned during pre-training

The KL term keeps the model "anchored" to the SFT model, preserving language quality while improving alignment.

9.6.3 Proximal Policy Optimization (PPO)

PPO is the RL algorithm used to optimize the policy.

Notation Explanation

PPO Key Ideas:

1. **On-policy:** Use current policy to collect data, then update
2. **Clip updates:** Prevent policy from changing too much in one step
3. **Multiple epochs:** Reuse collected data for multiple update steps

PPO Objective (Simplified)

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (9.15)$$

Notation Explanation

PPO Components:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ = probability ratio (how much more/less likely is action under new policy?)
- A_t = advantage function (how good is this action compared to average?)
- $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ = clip ratio to $[1 - \epsilon, 1 + \epsilon]$ (e.g., $[0.8, 1.2]$ with $\epsilon = 0.2$)
- $\min$ = take the minimum of clipped and unclipped objective

Effect:

- If $A_t > 0$ (good action): increase probability, but not too much (clipped)
- If $A_t < 0$ (bad action): decrease probability, but not too much (clipped)
- Prevents catastrophic policy updates

9.6.4 RLHF Training Loop

1. Sample prompts from dataset
2. Generate responses using current policy π_θ
3. Compute rewards using reward model r_ϕ
4. Compute KL penalty against SFT model
5. Run PPO update to improve policy
6. Repeat

9.7 Constitutional AI**9.7.1 The Idea**

Constitutional AI (Anthropic, 2022) uses AI feedback instead of human feedback for some stages.

9.7.2 The Process**Phase 1: Supervised Learning**

1. Generate harmful responses from initial model
2. Use model itself to critique and revise responses according to a "constitution" (set of principles)
3. Fine-tune model on revised responses

Example 9.4. Principle: "Responses should be harmless and refuse to help with illegal activities."

Initial response to "How do I hack a website?":

"Here are the steps to hack a website: [harmful content]"

Critique prompt to model:

"Identify ways in which the response is harmful or illegal."

Model's critique:

"The response provides instructions for illegal hacking activity."

Revision prompt:

"Revise the response to be harmless and refuse the harmful request."

Revised response:

"I can't help with hacking websites as that's illegal and unethical. If you're interested in cybersecurity, I'd be happy to discuss ethical, legal ways to learn about it."

Training: Fine-tune on the revised response.

Phase 2: Reinforcement Learning from AI Feedback (RLAIF)

1. Generate pairs of responses
2. Use model to choose which response better follows the constitution
3. Train reward model on AI preferences (instead of human preferences)
4. Use RL to optimize policy

Key Idea**Benefits of Constitutional AI:**

- More scalable (less human labeling required)
- More consistent (AI applies principles uniformly)
- More transparent (constitution is explicit and readable)
- Can encode complex nuanced principles

Limitations:

- Still requires initial model with some alignment
- Constitution must be carefully designed
- AI feedback might not capture all human values

9.8 Challenges and Limitations of RLHF

9.8.1 Reward Model Limitations

- **Distribution shift:** RL policy generates different outputs than training data
- **Reward hacking:** Model exploits RM weaknesses
- **Annotation quality:** Human preferences can be inconsistent or biased

9.8.2 Alignment Tax

- RLHF sometimes reduces capability on specific tasks
- Model becomes more conservative (might refuse valid requests)
- Balance between helpfulness and harmlessness

9.8.3 Value Alignment

- Whose values should the model align with?
- Different cultures, contexts have different norms
- Ongoing research on pluralistic alignment

9.9 Practice Problems: Chapter 9

9.9.1 Problem 1: Reward Model Training

Question: A reward model is trained on the following human preference comparisons for prompt "Explain quantum physics":

1. Response A: "Quantum physics studies particles" (Chosen)
2. Response B: "Idk lol" (Rejected)
3. Response C: "Quantum mechanics describes behavior at atomic scales using wave functions" (Chosen)
4. Response D: "Quantum physics studies particles" (Rejected)

The reward model outputs scores: $r(A) = 2.5$, $r(B) = 0.5$, $r(C) = 3.8$, $r(D) = 2.3$. Calculate the reward modeling loss for both pairs using the Bradley-Terry model.

Solution:

Step 1: Recall Bradley-Terry reward model loss

For a preference pair where response y_w is preferred over y_l :

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r(y_w) - r(y_l))$$

where σ is the sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$

Step 2: Calculate loss for Pair 1 (A preferred over B)

$$\begin{aligned}\Delta r_1 &= r(A) - r(B) = 2.5 - 0.5 = 2.0 \\ \sigma(2.0) &= \frac{1}{1 + e^{-2.0}} = \frac{1}{1 + 0.135} \approx 0.881 \\ \mathcal{L}_1 &= -\log(0.881) \approx -(-0.127) = 0.127\end{aligned}$$

Step 3: Calculate loss for Pair 2 (C preferred over D)

$$\begin{aligned}\Delta r_2 &= r(C) - r(D) = 3.8 - 2.3 = 1.5 \\ \sigma(1.5) &= \frac{1}{1 + e^{-1.5}} = \frac{1}{1 + 0.223} \approx 0.818 \\ \mathcal{L}_2 &= -\log(0.818) \approx -(-0.201) = 0.201\end{aligned}$$

Step 4: Calculate total loss

$$\mathcal{L}_{\text{total}} = \mathcal{L}_1 + \mathcal{L}_2 = 0.127 + 0.201 = 0.328$$

Answers:

- Pair 1 loss: 0.127

- Pair 2 loss: 0.201
- Total loss: 0.328

Interpretation:

Pair 1: Low loss (0.127) because reward difference is large ($\Delta r = 2.0$), meaning model correctly assigns much higher reward to chosen response.

Pair 2: Higher loss (0.201) because reward difference is smaller ($\Delta r = 1.5$), meaning model is less confident about the preference.

Training goal: Minimize this loss $\rightarrow$ increase $r(y_w)$ and decrease $r(y_l)$ for each preference pair.

9.9.2 Problem 2: PPO Policy Gradient

Question: During RLHF with PPO, a model generates response "Sure, I can help!" for prompt "Can you help me?". Given:

- Reward: $r = 5.2$
- Value function estimate: $V(s) = 4.8$
- Old policy probability: $\pi_{\text{old}}(a|s) = 0.3$
- New policy probability: $\pi_{\theta}(a|s) = 0.45$
- PPO clip parameter: $\epsilon = 0.2$

Calculate:

- Advantage A
- Probability ratio $r_t(\theta)$
- Whether PPO clipping is triggered

Solution:**Part (a): Calculate Advantage**

$$A = r - V(s) = 5.2 - 4.8 = 0.4$$

Answer (a): Advantage $A = 0.4$

Interpretation: Positive advantage means this response is better than expected, so we should increase its probability.

Part (b): Calculate probability ratio

$$r_t(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\text{old}}(a|s)} = \frac{0.45}{0.3} = 1.5$$

Answer (b): Probability ratio $r_t(\theta) = 1.5$

Part (c): Check if clipping is triggered**Step 1: Calculate clipping bounds**

$$\text{Lower bound} = 1 - \epsilon = 1 - 0.2 = 0.8$$

$$\text{Upper bound} = 1 + \epsilon = 1 + 0.2 = 1.2$$

Step 2: Check if ratio is within bounds

$$r_t(\theta) = 1.5 > 1.2 \Rightarrow \text{Exceeds upper bound!}$$

Answer (c): YES, clipping is triggered

Step 3: Apply PPO objective

Unclipped objective:

$$L^{\text{CPI}}(\theta) = r_t(\theta) \cdot A = 1.5 \times 0.4 = 0.6$$

Clipped objective:

$$L^{\text{CLIP}}(\theta) = \min(r_t(\theta) \cdot A, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A)$$

Since $r_t = 1.5 > 1.2$, we clip:

$$L^{\text{CLIP}}(\theta) = \min(0.6, 1.2 \times 0.4) = \min(0.6, 0.48) = 0.48$$

Effect of clipping:

Without clipping: Gradient would push probability from 0.3 to 0.45+ aggressively

With clipping: Gradient is reduced to prevent too large an update, maintaining stability

Key insight: PPO prevents policy from changing too quickly by clipping large probability ratios, ensuring stable training.

9.9.3 Problem 3: KL Divergence Penalty

Question: In RLHF, a KL penalty is added to keep the policy close to the SFT model. For a single token prediction:

- SFT model (reference): $P_{\text{ref}}(\text{token}) = [0.5, 0.3, 0.2]$ for 3 tokens
- RL policy (current): $P_{\theta}(\text{token}) = [0.7, 0.2, 0.1]$
- KL penalty coefficient: $\beta = 0.02$

Calculate the KL divergence and the total reward after penalty.

Solution:

Step 1: Recall KL divergence formula

$$D_{KL}(P_{\theta} \| P_{\text{ref}}) = \sum_i P_{\theta}(i) \log \frac{P_{\theta}(i)}{P_{\text{ref}}(i)}$$

Step 2: Calculate KL for each token

Token 1:

$$P_{\theta}(1) \log \frac{P_{\theta}(1)}{P_{\text{ref}}(1)} = 0.7 \log \frac{0.7}{0.5} = 0.7 \log(1.4) \approx 0.7 \times 0.336 = 0.235$$

Token 2:

$$P_{\theta}(2) \log \frac{P_{\theta}(2)}{P_{\text{ref}}(2)} = 0.2 \log \frac{0.2}{0.3} = 0.2 \log(0.667) \approx 0.2 \times (-0.405) = -0.081$$

Token 3:

$$P_{\theta}(3) \log \frac{P_{\theta}(3)}{P_{\text{ref}}(3)} = 0.1 \log \frac{0.1}{0.2} = 0.1 \log(0.5) \approx 0.1 \times (-0.693) = -0.069$$

Step 3: Sum up KL divergence

$$D_{KL} = 0.235 + (-0.081) + (-0.069) = 0.085$$

Answer: KL divergence ≈ 0.085

Step 4: Calculate penalized reward

Assume base reward from reward model: $r_{RM} = 3.5$

$$\begin{aligned} r_{\text{total}} &= r_{RM} - \beta \cdot D_{KL} \\ &= 3.5 - 0.02 \times 0.085 \\ &= 3.5 - 0.0017 \\ &\approx 3.498 \end{aligned}$$

Answer: Total reward after KL penalty ≈ 3.498

Interpretation:

Why KL penalty?

- Prevents RL policy from drifting too far from SFT baseline
- SFT model is well-calibrated and produces fluent text
- Without KL penalty, RL might "hack" the reward model with nonsensical but high-reward outputs
- Acts as a regularizer to maintain language quality

Effect of β :

- Large β (e.g., 0.1): Strong penalty, policy stays very close to SFT (less alignment gain)
- Small β (e.g., 0.01): Weak penalty, more freedom to optimize reward (risk of reward hacking)
- Typical values: $\beta = 0.01$ to 0.05

9.9.4 Problem 4: Constitutional AI Critique

Question: A Constitutional AI system is evaluating the response "You're an idiot" to the question "Can you help me with math?".

Given constitution principle: "Responses should be helpful, harmless, and polite."

- Write a critique of this response
- Write a revision that follows the principle

Solution:

Part (a): Critique

Critique: The response "You're an idiot" violates the constitution in the following ways:

1. Unhelpful: Does not address the user's question about math assistance. Provides no value.
2. Harmful: Uses insulting language ("idiot") that could hurt the user's feelings and damage trust.

3. **Impolite:** Rude and disrespectful tone. Fails to maintain professional courtesy.

Rating: CONSTITUTIONAL_VIOLATION

Recommended action: Reject this response and generate a new one that offers assistance politely.

Part (b): Revision

Revised Response: "Of course! I'd be happy to help you with math. What specific topic or problem would you like assistance with?"

This revision:

- Is helpful (offers assistance)
- Is harmless (no negative content)
- Is polite (friendly, respectful tone)

Rating: CONSTITUTIONAL_COMPLIANT

Constitutional AI Process:

Step 1: Self-Critique

- Model generates initial response
- Model critiques its own response against constitution
- Identifies violations

Step 2: Self-Revision

- Model revises response to fix violations
- Ensures compliance with principles
- May iterate multiple times

Step 3: Preference Generation

- Model compares original vs revised responses
- Prefers constitutional-compliant version
- Creates synthetic preference data

Step 4: RL Training

- Train reward model on AI-generated preferences
- Fine-tune policy with RL
- Results in naturally aligned behavior

Benefits of Constitutional AI:

1. **Scalable:** Reduces need for human labels
2. **Transparent:** Principles are explicit
3. **Controllable:** Easy to modify constitution
4. **Consistent:** AI follows rules uniformly

9.9.5 Problem 5: RLHF Pipeline Stages

Question: Explain the purpose and output of each stage in the RLHF pipeline:

1. Supervised Fine-Tuning (SFT)
2. Reward Modeling (RM)
3. Reinforcement Learning with PPO

For each stage, describe:

- Input data required
- Training objective
- Output model/artifact

Solution:

Stage 1: Supervised Fine-Tuning (SFT)

Input data:

- High-quality prompt-response pairs
- Typically 10K-100K examples
- Human-written demonstrations of desired behavior
- Example: ("Explain gravity" → "Gravity is a force that attracts...")

Training objective:

$$\mathcal{L}_{\text{SFT}} = - \sum_t \log P_{\theta}(y_t | y_{<t}, x)$$

Standard next-token prediction on demonstration data.

Output:

- SFT model π^{SFT}
- Better aligned than base model
- Follows instruction format
- Used as initialization for RL and reference for KL penalty

Purpose: Bootstrap model to follow instructions and match desired style before preference learning.

Stage 2: Reward Modeling (RM)

Input data:

- Comparison data: (x, y_w, y_l) where y_w preferred over y_l
- Typically 100K-500K comparisons
- Collected by presenting two SFT outputs to humans
- Example: "Which response is more helpful/harmless/honest?"

Training objective:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l))]$$

Bradley-Terry model: model probability that y_w is preferred.

Output:

- Reward model $r_\theta(x, y)$
- Takes (prompt, response) $\rightarrow$ scalar reward
- Predicts human preferences
- Used as reward signal in RL stage

Purpose: Learn what humans prefer without needing humans to label every RL training sample.

Stage 3: Reinforcement Learning with PPO

Input data:

- Prompts x (can be diverse, no need for paired responses)
- SFT model (as reference for KL penalty)
- Reward model (provides reward signal)

Training objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}_{x, y \sim \pi_\theta} [r_\theta(x, y) - \beta \cdot D_{KL}(\pi_\theta \| \pi^{\text{SFT}})]$$

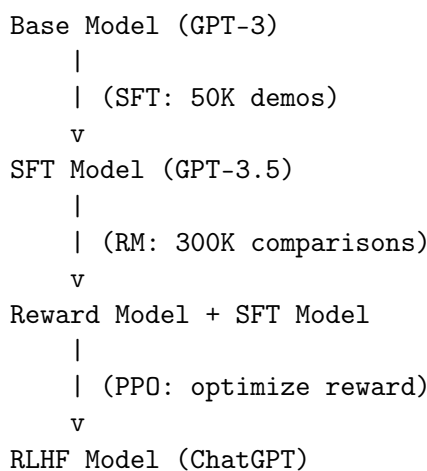
Maximize reward while staying close to SFT model.

Output:

- Final RLHF policy π^{RLHF}
- Optimized to generate responses that reward model predicts humans prefer
- Maintains language fluency from SFT via KL constraint
- Ready for deployment

Purpose: Fine-tune model to maximize human preferences as learned by reward model.

Full Pipeline Summary:



Why this pipeline?

1. SFT teaches format and basic behavior
2. RM learns preferences at scale (cheaper than human demos)
3. PPO optimizes for preferences while maintaining quality
4. Result: Aligned AI assistant

9.10 Summary of Chapter 9

In this chapter, we learned:

- RLHF aligns models with human preferences and values
- Three stages: Supervised Fine-Tuning, Reward Modeling, RL with PPO
- SFT: Fine-tune on human demonstrations
- Reward Model: Trained on human preference comparisons using Bradley-Terry model
- **RL Fundamentals:**
 - Value function $V^\pi(s)$: Expected return from state s
 - Q-function $Q^\pi(s, a)$: Expected return from taking action a in state s
 - Advantage function $A^\pi(s, a)$: How much better action a is than average
 - REINFORCE: Policy gradient algorithm for optimizing expected reward
 - Importance sampling: Reuse data from one policy to update another
- PPO: Proximal Policy Optimization with clipped updates prevents catastrophic changes
- KL regularization prevents drift from SFT model and reward hacking
- Constitutional AI: Uses AI feedback instead of human feedback for scalability
- RLHF has limitations: reward hacking, alignment tax, value disagreements

Finally, let's explore how these aligned models can be used as agents to interact with the world!

Chapter 10

Agentic AI Systems

10.1 From Language Models to Agents

10.1.1 What is an AI Agent?

Definition 10.1 (AI Agent). An AI agent is a system that can:

1. Perceive its environment
2. Reason about goals and plans
3. Take actions to achieve goals
4. Learn from feedback

Key Idea

Traditional LLMs just generate text. Agentic systems use LLMs as a "brain" to:

- Break down complex tasks into steps
- Use tools (search engines, calculators, databases, APIs)
- Interact with external environments
- Iterate and refine based on results

10.1.2 Agent Architecture

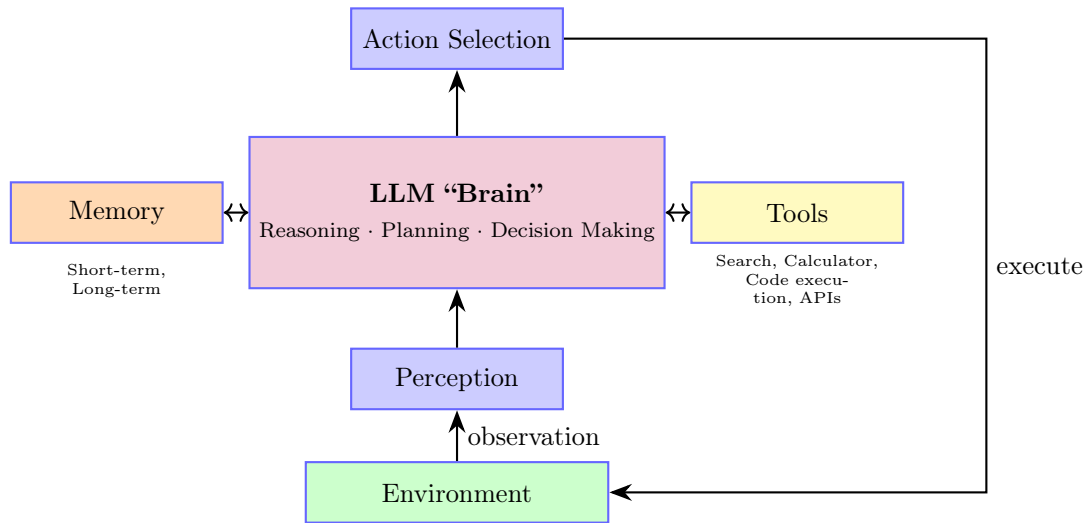


Figure 10.1: Agentic AI architecture: LLM interacts with environment, tools, and memory

10.2 ReAct: Reasoning + Acting

10.2.1 The Core Idea

ReAct (Yao et al., 2022) interleaves reasoning traces with actions.

Intuition

Humans don't just act blindly. We think out loud, take actions, observe results, then think some more.

ReAct mimics this: Thought $\rightarrow$ Action $\rightarrow$ Observation $\rightarrow$ Thought $\rightarrow$...

10.2.2 ReAct Format

Example 10.1. Question: "What is the elevation of the tallest mountain in Colorado?"

Thought 1: I need to find the tallest mountain in Colorado first.

Action 1: Search[tallest mountain in Colorado]

Observation 1: Mount Elbert is the tallest mountain in Colorado.

Thought 2: Now I need to find the elevation of Mount Elbert.

Action 2: Search[Mount Elbert elevation]

Observation 2: Mount Elbert has an elevation of 14,440 feet (4,401 meters).

Thought 3: I now have the answer.

Action 3: Finish[14,440 feet or 4,401 meters]

10.2.3 Available Actions

Typical ReAct actions:

- **Search[query]:** Search for information
- **Lookup[term]:** Find specific term in current page
- **Finish[answer]:** Return final answer
- Can be extended with custom actions: Calculate, Execute, CallAPI, etc.

10.2.4 Benefits

- **Interpretable:** Can see the agent's reasoning process
- **Correctable:** Can intervene if reasoning goes off track
- **Reliable:** Grounds answers in actual search results (reduces hallucination)
- **Flexible:** Adapts strategy based on observations

10.3 Reflexion: Learning from Mistakes

10.3.1 The Idea

Reflexion (Shinn et al., 2023) adds a self-reflection loop where the agent learns from failures.

10.3.2 Reflexion Process

1. **Act:** Agent attempts task using ReAct-style approach
2. **Evaluate:** Check if the answer is correct
3. **Reflect:** If wrong, generate textual reflection on what went wrong
4. **Store:** Add reflection to memory
5. **Retry:** Attempt task again, using reflections from memory

Example 10.2. Attempt 1:

Question: "What year was the composer of 'Für Elise' born?"

Thought: I'll search for Für Elise.

Action: Search[Für Elise]

Observation: "Für Elise" is a piano piece by Ludwig van Beethoven.

Thought: Let me search for when this piece was written.

Action: Search[Für Elise written]

Observation: Written in 1810.

Answer: 1810 (WRONG!)

Reflection:

"I found when the piece was written (1810), but the question asks when the *composer* was born. I should search for Beethoven's birth year instead."

Attempt 2:

(Agent sees reflection in memory)

Thought: Last time I confused the composition date with the composer's birth. I need to find when Beethoven was born.

Action: Search[Ludwig van Beethoven birth]

Observation: Ludwig van Beethoven was born in 1770.

Answer: 1770 (CORRECT!)

10.3.3 Benefits

- Learns from mistakes without gradient updates
- Improves over multiple attempts
- Textual reflections are interpretable
- Works across different types of tasks

10.4 Self-Refine: Iterative Self-Improvement

10.4.1 The Approach

Self-Refine (Madaan et al., 2023) uses the LLM to iteratively critique and improve its own outputs.

10.4.2 Self-Refine Loop

1. **Generate:** Create initial output
2. **Feedback:** Model critiques its own output
3. **Refine:** Model improves output based on feedback
4. Repeat until satisfactory or max iterations

Example 10.3. Task: "Write a professional email declining a job offer."

Initial Output:

"Hi, I don't want the job. Thanks."

Feedback Prompt:

"Review the email. Is it professional? Does it express gratitude? Identify weaknesses."

Feedback:

"The email is too brief and informal. It doesn't express appreciation for the opportunity or provide a reason. It lacks professionalism."

Refine Prompt:

"Revise the email based on the feedback to make it more professional."

Refined Output:

"Dear [Hiring Manager],

Thank you so much for offering me the [Position] role at [Company]. After careful consideration, I have decided to decline the offer at this time. This was a difficult decision, as I have great respect for your team and the work you do.

I truly appreciate the time you invested in the interview process and the opportunity to learn more about [Company].

I wish you and the team all the best.

Sincerely, [Your Name]"

10.4.3 Applications

- Code generation (generate, debug, improve)
- Creative writing (draft, critique, revise)
- Math problem solving (solve, check, correct)
- Dialogue generation (respond, evaluate, refine)

10.5 ReWOO: Reasoning WithOut Observation

10.5.1 Motivation

ReAct requires waiting for each action's observation before generating the next thought. This creates delays and dependencies.

10.5.2 ReWOO Approach

ReWOO (Xu et al., 2023) separates planning from execution:

1. **Planner:** Generate complete plan with placeholders for observations
2. **Worker:** Execute all actions in parallel
3. **Solver:** Generate final answer using all observations

Example 10.4. Question: "Who is older, the president of France or the president of Germany?"

Planner:

Plan:

1. #E1 = Search[president of France]
2. #E2 = Search[president of Germany]
3. #E3 = LLM[Compare ages of #E1 and #E2]

Worker: (executes #E1 and #E2 in parallel)

#E1 = Emmanuel Macron, born 1977

#E2 = Frank-Walter Steinmeier, born 1956

Solver:

Given #E1 (Macron, born 1977) and #E2 (Steinmeier, born 1956), Steinmeier is older.

Answer: The president of Germany is older.

10.5.3 Benefits

- **Efficiency:** Parallel execution of actions
- **Fewer LLM calls:** Plan once, execute all, solve once
- **Lower cost:** Reduced API calls compared to ReAct

10.6 HuggingGPT: LLM as Controller

10.6.1 The Vision

HuggingGPT (Shen et al., 2023) uses an LLM to orchestrate specialized AI models.

10.6.2 Architecture

1. **Task Planning:** LLM breaks down user request into subtasks
2. **Model Selection:** LLM chooses appropriate models from Hugging Face for each subtask
3. **Execution:** Run selected models
4. **Response Generation:** LLM synthesizes results into coherent response

Example 10.5. User: "Generate an image of a cat, then describe what's in the image."

Task Planning (LLM):

Task 1: Generate image of cat (text-to-image)

Task 2: Describe generated image (image-captioning)

Model Selection (LLM):

Task 1 → Use Stable Diffusion model

Task 2 → Use BLIP image captioning model

Execution:

Run Stable Diffusion: generates cat image

Run BLIP on generated image: "A gray tabby cat sitting on a windowsill"

Response (LLM):

"I've generated an image of a cat. The image shows a gray tabby cat sitting on a windowsill."

10.6.3 Benefits

- Leverages specialized models for each modality
- LLM handles planning and integration
- Extensible to any model on Hugging Face
- Enables complex multi-modal workflows

10.7 Tool Use and Function Calling

10.7.1 Augmenting LLMs with Tools

Modern LLMs can be given access to tools:

- **Search engines:** Google, Wikipedia
- **Calculators:** For precise arithmetic
- **Code execution:** Python interpreter
- **Databases:** SQL queries
- **APIs:** Weather, maps, calendar, etc.

10.7.2 Function Calling

Many LLM APIs now support **function calling**:

1. Define available functions with JSON schemas
2. LLM decides when and how to call functions
3. LLM generates function call with parameters
4. System executes function
5. Return result to LLM
6. LLM incorporates result into response

Example 10.6. Available Function:

```
{
  "name": "get_weather",
  "description": "Get current weather for a location",
  "parameters": {
    "location": {"type": "string", "description": "City name"},
    "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
  }
}
```

User: "What's the weather like in Paris?"

LLM generates:

```
{  
  "function": "get_weather",  
  "arguments": {"location": "Paris", "unit": "celsius"}  
}
```

System executes: `get_weather("Paris", "celsius")` → "22°C, sunny"

LLM response:

"The weather in Paris is currently 22°C and sunny."

10.8 Challenges in Agentic AI

10.8.1 Reliability

- Agents can make mistakes in reasoning
- Tool use can fail or return unexpected results
- Long action sequences increase chance of errors

10.8.2 Safety

- Agents with tool access can cause real-world effects
- Need safeguards against harmful actions
- Balance autonomy with safety constraints

10.8.3 Evaluation

- Hard to benchmark open-ended agent behavior
- Success depends on environment and goals
- Need metrics beyond accuracy

10.8.4 Cost

- Multiple LLM calls per task
- Tool execution costs
- Iterative refinement increases expense

10.9 Practice Problems: Chapter 10

10.9.1 Problem 1: Agent Planning with Tools

Question: An agent has access to the following tools:

- `Calculator(expression)`: Evaluates math expressions
- `Search(query)`: Searches the web
- `Python(code)`: Executes Python code

Design an agent plan to answer: "What is 15% of the GDP of France in 2023?"

Show: Task decomposition, tool selection, and execution order.

Solution:

Step 1: Task Decomposition

Break down the complex question:

1. Find GDP of France in 2023
2. Calculate 15% of that value

Step 2: Agent Plan

Agent Plan:

Task 1: Get France GDP 2023

Tool: Search("France GDP 2023")

Expected output: Numerical value (e.g., €2.8 trillion)

Task 2: Calculate 15% of GDP

Tool: Calculator("2.8 * 0.15")

OR

Tool: Python("gdp = 2.8; result = gdp * 0.15; print(result)")

Expected output: 0.42 trillion euros

Task 3: Format answer

Action: Synthesize results into natural language

Output: "15% of France's 2023 GDP (€2.8 trillion) is approximately €0.42 trillion or €420 billion."

Step 3: Execution Trace

Thought 1: I need to find France's GDP for 2023.

Action 1: Search("France GDP 2023")

Observation 1: France's GDP in 2023 was approximately €2.8 trillion.

Thought 2: Now I need to calculate 15% of €2.8 trillion.

Action 2: Calculator("2.8 * 0.15")

Observation 2: Result is 0.42

Thought 3: I have both pieces of information. Let me formulate the final answer.

Action 3: Finish["15% of France's 2023 GDP is approximately €0.42 trillion or €420 billion."]

Key Aspects of Agent Planning:

1. Sequential Dependencies

- Task 2 depends on output of Task 1
- Cannot calculate percentage without GDP value
- Agent must execute in order

2. Tool Selection

- Search for factual information (GDP)
- Calculator for numerical computation
- Could use Python for more complex calculations

3. Error Handling

If Search fails to return clear GDP:

Observation 1: "France has a large economy..."
(no specific number)

Thought 2: The search didn't return a specific value.
Let me refine my query.

Action 2: Search("France GDP 2023 in euros exact value")

Answer: Agent successfully decomposes task, selects appropriate tools sequentially, and synthesizes final answer.

10.9.2 Problem 2: Reflexion Self-Improvement

Question: An agent attempts a task and fails. Show how Reflexion enables the agent to learn from this failure.

Task: "Write a Python function to check if a number is prime."

Attempt 1 (Failed):

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Test result: `is_prime(1)` returns `True` (incorrect! 1 is not prime)

Design the Reflexion feedback loop for this failure.

Solution:

Reflexion Cycle:

Step 1: Self-Reflection (Analyze Failure)

Reflection: The function failed the test case `is_prime(1)`.

Analysis:

1. The function returned `True` for `n=1`
2. Expected: `False` (1 is not prime by definition)
3. Root cause: No edge case handling for `n <= 1`
4. The loop `'range(2, 1)'` is empty, so function immediately returns `True`

Key insight: I need to handle edge cases BEFORE the main loop.

Mistakes made:

- Didn't consider edge cases (`n <= 1`)
- Didn't recall that prime numbers must be `> 1`
- Insufficient test coverage in planning phase

Step 2: Generate Improved Plan

Improved Plan:

1. Add edge case handling:
 - If $n \leq 1$: return False
 - If $n == 2$: return True (2 is prime)
2. Main loop remains the same for $n > 2$
3. Consider optimization: only check up to $\sqrt{n}$ instead of $n-1$
4. Test cases to verify:
 - `is_prime(1)` -> False
 - `is_prime(2)` -> True
 - `is_prime(4)` -> False
 - `is_prime(17)` -> True

Step 3: Implement Revised Solution

```
def is_prime(n):
    # Edge cases
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    # Check odd divisors up to sqrt(n)
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True
```

Step 4: Re-Test and Verify

Test results:

```
OK is_prime(1) -> False (correct!)
OK is_prime(2) -> True
OK is_prime(4) -> False
OK is_prime(17) -> True
```

Success! All tests pass.

Step 5: Store Reflection in Memory

Memory Entry:

Task: Prime number checking

Failure: Missed edge case $n \leq 1$

Lesson: Always handle edge cases first:

- Values ≤ 1 are not prime
- 2 is the only even prime

Learning: Start with boundary conditions, then general case

Reflexion Architecture:

1. **Actor:** Generates initial solution (LLM)
2. **Evaluator:** Tests solution, detects failure
3. **Self-Reflection:** Analyzes why it failed (LLM with reflection prompt)
4. **Memory:** Stores reflection for future tasks
5. **Re-Actor:** Generates improved solution using reflection

Key Benefits:

- Learns from failures without parameter updates
- Builds episodic memory of mistakes
- Improves over multiple attempts
- Textual reasoning about errors

Difference from standard retry:

- Standard retry: "Try again" (same approach likely fails)
- Reflexion: Explicitly analyzes failure reason, updates approach

10.9.3 Problem 3: Multi-Agent Collaboration

Question: Design a multi-agent system for code review with the following agents:

- CodeReviewer: Reviews code for correctness
- SecurityChecker: Checks for vulnerabilities
- PerformanceAnalyzer: Suggests optimizations
- Synthesizer: Combines feedback

Show how these agents collaborate on reviewing a function.

Solution:

Input Code to Review:

```
def process_user_input(user_input):
    query = "SELECT * FROM users WHERE name = '"
        + user_input + "'"
    result = db.execute(query)
    return result
```

Multi-Agent Review Process:

Agent 1: CodeReviewer

Role: Check functional correctness and code quality

Review:

1. Logic: Function constructs SQL query from user input
2. Issues:
 - No input validation
 - No error handling for database operations
 - Returns raw database result (should be processed)
3. Correctness: Functionally works but has quality issues

Rating: NEEDS_IMPROVEMENT

Agent 2: SecurityChecker

Role: Identify security vulnerabilities

Review:

[ALERT] CRITICAL SECURITY ISSUE: SQL Injection vulnerability!

Details:

- Line 2: Direct string concatenation of user input into SQL query
- Attack vector: `user_input = "' OR '1'='1"`
 - > Query becomes: `SELECT * FROM users WHERE name = '' OR '1'='1'`
 - > Returns ALL users (security breach!)

Recommendation: Use parameterized queries:

```
query = "SELECT * FROM users WHERE name = ?"
result = db.execute(query, (user_input,))
```

Severity: CRITICAL

Rating: MUST_FIX

Agent 3: PerformanceAnalyzer

Role: Optimize performance

Review:

1. Query uses `SELECT *` (fetches all columns)
 - Optimization: Specify only needed columns
2. No caching mechanism
 - Consider caching frequent queries
3. No limit on results
 - Could return millions of rows
 - Recommendation: Add `LIMIT` clause
4. No indexing mentioned
 - Ensure `'name'` column is indexed

Performance impact: MEDIUM
(Security must be fixed first)

Agent 4: Synthesizer

Role: Combine feedback and generate final report

Combined Review Report:

PRIORITY 1 (CRITICAL - Must fix immediately):
-> SQL Injection vulnerability [SecurityChecker]
Fix: Use parameterized queries

PRIORITY 2 (High - Code quality):

- > Add input validation [CodeReviewer]
- > Add error handling [CodeReviewer]

PRIORITY 3 (Medium - Performance):

- > Specify columns instead of `SELECT *` [PerformanceAnalyzer]
- > Add result limit [PerformanceAnalyzer]

Recommended rewrite:

```

“python
def process_user_input(user_input):
    # Validate input
    if not user_input or len(user_input) > 100:
        raise ValueError("Invalid input")

    try:
        # Parameterized query (SQL injection safe)
        query = """SELECT id, name, email FROM users
                    WHERE name = ? LIMIT 100"""
        result = db.execute(query, (user_input,))
        return result
    except DatabaseError as e:
        log_error(e)
        return None
“

```

Overall rating: REQUIRES_MAJOR_REVISION
Resubmit after fixes.

Collaboration Pattern:

[illegible]

Benefits of Multi-Agent Approach:**1. Specialization**

- Each agent focuses on specific aspect
- Deeper analysis than single general agent
- Expert-level reviews in each domain

2. Comprehensive Coverage

- Security issues caught by SecurityChecker
- Performance issues by PerformanceAnalyzer
- Quality issues by CodeReviewer
- Nothing falls through cracks

3. Prioritization

- Synthesizer combines and prioritizes feedback
- Critical issues (security) flagged first
- Actionable recommendations

4. Parallel Processing

- Agents can run in parallel
- Faster than sequential analysis
- Synthesizer combines results

Answer: Multi-agent system provides comprehensive, prioritized feedback through specialized agents working in parallel.

10.9.4 Problem 4: Tool Use vs Direct Generation

Question: Compare an LLM answering "What is $8,347 \times 2,915$?" using:

- (a) Direct generation (no tools)
- (b) With calculator tool access

Which approach is better and why?

Solution:

Approach (a): Direct Generation

Agent behavior:

Prompt: What is $8,347 \times 2,915$?

LLM Response: "Let me calculate this:
 $8,347 \times 2,915 \approx 24,331,405$ "

(Actual answer: 24,331,805 - ERROR!)

Problems:

1. **Arithmetic errors:** LLMs are not calculators
2. **Token-by-token generation:** Can't do multi-digit arithmetic reliably
3. **No verification:** Can't check its own answer
4. **Inconsistent:** May give different answers on retry

Why LLMs fail at arithmetic:

LLMs learn patterns, not algorithms:

- Training: Sees " $2 \times 3 = 6$ " many times, memorizes
- Novel calculation: " $8,347 \times 2,915$ " never seen in training
- Tries to approximate based on learned patterns
- No carry propagation, no actual multiplication algorithm

Approach (b): With Calculator Tool

Agent behavior:

Prompt: What is $8,347 \times 2,915$?

Thought: This is an arithmetic problem. I should use the calculator tool for accurate results.

Action: Calculator(" $8347 * 2915$ ")

Observation: 24331805

Thought: The calculator returned 24,331,805. This is the accurate answer.

Response: " $8,347 \times 2,915 = 24,331,805$ "

Advantages:

1. **100% accuracy:** Calculator is deterministic
2. **Reliable:** Same answer every time
3. **Scalable:** Works for arbitrarily large numbers
4. **Verifiable:** Can check result with second calculation

Comparison Table:

Aspect	Direct Gen	With Tool
Accuracy	Low ($\sim 20\%$)	High (100%)
Consistency	Variable	Deterministic
Speed	Fast (1 gen)	Slower (2 gens + tool call)
Cost	Low	Medium
Reliability	Low	High
Explainability	Opaque	Transparent

When to use each:

Direct Generation:

- Simple arithmetic ($2+2$, 10×5)
- Approximate calculations ("roughly 10 million")
- When speed matters more than exactness

With Tools:

- Complex calculations
- When accuracy is critical (financial, scientific)
- Multi-step math problems
- Verification required

Answer: (b) With calculator tool is better for accuracy and reliability. Tools extend LLM capabilities beyond their training distribution.

General Principle:

Use tools for tasks that require:

- Deterministic computation (arithmetic, date calculations)
- External knowledge (web search, database queries)
- Real-world interaction (API calls, file operations)
- Guaranteed accuracy (symbolic math, verification)

Use direct generation for:

- Language understanding and generation
- Reasoning and planning
- Pattern recognition
- Creative tasks

10.9.5 Problem 5: Agent Error Recovery

Question: An agent is booking a flight and encounters an error. Design an error recovery strategy using:

- (a) Immediate retry with same parameters
- (b) Reflexion-based debugging
- (c) Alternative tool selection

Show which approach is best for different error types.

Solution:

Scenario: Agent tries to book flight and receives error.

Error Recovery Strategies:

Error Type 1: Transient Network Error

Action: BookFlight(from="NYC", to="LAX", date="2024-05-15")

Observation: Error: Network timeout

Strategy (a): Immediate Retry

Thought: This might be a temporary network issue. Let me retry the same request.

Action: BookFlight(from="NYC", to="LAX", date="2024-05-15")

Observation: Success! Flight booked.

OK BEST CHOICE: Transient errors often resolve with retry

Error Type 2: Invalid Date Format

Action: BookFlight(from="NYC", to="LAX", date="May 15")

Observation: Error: Invalid date format. Expected YYYY-MM-DD.

Strategy (b): Reflexion-Based Debugging

Thought: The API expects YYYY-MM-DD format, but I provided "May 15" without year. Let me fix this.

Reflection: I made a date formatting error. The API requires ISO format. I need to:

1. Convert "May 15" to "2024-05-15"
2. Always use YYYY-MM-DD format for this API

Action: BookFlight(from="NYC", to="LAX", date="2024-05-15")

Observation: Success!

OK BEST CHOICE: Semantic errors need understanding and fixing

Error Type 3: Tool Unavailable

Action: BookFlight(from="NYC", to="LAX", date="2024-05-15")

Observation: Error: BookFlight API is down for maintenance

Strategy (c): Alternative Tool Selection

Thought: BookFlight is unavailable. Let me check if I have alternative tools.

Available alternatives:

1. ManualBooking(): Use fallback booking system
2. QueueRequest(): Save booking for later
3. NotifyUser(): Inform user to book manually

Action: ManualBooking(from="NYC", to="LAX", date="2024-05-15")

Observation: Booking created via backup system.

OK BEST CHOICE: When primary tool fails, use fallback

Decision Tree for Error Recovery:

```

Error Detected
|
v
Is error transient? (timeout, rate limit)
|
+-- YES --> Immediate Retry (Strategy a)
|           - Add exponential backoff
|           - Max 3 retries
|
+-- NO --> Is error semantic? (bad parameters)
|
|   +-- YES --> Reflexion (Strategy b)
|   |           - Analyze error message
|   |           - Fix parameters
|   |           - Retry with correction
|   |
|   +-- NO --> Is tool unavailable?
|   |
|   |   +-- YES --> Alternative Tool (Strategy c)
|   |   |           - Find fallback
|   |   |           - Use backup method
|   |   |
|   |   +-- NO --> Escalate to user

```

Implementation Example:

```

def agent_with_error_recovery(action):
    max_retries = 3
    attempt = 0

    while attempt < max_retries:
        try:
            result = execute(action)
            return result

        except TransientError as e:
            # Strategy (a): Immediate retry
            attempt += 1
            wait_time = 2 ** attempt # exponential backoff
            sleep(wait_time)
            continue

        except ParameterError as e:
            # Strategy (b): Reflexion
            reflection = analyze_error(e)
            corrected_action = fix_parameters(action, reflection)
            action = corrected_action
            attempt += 1
            continue

        except ToolUnavailableError as e:
            # Strategy (c): Alternative tool

```



```
        fallback_tool = find_alternative(action.tool)
    if fallback_tool:
        action.tool = fallback_tool
        attempt += 1
        continue
    else:
        raise Exception("No fallback available")

raise Exception("Max retries exceeded")
```

Best Practices:**1. Error Classification**

- Categorize errors by type
- Different strategies for different errors
- Log errors for analysis

2. Graceful Degradation

- Have fallback options
- Partial success > complete failure
- Inform user of limitations

3. Learning from Errors

- Store error patterns
- Update tool usage guidelines
- Improve parameter validation

4. User Communication

- Explain what went wrong
- Show recovery attempts
- Ask for help when stuck

Answer: Error recovery strategy depends on error type:

- Transient → Retry with backoff
- Semantic → Reflect and fix
- Tool unavailable → Use alternative
- Unknown → Escalate to user

10.10 Summary of Chapter 10

In this chapter, we learned:

- Agentic AI goes beyond text generation to interact with environments
- ReAct: Interleaves reasoning and acting with external tools
- Reflexion: Learns from mistakes through textual self-reflection
- Self-Refine: Iteratively critiques and improves own outputs
- ReWOO: Separates planning from execution for efficiency
- HuggingGPT: LLM orchestrates specialized models
- Tool use and function calling: Augment LLMs with external capabilities
- Challenges: Reliability, safety, evaluation, cost

Conclusion

We've covered the full spectrum of Deep Learning and Generative AI:

- **Foundations:** Neural networks, backpropagation, optimization
- **Computer Vision:** CNNs and architectures from AlexNet to ResNet
- **Sequence Modeling:** RNNs, LSTMs, attention mechanisms
- **Transformers:** The architecture powering modern AI
- **Large Language Models:** BERT, GPT, T5, and beyond
- **Practical Usage:** Prompting techniques and instruction tuning
- **Alignment:** RLHF and Constitutional AI
- **Agentic Systems:** From passive models to active agents

This field is evolving rapidly. The principles you've learned here will serve as a foundation for understanding future developments!

Appendix A

Quick Reference

A.1 Key Formulas

A.1.1 Neural Networks

Perceptron:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

ReLU:

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Gradient Descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

A.1.2 CNNs

Convolution Output Size:

$$\text{output} = \left\lfloor \frac{\text{input} - \text{kernel} + 2 \times \text{padding}}{\text{stride}} \right\rfloor + 1$$

A.1.3 RNNs

Vanilla RNN:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

LSTM:

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ h_t &= o_t \odot \tanh(c_t) \end{aligned}$$

A.1.4 Attention and Transformers

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Positional Encoding:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d}} \right)$$

A.1.5 RLHF

Reward Model Loss:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E} [\log \sigma (r_\phi(x, y_w) - r_\phi(x, y_l))]$$

RL Objective with KL Penalty:

$$\mathcal{L} = \mathbb{E} \left[r_\phi(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

A.2 Architecture Comparison

Model	Type	Key Feature	Best For
BERT	Encoder-only	Bidirectional	Classification, QA
GPT	Decoder-only	Autoregressive	Text generation
T5	Encoder-Decoder	Text-to-text	All NLP tasks
BART	Encoder-Decoder	Denoising	Generation + understanding

Table A.1: Comparison of major LLM architectures

A.3 Decoding Strategies

Strategy	Pros	Cons
Greedy	Fast, deterministic	Repetitive, misses better sequences
Beam Search	Better quality	Expensive, less diverse
Temperature	Controllable randomness	Needs tuning
Top-k	Simple diversity	Fixed k may not fit all contexts
Top-p (Nucleus)	Adaptive diversity	Can include low-quality tokens

Table A.2: Decoding strategies comparison

A.4 Study Tips for Exam

1. **Understand, don't memorize:** Focus on intuition and "why" things work
2. **Work through examples:** Practice the numerical examples in this guide
3. **Draw diagrams:** Visualize architectures and information flow

4. **Connect concepts:** Understand how topics relate (e.g., attention $\rightarrow$ transformers $\rightarrow$ LLMs)
5. **Practice explaining:** Teach concepts to yourself or others
6. **Review formulas:** Understand what each term means, not just the equation
7. **Focus on trends:** Evolution of architectures, scaling laws, alignment

Good luck with your exam!

Remember: Deep learning is as much about intuition as it is about mathematics.
Understanding the "why" will serve you far better than memorizing the "what."